

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng Tác tử AI nội bộ hỗ trợ kiểm thử xâm nhập
sử dụng RAG và MCP với Bộ nhớ Dài hạn
(Long-Term Memory)**

NGUYỄN ĐÌNH PHÚC

phuc.nd215629@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Nguyễn Quốc Khánh

TS. Trần Hải Anh

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 01/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Tác tử AI nội bộ hỗ trợ kiểm thử xâm nhập
sử dụng RAG và MCP với Bộ nhớ Dài hạn
(Long-Term Memory)

NGUYỄN ĐÌNH PHÚC

phuc.nd215629@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: ThS. Nguyễn Quốc Khánh

TS. Trần Hải Anh

Chữ kí GVHD

Khoa: Kỹ thuật máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 01/2026

LỜI CẢM ƠN

Để hoàn thành đồ án tốt nghiệp này, trước hết em xin bày tỏ lòng biết ơn sâu sắc tới thầy Nguyễn Quốc Khánh và thầy Trần Hải Anh vì đã tận tình hướng dẫn, định hướng chuyên môn và luôn dành những góp ý quý báu trong suốt quá trình thực hiện đồ án. Em xin chân thành cảm ơn bố mẹ và gia đình đã luôn là hậu phương vững chắc, quan tâm, động viên và ủng hộ em vô điều kiện trong suốt chặng đường học tập và nghiên cứu. Em cũng xin gửi lời cảm ơn tới các anh chị, đồng nghiệp và bạn bè đã luôn sẵn sàng trao đổi chuyên môn, chia sẻ kinh nghiệm thực tế, cũng như tạo động lực tinh thần trong những giai đoạn khó khăn của đồ án. Đặc biệt, em xin gửi lời cảm ơn chân thành tới anh Huy - đàn anh của em tại công ty CyStack, người đã tạo điều kiện cho em mượn và sử dụng tài nguyên GPU để triển khai hệ thống mô hình ngôn ngữ lớn, góp phần quan trọng vào việc hoàn thiện đồ án. Cuối cùng, em muốn dành lời cảm ơn cho chính bản thân mình - cảm ơn vì đã không bỏ cuộc trước những bài toán phức tạp, đã giữ vững kỷ luật, kiên trì theo đuổi mục tiêu và nỗ lực hết mình trong từng dòng mã nguồn, để có thể khép lại quãng đời sinh viên bằng một kết quả trọn vẹn và ý nghĩa.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án tập trung xây dựng một trợ lý kiểm thử xâm nhập chạy ngay trên máy cá nhân (LPAA - Local Pentest AI Agent). Mục tiêu là vừa bảo mật dữ liệu, vừa hỗ trợ các thao tác kỹ thuật một cách chính xác. Hiện nay, các giải pháp kiểu single-agent - “một agent làm tất cả” thường gặp khó khăn. Hướng tiếp cận này tuy triển khai nhanh nhưng thường gặp vấn đề về khả năng quản lý ngữ cảnh, tính kiểm soát đối với các hành vi gọi công cụ và khả năng mở rộng trong tương lai. Ngoài ra, cách tìm kiếm văn bản thông thường hay bỏ sót các thông tin kỹ thuật quan trọng. Bộ nhớ lịch sử đơn giản cũng không đủ để ghi nhớ quá trình pentest kéo dài. Để giải quyết các vấn đề này, đồ án áp dụng kiến trúc đa tác tử (multi-agent). Hệ thống chia việc cho các agent chuyên biệt: RAG Agent đóng vai trò tìm kiếm và truy xuất tri thức, Pentest Agent giúp tự động hóa quá trình thực thi công cụ và Memory Agent giúp duy trì bộ nhớ dài hạn. Về khả năng truy xuất, đồ án dùng kỹ thuật Hybrid Search kết hợp Reranking để cho ra tập ứng viên đầy đủ nhất. Về bộ nhớ, hệ thống áp dụng kiến trúc MemGPT cho khả năng lưu trữ thông tin quan trọng và truy xuất thông tin khi cần. Về mặt thực thi công cụ, hệ thống sử dụng giao thức MCP chạy trong môi trường Docker sandbox để đảm bảo sự cách ly và an toàn trong môi trường pentest. Kết quả là đồ án đã tạo ra một hệ thống có kiến trúc rõ ràng và vận hành tin cậy. Hệ thống không chỉ giúp pentester tra cứu chính xác, thực thi công cụ có kiểm soát mà còn duy trì được dữ liệu xuyên suốt các phiên làm việc kéo dài.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

The thesis focuses on developing a Local Pentest AI Agent (LPAA), a penetration testing assistant designed to run entirely on a local machine. The primary objective is to ensure data privacy while providing precise support for technical operations. Currently, single-agent solutions - where “one agent does it all” - face significant challenges. They are prone to noise when processing large datasets, often repeat actions, and are difficult to control. Furthermore, standard text-based search methods frequently miss critical technical details, and simple conversational history memory is insufficient for retaining context throughout prolonged penetration testing processes. To address these issues, the thesis adopts a multi-agent architecture. The system delegates tasks to specialized agents: the RAG Agent handles knowledge retrieval, the Pentest Agent automates tool execution, and the Memory Agent maintains long-term memory. Regarding retrieval capabilities, the project employs Hybrid Search combined with Reranking to generate the most comprehensive set of candidates. Regarding memory, the system implements the MemGPT architecture to persistently store critical facts and retrieve information as needed. Regarding execution, the system utilizes the Model Context Protocol (MCP) within a Docker sandbox environment to ensure isolation and safety. As a result, the thesis has created a system with a well-defined architecture and reliable operation. The system not only assists pentesters with accurate information lookup and controlled tool execution but also maintains data consistency across extended sessions.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	6
2.1 Khảo sát hiện trạng	6
2.1.1 Khảo sát nhu cầu người dùng	6
2.1.2 Khảo sát các hệ thống hiện có	7
2.2 Tổng quan chức năng	14
2.2.1 Biểu đồ ca sử dụng tổng quát	14
2.2.2 Biểu đồ ca sử dụng phân rã Tiếp nhận yêu cầu	16
2.2.3 Biểu đồ ca sử dụng phân rã Nạp dữ liệu vào kho tri thức.....	17
2.2.4 Biểu đồ ca sử dụng phân rã Truy xuất dữ liệu từ kho tri thức	18
2.2.5 Biểu đồ ca sử dụng phân rã Nạp thông tin vào kho trí nhớ.....	19
2.2.6 Biểu đồ ca sử dụng phân rã Truy xuất thông tin từ kho trí nhớ.....	19
2.2.7 Biểu đồ ca sử dụng phân rã Điều phối công cụ pentest.....	20
2.2.8 Biểu đồ ca sử dụng phân rã Thực thi công cụ pentest.....	21
2.2.9 Biểu đồ ca sử dụng phân rã Ghi log và lưu trữ dữ liệu tương tác	22
2.2.10 Quy trình nghiệp vụ.....	23
2.3 Đặc tả chức năng	25
2.3.1 Đặc tả ca sử dụng Tiếp nhận yêu cầu.....	25
2.3.2 Đặc tả ca sử dụng Nạp dữ liệu vào kho tri thức	26
2.3.3 Đặc tả ca sử dụng Nạp thông tin vào kho trí nhớ	27

2.3.4 Đặc tả ca sử dụng Truy xuất dữ liệu từ kho tri thức.....	28
2.3.5 Đặc tả ca sử dụng Truy xuất thông tin từ kho trí nhớ	29
2.3.6 Đặc tả ca sử dụng Điều phối công cụ pentest	30
2.3.7 Đặc tả ca sử dụng Thực thi công cụ pentest	31
2.4 Yêu cầu phi chức năng	32
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	34
3.1 LangChain - Khung xây dựng tác tử và luồng xử lý ngôn ngữ	34
3.1.1 Khái quát công nghệ.....	34
3.1.2 Vai trò trong hệ thống.....	36
3.1.3 Lí do lựa chọn công nghệ.....	36
3.2 LangGraph - Kiến trúc điều phối agent theo đồ thị.....	36
3.2.1 Khái quát công nghệ.....	36
3.2.2 Vai trò trong hệ thống.....	38
3.2.3 Lí do lựa chọn công nghệ.....	38
3.3 Ollama - Nền tảng triển khai và vận hành LLM cục bộ.....	38
3.3.1 Khái quát công nghệ.....	38
3.3.2 Vai trò trong hệ thống.....	40
3.3.3 Lí do lựa chọn công nghệ.....	40
3.4 RAG - Retrieval Augmented Generation	40
3.4.1 Khái quát kỹ thuật.....	40
3.4.2 Vai trò trong hệ thống.....	43
3.4.3 Lí do lựa chọn công nghệ.....	43
3.5 Docling kết hợp Semantic Chunking - Chuyển đổi và phân đoạn tài liệu	43
3.5.1 Khái quát kỹ thuật.....	43
3.5.2 Vai trò trong hệ thống.....	44
3.5.3 Lí do lựa chọn công nghệ.....	45

3.6 Qdrant và FastEmbed - Hạ tầng truy xuất tri thức dựa trên véc-tơ.....	46
3.6.1 Khái quát công nghệ.....	46
3.6.2 Vai trò trong hệ thống.....	47
3.6.3 Lí do lựa chọn công nghệ.....	47
3.7 Kiến trúc bộ nhớ dài hạn theo hướng MemGPT.....	48
3.7.1 Khái quát hướng tiếp cận	48
3.7.2 Vai trò trong hệ thống.....	49
3.7.3 Lí do lựa chọn công nghệ.....	49
3.8 Model Context Protocol - Giao thức ngữ cảnh mô hình	50
3.8.1 Khái quát công nghệ.....	50
3.8.2 Vai trò trong hệ thống.....	52
3.8.3 Lí do lựa chọn công nghệ.....	52
3.9 Containerization & Docker - Đóng gói hệ thống & sandbox thực thi công cụ	52
3.9.1 Khái quát công nghệ.....	52
3.9.2 Vai trò trong hệ thống.....	53
3.9.3 Lí do lựa chọn công nghệ.....	54
CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	55
4.1 Lựa chọn kiến trúc hệ thống.....	55
4.1.1 Khái niệm kiến trúc multi-agent theo kiểu subagents.....	55
4.1.2 Lí do lựa chọn kiến trúc multi-agent theo kiểu subagents	56
4.2 Kiến trúc tổng quan của hệ thống	57
4.2.1 Các thành phần chính	57
4.2.2 Luồng xử lý tổng quát.....	59
4.3 Thiết kế agent.....	60
4.3.1 Danh mục các agent	60

4.3.2 Chi tiết các hàm gọi Agent	61
4.3.3 Thiết kế System Prompt cho Core Orchestrator Agent	64
4.3.4 Thiết kế System Prompt cho RAG Agent	65
4.3.5 Thiết kế System Prompt cho Memory Agent	66
4.3.6 Thiết kế System Prompt cho Pentest Agent	66
4.4 Thiết kế cơ sở dữ liệu	68
4.4.1 Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)	68
4.4.2 Thiết kế cơ sở dữ liệu véc-tơ (Qdrant)	70
4.5 Xây dựng ứng dụng	71
4.5.1 Thư viện và công cụ sử dụng	71
4.5.2 Kết quả đạt được	72
4.5.3 Minh họa các chức năng chính	73
4.6 Kiểm thử	76
4.6.1 Chiến lược và kỹ thuật kiểm thử	77
4.6.2 Các trường hợp kiểm thử đã thực hiện	77
4.6.3 Kiểm thử quy trình pentest của toàn bộ hệ thống multi-agent	82
4.6.4 Tổng hợp kết quả kiểm thử	83
4.7 Triển khai	84
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	86
5.1 Đánh giá và lựa chọn Open-Source Embedding Models phục vụ cho hệ thống RAG	86
5.1.1 Đặt vấn đề	86
5.1.2 Giải pháp đề xuất	86
5.1.3 Kết quả đạt được	88
5.2 Hybrid Search kết hợp Reranking nhằm cải thiện truy xuất trong RAG	89
5.2.1 Đặt vấn đề	89

5.2.2 Giải pháp đề xuất	90
5.2.3 Kết quả đạt được	92
5.3 Kiến trúc bộ nhớ dài hạn theo hướng MemGPT	93
5.3.1 Đặt vấn đề	93
5.3.2 Giải pháp đề xuất	93
5.3.3 Kết quả đạt được	96
5.4 Xây dựng Kali MCP Server hỗ trợ phương thức MCP HTTP streamable....	97
5.4.1 Đặt vấn đề	97
5.4.2 Giải pháp đề xuất	97
5.4.3 Kết quả đạt được	98
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	99
6.1 Kết luận	99
6.2 Hướng phát triển.....	100
TÀI LIỆU THAM KHẢO.....	103

DANH MỤC HÌNH VẼ

Hình 2.1	Sơ đồ kiến trúc PentestAgent	8
Hình 2.2	Sơ đồ kiến trúc của HexStrike AI MCP Agents v6.0	10
Hình 2.3	Sơ đồ kiến trúc của Strix	12
Hình 2.4	Biểu đồ ca sử dụng tổng quát	16
Hình 2.5	Biểu đồ ca sử dụng Tiếp nhận yêu cầu	17
Hình 2.6	Biểu đồ ca sử dụng Nạp dữ liệu vào kho tri thức	18
Hình 2.7	Biểu đồ ca sử dụng Truy xuất dữ liệu từ kho tri thức	18
Hình 2.8	Biểu đồ ca sử dụng Nạp thông tin vào kho trí nhớ	19
Hình 2.9	Biểu đồ ca sử dụng Truy xuất thông tin từ kho trí nhớ	20
Hình 2.10	Biểu đồ ca sử dụng Điều phối công cụ pentest	21
Hình 2.11	Biểu đồ ca sử dụng Thực thi công cụ pentest	22
Hình 2.12	Biểu đồ ca sử dụng Ghi log và lưu trữ dữ liệu tương tác	22
Hình 2.13	Biểu đồ hoạt động Quy trình nghiệp vụ của hệ thống LPAA	23
Hình 3.1	Mô tả kiến trúc tổng quát của Ollama	39
Hình 3.2	Mô tả quy trình RAG	41
Hình 3.3	Kiến trúc tổng quát của Qdrant [16]	46
Hình 4.1	Kiến trúc hệ thống LPAA	57
Hình 4.2	Luồng xử lý của Conversation Graph	59
Hình 4.3	Biểu đồ trình tự cho quá trình lưu/cập nhật core memory	62
Hình 4.4	Biểu đồ trình tự cho quá trình Truy xuất dữ liệu từ kho tri thức	63
Hình 4.5	Biểu đồ trình tự quá trình Điều phối và Thực thi công cụ pentest	64
Hình 4.6	Sơ đồ thiết kế cơ sở dữ liệu	68
Hình 4.7	Minh họa kết quả nạp dữ liệu cho Kho tri thức của Agent theo phiên làm việc	73
Hình 4.8	Minh họa kết quả truy xuất RAG	74
Hình 4.9	Minh họa giao diện quản lý core memory	75
Hình 4.10	Minh họa giao diện hiển thị lời gọi công cụ theo thời gian thực	75
Hình 4.11	Minh họa giao diện hiển thị lời gọi công cụ	76
Hình 4.12	Kết quả kiểm thử khả năng điều phối của Core Orchestrator	78
Hình 4.13	Kết quả kiểm thử khả năng ưu tiên kết quả thực thi so với ký ức cũ	79
Hình 4.14	Kết quả kiểm thử khả năng lưu trữ và cập nhật bộ nhớ của Memory Agent	80

Hình 4.15	Kết quả kiểm thử khả năng truy xuất và tổng hợp tri thức của RAG Agent	81
Hình 4.16	Kết quả kiểm thử khả năng lựa chọn và thực thi công cụ của Pentest Agent	82
Hình 4.17	Kết quả kiểm thử quy trình pentest của toàn bộ hệ thống multi-agent	83
Hình 5.1	So sánh hiệu suất truy xuất của các mô hình nhúng	88
Hình 5.2	Chiến thuật RAG sử dụng Hybrid Search kết hợp Reranking	92
Hình 5.3	Chiến lược duy trì Long-term Memory của MemGPT [29]	94
Hình 5.4	Biểu đồ trình tự cho quá trình Nạp trí nhớ và xử lý hàng đợi thông điệp	95

DANH MỤC BẢNG BIỂU

Bảng 2.1	Bảng đặc tả ca sử dụng Tiếp nhận yêu cầu	25
Bảng 2.2	Bảng đặc tả ca sử dụng Nạp dữ liệu vào kho tri thức	26
Bảng 2.3	Bảng đặc tả ca sử dụng Nạp thông tin vào kho trí nhớ	27
Bảng 2.4	Bảng đặc tả ca sử dụng Truy xuất dữ liệu từ kho tri thức	28
Bảng 2.5	Bảng đặc tả ca sử dụng Truy xuất thông tin từ kho trí nhớ . . .	29
Bảng 2.6	Bảng đặc tả ca sử dụng Điều phối công cụ pentest	30
Bảng 2.7	Bảng đặc tả ca sử dụng Thực thi công cụ pentest	31
Bảng 4.1	Danh sách thư viện và công cụ sử dụng	71
Bảng 4.2	Thống kê khối lượng mã nguồn của hệ thống	72
Bảng 4.3	Thống kê các container Docker của hệ thống	73
Bảng 5.1	Các mô hình nhúng mã nguồn mở được khảo sát	87
Bảng 5.2	Các loại câu hỏi và trọng tâm đánh giá	88

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
LPA	Local Pentest AI Agent	Tác tử AI nội bộ hỗ trợ kiểm thử xâm nhập
RAG	Retrieval-Augmented Generation	Kỹ thuật nâng cao khả năng của mô hình sinh bằng cách kết hợp với tri thức bên ngoài
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MCP	Model Context Protocol	Giao thức Ngữ cảnh Mô hình
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
AI	Artificial Intelligence	Trí tuệ Nhân tạo
Pentest	Penetration Testing	Kiểm thử Xâm nhập
PoC	Prove of Concept	Bằng chứng Khái niệm

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích
Agent	Tác tử
Agent-loop	Kiến trúc vòng lặp tác tử
Benchmark	Đánh giá hiệu năng
Client-server	Mô hình khách-chủ
Context window	Cửa sổ ngữ cảnh
Core memory	Trí nhớ lõi
Embedding model	Mô hình nhúng
Endpoint	Điểm giao tiếp giữa client và hệ thống
Framework	Bộ các đoạn code và thư viện đóng gói sẵn
In-memory	Lưu trữ dữ liệu trong bộ nhớ
Intelligent Decision Engine	Bộ máy quyết định thông minh
Message queue	Hàng đợi thông điệp
Metadata	Siêu dữ liệu
Multi-agent	Kiến trúc đa tác tử
Offline-first	Ưu tiên ngoại tuyến
Payload	Đoạn mã dữ liệu
Prompt	Câu lệnh trong hệ thống trí tuệ nhân tạo
Prompt Injection	Tấn công chèn mã vào hệ thống AI
Quantization	Kỹ thuật lượng tử hóa
Rate limit	Cơ chế hạn chế truy cập
Recall memory	Trí nhớ hồi tưởng
Sandbox	Môi trường cách ly
Semantic chunking	Phân mảnh dữ liệu dựa trên ngữ nghĩa
Streaming	Truyền theo luồng
System Prompt	Chỉ thị hệ thống
Timeout	Hết thời gian chờ
Vector embedding	Véc-tơ nhúng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương này sẽ trình bày cái nhìn tổng quan về bối cảnh thực tế và tính cấp thiết của vấn đề, qua đó làm rõ lý do hình thành nên đề tài. Nội dung chương tập trung xác định các mục tiêu cốt lõi và phạm vi nghiên cứu cụ thể, đồng thời đề xuất các giải pháp công nghệ chủ đạo sẽ được áp dụng để giải quyết bài toán đặt ra.

1.1 Đặt vấn đề

Trong lĩnh vực an toàn thông tin nói chung và kiểm thử xâm nhập (penetration testing - pentest) nói riêng, các nhân viên kiểm thử xâm nhập (pentester) thường xuyên phải làm việc với khối lượng lớn các tài liệu kỹ thuật cũng như các thông tin nhạy cảm. Các tài liệu này bao gồm sơ đồ hệ thống, kiến trúc mạng, đặc tả dịch vụ hay log bảo mật. Việc tra cứu thủ công trong lượng dữ liệu khổng lồ này đòi hỏi sự chính xác cao và tốn kém nhiều thời gian, đặc biệt là khi các thông tin cần thiết thường nằm rải rác và thiếu sự liên kết với nhau.

Một thách thức lớn khác trong thực tế triển khai là yêu cầu làm việc trong các môi trường không truy cập Internet, nơi tính bí mật của dữ liệu được đặt lên hàng đầu. Trong bối cảnh này, việc quản lý và khai thác tri thức từ kho dữ liệu lưu trữ cục bộ trở thành một bài toán khó, cả về tốc độ tìm kiếm lẫn tính chính xác của truy xuất. Bên cạnh đó, sự đa dạng và phức tạp của các công cụ pentest hiện đại cũng là một rào cản đáng kể. Việc các pentester phải ghi nhớ chính xác cú pháp, tham số và định dạng đầu ra của hàng loạt công cụ trong điều kiện thời gian hạn chế, lại thiếu sự hỗ trợ từ Internet để tra cứu, khiến cho hiệu suất làm việc suy giảm, trong khi rủi ro về bảo mật lại gia tăng.

Thực tế trong quá trình làm việc với vai trò là một kiểm thử viên, tôi từng tham gia pentest cho một hệ thống nội bộ của doanh nghiệp, trong đó phát hiện một máy chủ web sử dụng Apache phiên bản cũ. Để đánh giá mức độ rủi ro, tôi cần xác định nhanh phiên bản này có tồn tại lỗ hổng đã biết hay không, các điều kiện khai thác tương ứng và cú pháp chính xác để cấu hình công cụ như Nmap hay Metasploit. Tuy nhiên, trong môi trường offline, việc tra cứu thủ công tài liệu CVE, hướng dẫn khai thác hay tham số công cụ thường mất nhiều thời gian và dễ bỏ sót thông tin quan trọng. Cách thức tra cứu này làm chậm quá trình ra quyết định và ảnh hưởng đến hiệu quả pentest, đặc biệt khi phải đồng thời đối chiếu với sơ đồ mạng, cấu hình dịch vụ và các kết quả quét trong quá khứ.

Những hạn chế nêu trên cho thấy nhu cầu cấp thiết trong việc tối ưu hóa khả năng khai thác tri thức và hỗ trợ ra quyết định cho pentester trong môi trường nội

bộ. Việc xây dựng một hệ thống thông minh có khả năng lưu trữ, tìm kiếm, gợi ý thông tin, đồng thời hỗ trợ thao tác công cụ và tổng hợp kết quả sẽ giúp chuẩn hóa quy trình đánh giá an ninh và tiết kiệm đáng kể nguồn lực con người.

Kết quả nghiên cứu và xây dựng sản phẩm của đồ án không chỉ mang lại giá trị trực tiếp cho lĩnh vực an ninh thông tin mà còn mở ra tiềm năng ứng dụng thực tiễn rộng rãi. Mô hình giải pháp này hoàn toàn có thể được triển khai cho các lĩnh vực khác có yêu cầu khắt khe về bảo mật và năng lực xử lý thông tin ngoại tuyến, điển hình như tài chính - ngân hàng, y tế hay tại các bộ phận yêu cầu tính bảo mật thông tin cao trong doanh nghiệp.

1.2 Mục tiêu và phạm vi đề tài

Sự phát triển mạnh mẽ của các ứng dụng hỗ trợ mô hình ngôn ngữ lớn (Large Language Model - LLM) như Claude Desktop, LM Studio hay PentestGPT đã mở ra nhiều hướng đi mới trong việc tương tác và phân tích dữ liệu kỹ thuật. Claude Desktop gây ấn tượng với khả năng xử lý ngữ cảnh chuyên sâu nhờ hạ tầng điện toán đám mây, trong khi LM Studio mang lại sự linh hoạt thông qua việc vận hành các mô hình mã nguồn mở ngay trên thiết bị cá nhân, bước đầu đáp ứng nhu cầu bảo mật cơ bản. Bên cạnh đó, PentestGPT đại diện cho hướng tiếp cận chuyên biệt với các mô-đun được thiết kế riêng cho quy trình pentest, hỗ trợ người dùng từ việc sinh lệnh đến phân tích kết quả theo một cấu trúc định sẵn.

Tuy nhiên, khi đặt trong bối cảnh kiểm thử xâm nhập nội bộ với những yêu cầu khắt khe về bảo mật và tính vận hành offline (ngoại tuyến), các công cụ kể trên đều bộc lộ những điểm yếu đáng kể về mặt kiến trúc và an toàn thông tin. Đối với Claude Desktop, do phụ thuộc hoàn toàn vào kết nối Internet và năng lực xử lý của các LLM chạy trên cloud, nguy cơ lộ lọt thông tin mật của dự án là điều không thể tránh khỏi. Ngược lại, LM Studio đã bước đầu giải quyết được vấn đề này khi mang khả năng tính toán của LLM về máy cá nhân. Tuy nhiên, thách thức lớn đối với nền tảng chạy LLM nội bộ này đó là ở cơ chế quản lý bộ nhớ hội thoại chưa tối ưu. Khi làm việc trong các phiên kiểm thử xâm nhập kéo dài, nền tảng này thường xuyên gặp phải hiện tượng tràn ngữ cảnh và suy giảm hiệu năng trên phần cứng giới hạn. Trong khi đó, PentestGPT dù có cấu trúc hoạt động tốt nhưng thiếu khả năng truy xuất tri thức nội bộ, không hỗ trợ bộ nhớ dài hạn khiến giải pháp này chưa đáp ứng được nhu cầu của các pentester về một công cụ có khả năng hiểu và duy trì ngữ cảnh xuyên suốt quá trình kiểm thử.

Những bất cập về tính bảo mật và khả năng vận hành nêu trên đặt ra yêu cầu cấp thiết về một giải pháp chuyên biệt, hoạt động hoàn toàn cục bộ, đồng thời có khả năng khai thác tri thức kỹ thuật và tương tác mượt mà với các công cụ chuyên dụng.

Xuất phát từ nhu cầu thực tiễn đó, đề tài xác định mục tiêu trọng tâm là nghiên cứu và xây dựng một hệ thống tác tử AI cục bộ dành riêng cho lĩnh vực kiểm thử xâm nhập (Local Pentest AI Agent - LPAA). Hệ thống này được thiết kế để hiểu và xử lý ngôn ngữ tự nhiên, truy xuất nhanh chóng nguồn tri thức nội bộ, duy trì bộ nhớ dài hạn xuyên suốt quá trình kiểm thử và có khả năng điều phối tự động các công cụ pentest trong điều kiện không có kết nối Internet.

Phạm vi nghiên cứu của đề tài tập trung vào việc thiết kế kiến trúc và hiện thực hóa một nguyên mẫu hệ thống đa tác tử (multi-agent). Hệ thống này bao gồm các thành phần cốt lõi như tác tử trung tâm làm công việc điều phối tác vụ (Core Orchestrator), tác tử quản lý bộ nhớ (Memory Agent) theo mô hình MemGPT, tác tử truy xuất tri thức (RAG Agent) áp dụng kỹ thuật hybrid Search với reranking 5.3 và tác tử thực thi công cụ pentest (Pentest Agent) thông qua giao thức ngữ cảnh mô hình (Model Context Protocol - MCP). Hướng tiếp cận của đề tài không tập trung vào việc huấn luyện lại mô hình ngôn ngữ lớn, mà chú trọng vào việc tích hợp, điều phối và tùy biến các công nghệ tiên tiến sẵn có để kiến tạo một giải pháp tổng thể, hoàn chỉnh và đáp ứng được các tiêu chuẩn khắt khe của quy trình pentest nội bộ.

1.3 Định hướng giải pháp

Định hướng giải pháp của đề tài được xây dựng dựa trên sự tích hợp các công nghệ mã nguồn mở tiên tiến, hướng tới việc hình thành một mô hình AI agent toàn diện, hỗ trợ cho công việc kiểm thử xâm nhập trong môi trường nội bộ. Cụ thể, đồ án sử dụng nền tảng Ollama để vận hành các mô hình ngôn ngữ lớn ngay trên máy trạm cục bộ. Giải pháp này loại bỏ hoàn toàn sự phụ thuộc vào Internet, đồng thời đảm bảo tính riêng tư và an toàn dữ liệu cho các hoạt động kiểm thử xâm nhập. Bên cạnh đó, đồ án sử dụng LangGraph để điều phối các luồng công việc. Công cụ này đóng vai trò trung tâm giúp kiểm soát chặt chẽ từng bước xử lý. Nó cũng cho phép tùy chỉnh linh hoạt để phù hợp với từng loại yêu cầu cụ thể của pentester.

Để giải quyết bài toán giới hạn ngữ cảnh thường gặp ở các mô hình ngôn ngữ lớn, đề tài ứng dụng kiến trúc MemGPT làm cơ sở quản lý bộ nhớ dài hạn (long-term memory) cho agent. Thay vì lưu trữ toàn bộ lịch sử hội thoại một cách tuyến tính, dễ gây quá tải bộ nhớ, kiến trúc này thực hiện phân tách dữ liệu thành vùng trí nhớ lõi (core memory) và vùng trí nhớ hồi tưởng (recall memory), cho phép agent ghi nhớ một cách bền vững các thông tin quan trọng mà vẫn đảm bảo hiệu năng xử lý mượt mà. Song song với đó, mô-đun tìm kiếm tăng cường RAG (Retrieval-Augmented Generation) được tích hợp theo cơ chế hybrid search kết hợp với reranking, nhằm nâng cao độ chính xác khi truy xuất tri thức từ kho dữ liệu kỹ thuật chuyên ngành.

Về khả năng tương tác với hệ thống bên ngoài, giải pháp tận dụng giao thức

MCP với cơ chế HTTP streamable để thiết lập kênh kết nối ổn định giữa AI agent và các công cụ pentest chuyên dụng. Việc áp dụng giao thức này cho phép agent gửi lệnh thực thi, giám sát tiến trình và tiếp nhận kết quả dưới dạng luồng dữ liệu liên tục theo thời gian thực. Cơ chế này không chỉ đảm bảo tính ổn định trong quá trình truyền tải thông tin mà còn tạo điều kiện thuận lợi cho việc mở rộng hệ thống và tích hợp thêm các công cụ mới trong tương lai.

Thông qua các giải pháp công nghệ được đề xuất, đề tài mong muốn xây dựng một mô hình ứng dụng thực tế cho việc triển khai mô hình ngôn ngữ lớn chạy cục bộ trong lĩnh vực kiểm thử xâm nhập. Hệ thống hướng tới việc hỗ trợ trực tiếp cho các pentester trong quá trình tra cứu thông tin và thao tác công cụ, đồng thời tạo tiền đề thuận lợi cho việc nghiên cứu và phát triển các AI agent chuyên biệt cho lĩnh vực này trong tương lai.

1.4 Bố cục đồ án

Cấu trúc phần còn lại của đồ án được tổ chức theo trình tự logic từ khảo sát yêu cầu, xây dựng cơ sở lý thuyết đến thiết kế, triển khai và đánh giá kết quả. Chương 2 tập trung vào việc khảo sát hiện trạng và phân tích yêu cầu hệ thống, bao gồm các yêu cầu chức năng, phi chức năng, quy trình nghiệp vụ và trình bày các ca sử dụng cụ thể. Nội dung chương này đóng vai trò quan trọng trong việc xác định phạm vi đề tài và làm rõ nhu cầu thực tế của người dùng trong bối cảnh kiểm thử bảo mật cụ thể.

Trên cơ sở các yêu cầu đã xác định, Chương 3 đi sâu vào giới thiệu các công nghệ nền tảng và cơ sở lý thuyết được áp dụng trong đồ án. Các nội dung trọng tâm bao gồm framework xây dựng agent, cơ chế quản lý bộ nhớ dài hạn, nền tảng vận hành mô hình ngôn ngữ lớn cục bộ, phương pháp tìm kiếm RAG và giao thức MCP sử dụng kênh truyền HTTP streamable. Mục tiêu của chương là thiết lập nền tảng kỹ thuật vững chắc để phục vụ cho giai đoạn thiết kế và xây dựng hệ thống ở phần tiếp theo.

Kế thừa nền tảng lý thuyết, Chương 4 trình bày chi tiết quá trình thiết kế kiến trúc, hiện thực hóa hệ thống và đánh giá kết quả. Nội dung bao gồm việc lựa chọn mô hình kiến trúc, thiết kế cơ sở dữ liệu, giao diện người dùng và đặc tả chi tiết các mô-đun như Core Orchestrator Agent, Memory Agent, RAG Agent và Pentest Agent. Chương này cũng công bố kết quả xây dựng thực tế, đồng thời trình bày các kịch bản kiểm thử chức năng và đánh giá hiệu năng hệ thống trên các tình huống mô phỏng sát với thực tế triển khai.

Tiếp theo, Chương 5 tập trung tổng hợp và thảo luận các đóng góp nổi bật của đồ án. Trong đó, nội dung chương làm rõ hiệu quả của cơ chế bộ nhớ dài hạn theo

định hướng MemGPT, cũng như việc áp dụng hybrid search kết hợp reranking để nâng cao chất lượng truy xuất trong hệ thống RAG. Bên cạnh đó, quá trình xây dựng Kali MCP Server, một máy chủ MCP cung cấp bộ công cụ pentest tiêu chuẩn được triển khai trong môi trường cách ly, cũng được trình bày như một giải pháp cho phép LLM thực thi các thao tác kỹ thuật một cách an toàn và có kiểm soát. Cuối cùng, chương thực hiện đối chiếu và so sánh các mô hình nhúng mã nguồn mở phục vụ cho RAG, từ đó lựa chọn mô hình tối ưu, đảm bảo cân bằng giữa hiệu năng suy luận và tài nguyên hệ thống khi triển khai thực tế.

Cuối cùng, Chương 6 đúc kết lại mức độ hoàn thành các mục tiêu đã đề ra ban đầu, nhìn nhận thẳng thắn các hạn chế còn tồn tại và đề xuất định hướng phát triển trong tương lai. Các hướng đi tiếp theo bao gồm việc tinh chỉnh mô hình ngôn ngữ chuyên sâu cho lĩnh vực bảo mật, tích hợp thêm khả năng xử lý hình ảnh cho hệ thống RAG, bổ sung thêm các công cụ pentest và cải thiện khả năng hoạt động độc lập của hệ thống bằng cách xây dựng các kịch bản làm việc cụ thể (playbook).

Kết chương

Tóm lại, chương này đã phác thảo bức tranh toàn cảnh về nhu cầu cấp thiết của việc hỗ trợ kiểm thử xâm nhập trong môi trường nội bộ, đồng thời chỉ ra những hạn chế của các giải pháp hiện có, từ đó định hình mục tiêu xây dựng đề tài. Các giải pháp công nghệ như Ollama, LangGraph, kiến trúc bộ nhớ MemGPT, cơ chế RAG và giao thức MCP, được đề xuất nhằm giải quyết triệt để bài toán vận hành offline và bảo mật dữ liệu. Đây chính là cơ sở quan trọng để định hình quá trình thiết kế và hiện thực hóa hệ thống trong các chương tiếp theo. Tiếp nối phần này, Chương 2 sẽ đi sâu vào khảo sát hiện trạng và phân tích chi tiết yêu cầu kỹ thuật của hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương này phân tích nhu cầu thực tiễn và khảo sát các giải pháp hiện có để định hình phạm vi hệ thống LPAA. Trên cơ sở đó, nội dung đi sâu vào việc xác lập yêu cầu chức năng qua biểu đồ ca sử dụng và quy trình nghiệp vụ, đồng thời thiết lập các tiêu chuẩn phi chức năng về hiệu năng và bảo mật nhằm đáp ứng đặc thù của môi trường pentest nội bộ.

2.1 Khảo sát hiện trạng

Quá trình khảo sát hiện trạng được xây dựng dựa trên kết quả của khảo sát nhu cầu người dùng và phân tích các sản phẩm hiện có. Với đối tượng trọng tâm là các pentester, hệ thống cần đáp ứng các tiêu chuẩn khắt khe về bảo mật dữ liệu, tìm kiếm tri thức và chạy công cụ trong môi trường an toàn. Từ việc tổng hợp nhu cầu này và so sánh với các giải pháp tương tự, đề án sẽ định hình được các tính năng cần thiết để khắc phục những hạn chế mà các công cụ hiện nay đang gặp phải.

2.1.1 Khảo sát nhu cầu người dùng

Thời gian làm việc thực tế cùng đội ngũ pentest tại CyStack đã giúp tôi nhận ra những hạn chế trong quy trình kiểm thử xâm nhập hiện tại. Đầu tiên là gánh nặng về thông tin. Pentester thường xuyên phải xử lý một lượng lớn tài liệu kỹ thuật và đặc tả hệ thống phức tạp. Việc tra cứu thủ công, đặc biệt trong môi trường offline không có Internet, thường kém hiệu quả và dễ dẫn đến việc bỏ sót thông tin khi phân tích.

Tiếp đến là rào cản khi vận hành công cụ. Dù các tác vụ như quét cổng hay đánh giá cấu hình của dịch vụ mang tính lặp lại, nhưng cú pháp lệnh lại đòi hỏi độ chính xác cao. Khi không thể tra cứu trực tuyến, áp lực phải ghi nhớ mọi tham số kỹ thuật không chỉ gây mất thời gian mà còn làm tăng tỷ lệ sai sót.

Cuối cùng là kỳ vọng về khả năng “ghi nhớ”. Các mô hình ngôn ngữ lớn chạy cục bộ thường mất ngữ cảnh trong những phiên làm việc kéo dài do giới hạn về tài nguyên được cung cấp cho cửa sổ ngữ cảnh. Do đó, nhu cầu cấp thiết là một giải pháp có bộ nhớ dài hạn (long-term memory) để xâu chuỗi sự kiện và hỗ trợ ra quyết định chính xác mà không tạo ra “áp lực” quá lớn lên cửa sổ ngữ cảnh.

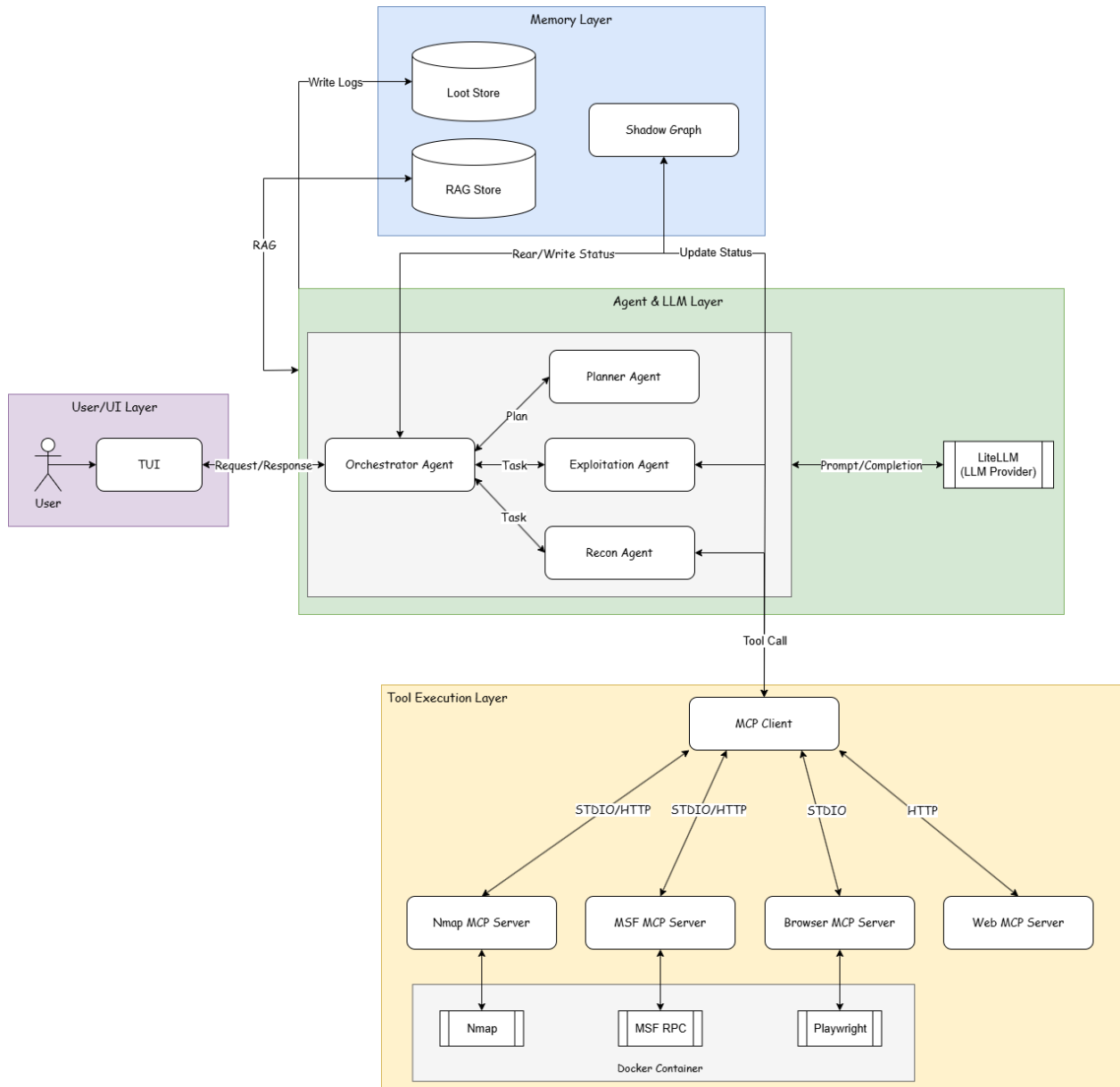
Từ thực tế đó, mục tiêu đặt ra là xây dựng một hệ thống toàn diện: vừa truy xuất thông tin nhanh, hỗ trợ thao tác công cụ tự động, vừa duy trì được ngữ cảnh dài hạn và hoạt động ổn định ngay cả khi offline.

2.1.2 Khảo sát các hệ thống hiện có

Để làm nổi bật thêm nhu cầu cấp thiết của đề tài, tôi đã tiến hành khảo sát và phân tích ba hệ thống đại diện tiêu biểu cho các định hướng kiến trúc khác nhau trong lĩnh vực hỗ trợ kiểm thử bảo mật bằng AI, bao gồm PentestAgent [1], HexStrike AI MCP Agents v6.0 [2] và Strix [3]. Trọng tâm của việc đánh giá được đặt vào năm khía cạnh kỹ thuật cốt lõi: cấu trúc tổng thể và luồng xử lý dữ liệu; cơ chế điều phối tác vụ cùng với khả năng tích hợp công cụ; phương thức tổ chức và tích lũy tri thức theo thời gian; mức độ sẵn sàng đáp ứng yêu cầu vận hành ngoại tuyến (offline-first); và năng lực truy vết phục vụ cho công tác hậu kiểm. Những phân tích này đóng vai trò then chốt trong việc xác định các ưu điểm cần kế thừa, đồng thời chỉ ra những điểm yếu cần được giải quyết. Đây chính là tiền đề quan trọng để định hướng thiết kế cho hệ thống LPAA, đảm bảo sự hiệu quả khi tích hợp công nghệ RAG, giao thức MCP và cơ chế bộ nhớ dài hạn cho đồ án.

a, GH05TCREW/pentestagent

Hệ thống PentestAgent đại diện cho một hướng tiếp cận hiện đại trong việc ứng dụng AI agent vào quy trình kiểm thử xâm nhập, nổi bật với cơ chế điều phối multi-agent linh hoạt. Về mặt kiến trúc, hệ thống được thiết kế theo mô hình phân lớp, bao gồm lớp tương tác LLM nhằm chuẩn hóa việc sử dụng các nhà cung cấp mô hình ngôn ngữ, lớp điều phối chịu trách nhiệm quản lý vòng lặp tác vụ, và lớp tích hợp công cụ thực thi (tool registry/executor) dựa trên chuẩn kết nối MCP. Bên cạnh đó, kiến trúc này còn chú trọng đến việc tổ chức tri thức thông qua sự kết hợp giữa hệ thống ghi chú (notes) và một đồ thị tri thức nhẹ (ShadowGraph). Đặc biệt, hệ thống đã xây dựng một lớp thực thi công cụ an toàn bằng cách triển khai các công cụ pentest trong môi trường Docker. Cách tổ chức bài bản này cho phép hệ thống vận hành song song hai vai trò: vừa là trợ lý ảo tương tác với người dùng, vừa là bộ xử lý kỹ thuật có khả năng chuyển hóa trực tiếp ý định của người dùng thành các thao tác kiểm thử cụ thể.



Hình 2.1: Sơ đồ kiến trúc PentestAgent

Về cơ chế vận hành, PentestAgent áp dụng mô hình agent-loop kết hợp với cơ chế quản lý trạng thái, trong đó quy trình xử lý được thực hiện tuần tự qua các pha suy luận, gọi công cụ rồi tổng hợp kết quả, đảm bảo mọi tác vụ chỉ kết thúc khi có tín hiệu hoàn tất rõ ràng. Trên nền tảng này, hệ thống mở rộng khả năng xử lý thông qua mô hình multi-agent, bao gồm một bộ điều phối trung tâm (orchestrator) và các nhóm tác nhân con (worker pool) thực thi nhiệm vụ. Kiến trúc này cho phép chia nhỏ các vấn đề phức tạp và thực thi song song theo nhiều nhánh kiểm thử khác nhau, tạo nên lợi thế đáng kể trong quá trình kiểm thử xâm nhập.

Tại lớp tích hợp công cụ, hệ thống thực hiện phân tách rõ ràng giữa khả năng suy luận của LLM và năng lực thực thi kỹ thuật thông qua cơ chế đăng ký và kiểm tra tính hợp lệ của công cụ. Đặc biệt, việc hỗ trợ giao thức MCP để kết nối với các

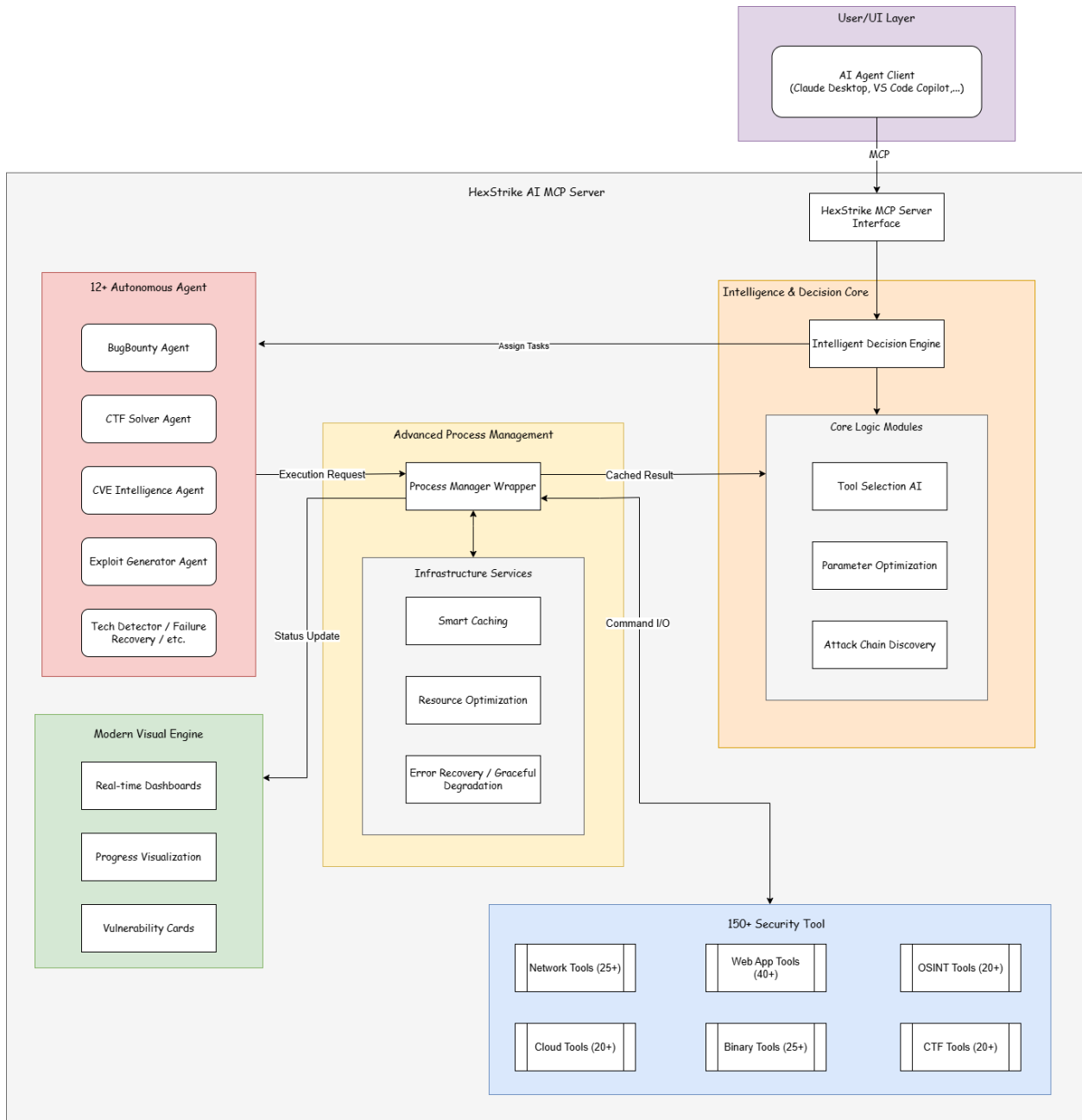
máy chủ công cụ bên ngoài (MCP Server) giúp giảm thiểu độ phụ thuộc của hệ thống và gia tăng khả năng mở rộng hệ sinh thái công cụ. Tuy nhiên, mô hình thực thi hiện tại vẫn thiên về cơ chế gửi yêu cầu và chờ phản hồi hoàn tất, dẫn đến hạn chế trong khả năng giám sát tiến trình theo thời gian thực (streaming). Điều này có thể ảnh hưởng đến trải nghiệm người dùng khi vận hành các công cụ đòi hỏi thời gian xử lý dài hoặc sinh ra lượng log lớn.

Liên quan đến quản lý tri thức và bộ nhớ, hệ thống kết hợp linh hoạt giữa bộ nhớ hội thoại có tóm tắt, hệ thống ghi chú và đồ thị tri thức nhẹ (ShadowGraph) hỗ trợ cho quá trình suy luận. Cách tiếp cận này tạo điều kiện thuận lợi cho việc tái sử dụng các phát hiện trong quá khứ cũng như cung cấp ngữ cảnh dựa trên mối quan hệ logic giữa các mảnh ký ức trên đồ thị. Đây là một hướng triển khai tốt để duy trì bộ nhớ ngữ nghĩa lâu dài cho từng phiên làm việc. Tuy nhiên PentestAgent chưa có phương thức để lưu lại log cuộc hội thoại hay các lời gọi công cụ (chủ yếu sử dụng bộ nhớ in-memory) do đó không duy trì được bộ nhớ sau khi phiên làm việc kết thúc. Hơn nữa, cơ chế RAG hiện tại chủ yếu dựa trên truy xuất véc-tơ đơn thuần, thiếu sự hỗ trợ của hybrid search và reranking (2 khái niệm này sẽ được giải thích kĩ hơn ở Chương 5), thường đạt hiệu suất thấp trong việc xác định chính xác các tài liệu đặc thù như mã lỗi CVE, chuỗi cấu hình, cách khai thác hay payload cụ thể.

Tổng hợp lại, PentestAgent thể hiện ưu thế vượt trội ở khả năng điều phối multi-agent, tích hợp công cụ linh hoạt theo chuẩn MCP và tổ chức dữ liệu thông qua hệ thống ghi chú kết hợp đồ thị tri thức. Tuy nhiên, khi đối chiếu với các mục tiêu của đề tài, hệ thống này vẫn bộc lộ những hạn chế cần được giải quyết. Các hạn chế chính bao gồm thiếu hụt bộ nhớ dài hạn có khả năng tồn tại qua các phiên làm việc, hạn chế trong quy mô và độ chính xác của cơ chế truy xuất tri thức, cũng như chưa hoàn thiện quy trình lưu vết dữ liệu để phục vụ cho công tác hậu kiểm sau này.

b, HexStrike AI MCP Agents v6.0 (hexstrike-ai)

HexStrike AI v6.0 đại diện cho bước tiến từ một nền tảng điều phối công cụ đơn thuần sang mô hình kiến trúc multi-agent, với trọng tâm là sự phối hợp giữa hơn 150 công cụ bảo mật và hơn 12 AI agent hỗ trợ quá trình tự động hóa. Xét về mặt tổng thể, hệ thống vẫn duy trì mô hình client-server nhưng đã được nâng cấp đáng kể về khả năng ra quyết định. Tầng client (sử dụng AI agent như Claude/GPT/Copilot) kết nối thông qua giao thức FastMCP, đóng vai trò là lớp điều phối và lên kế hoạch chính của hệ thống. Trong khi đó, tầng backend (HexStrike AI MCP Server) không chỉ là nơi thực thi lệnh mà còn là nơi tích hợp Intelligent Decision Engine (Bộ máy quyết định thông minh) để phân tích mục tiêu, lựa chọn chiến lược và tối ưu hóa tham số cho công cụ.



Hình 2.2: Sơ đồ kiến trúc của HexStrike AI MCP Agents v6.0

Quy trình vận hành của HexStrike v6.0 tuân thủ một luồng xử lý khép kín và hoàn toàn tự động. Khi nhận được yêu cầu từ AI Client, hệ thống không thực thi ngay lập tức mà chuyển qua Intelligent Decision Engine. Tại đây, các agent chuyên biệt như BugBounty Agent, CTF Solver Agent hay Exploit Generator Agent sẽ được điều phối để xử lý các tác vụ tương ứng với chức năng. Hệ thống sử dụng cơ chế Smart Caching với thuật toán LRU (Least Recently Used) để quản lý kết quả và tài nguyên, giúp giảm thiểu độ trễ và tránh việc lặp lại các công cụ không cần thiết. Đồng thời, mô-đun Error Recovery/Graceful Degradation đảm bảo tính bền vững của quy trình bằng cách tự động xử lý các lỗi timeout hoặc rate-limit, cho phép hệ thống “suy thoái nhẹ nhàng” (graceful degradation) thay vì dừng hoàn toàn các

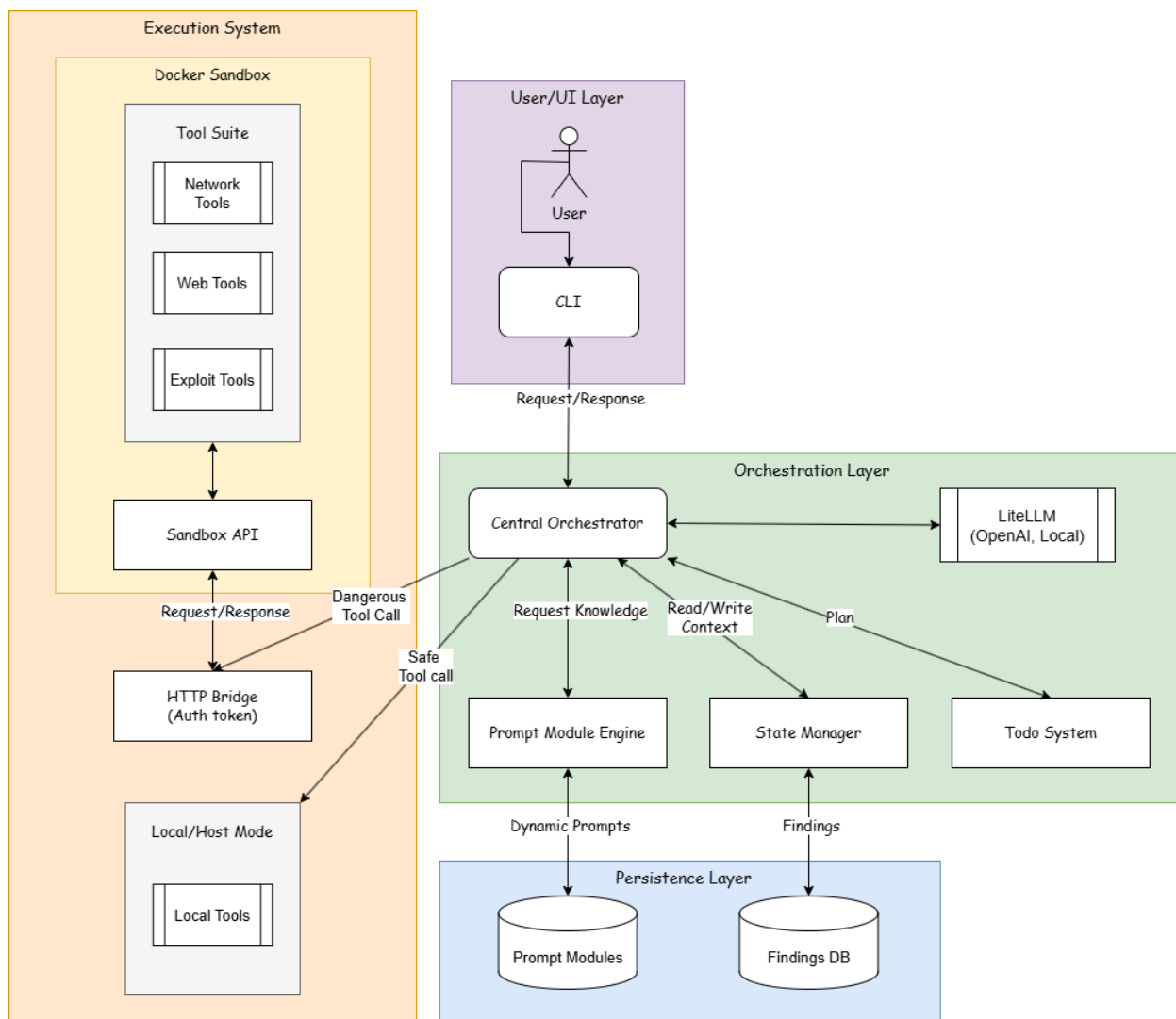
hoạt động khi gặp sự cố.

Mặc dù sở hữu năng lực thực thi công cụ ẩn tượng với tốc độ nhanh gấp nhiều lần so với cách làm việc truyền thống, kiến trúc HexStrike v6.0 vẫn bộc lộ những hạn chế nhất định khi so sánh với yêu cầu và mục tiêu đặt ra của đề tài. Thứ nhất, cơ chế bộ nhớ hiện tại (Smart Caching) thiên về lưu trữ kết quả đầu ra của công cụ (output caching) hơn là lưu trữ tri thức ngữ nghĩa (semantic memory). Hệ thống thiếu vắng một tầng RAG nội bộ để tích lũy kinh nghiệm từ quá khứ, khiến khả năng "học" của agent bị giới hạn trong phiên hiện tại. Thứ hai, dù có Vulnerability Intelligence Agent, hệ thống bản chất là một MCP Server, do đó vẫn phụ thuộc lớn vào các AI Client bên ngoài (như Claude Desktop, VS Code Copilot,...), chưa đảm bảo được tính "offline-first" - một yếu tố then chốt cho các môi trường làm việc yêu cầu tính bí mật của thông tin.

Tổng hợp lại, HexStrike v6.0 là một giải pháp xuất sắc cho bài toán tự động hóa quy trình kiểm thử xâm nhập nhờ vào kho công cụ đồ sộ và khả năng quản lý tiến trình thông minh. Tuy nhiên, HexStrike chỉ đóng vai trò là tài liệu tham khảo giá trị về mặt kỹ thuật thực thi công cụ. Đối với mục tiêu xây dựng một AI agent sở hữu bộ nhớ ngữ nghĩa dài hạn, kiến trúc này chưa thể giải quyết trọn vẹn bài toán quản lý tri thức cũng như yêu cầu hoạt động độc lập, không phụ thuộc vào LLM thương mại.

c, Strix

Strix được định hình là một framework mã nguồn mở chuyên biệt cho công việc tự động hóa quy trình kiểm thử bảo mật, hoạt động dựa trên cơ chế mô phỏng hành vi của một hacker thực thụ ("Strix are autonomous AI agents that act just like real hackers" [3]). Hệ thống này có khả năng tự động xây dựng kế hoạch tấn công, thực thi các công cụ hoặc PoC để xác minh lỗ hổng và tổng hợp báo cáo kết quả chi tiết. Xét về khía cạnh kiến trúc, Strix được tổ chức thành các tầng chức năng có mối liên kết chặt chẽ. Tầng giao diện người dùng đảm nhận vai trò cấu hình mục tiêu và khởi tạo môi trường vận hành. Tầng quản lý trạng thái và ngữ cảnh, chịu trách nhiệm lưu trữ toàn bộ lịch sử tương tác, hành động và các kết quả thu thập được. Tiếp đến là tầng vận hành trung tâm đóng vai trò điều phối sự tương tác giữa LLM và công cụ theo quy trình lặp nhiều bước. Cuối cùng, hệ thống tích hợp tầng thực thi công cụ và tầng sandbox sử dụng Docker, nhằm đảm bảo tính cách ly an toàn giữa môi trường chạy công cụ và tiến trình xử lý của agent.



Hình 2.3: Sơ đồ kiến trúc của Strix

Cơ chế vận hành của Strix tuân thủ theo mô hình agent-loop kinh điển, trong đó tại mỗi chu kỳ, LLM sẽ phân tích, lập kế hoạch và khởi tạo các lời gọi công cụ. Hệ thống sau đó thực thi các yêu cầu, thu thập dữ liệu quan sát, cập nhật trạng thái và lặp lại quy trình này cho đến khi hoàn thành mục tiêu hoặc đạt ngưỡng giới hạn. Điểm đột phá trong kiến trúc của Strix nằm ở lớp thực thi công cụ hoạt động theo chế độ kép (dual-mode). Cụ thể, bên cạnh các công cụ chạy cục bộ, các công cụ được đánh dấu cần cách ly sẽ được chuyển hướng sang một máy chủ thực thi riêng biệt chạy trong môi trường Docker sandbox. Quá trình giao tiếp này được thực hiện thông qua giao thức HTTP kèm cơ chế xác thực bằng Authentication Token (mã thông báo xác thực), giúp tách biệt hoàn toàn môi trường suy luận khỏi môi trường thực thi kỹ thuật, qua đó giảm thiểu tối đa rủi ro an ninh cho máy chủ vận hành agent.

Để giải quyết bài toán giới hạn tài nguyên tính toán và giúp giảm thiểu chi phí token, Strix áp dụng chiến lược tóm tắt cuộc hội thoại. Khi ngữ cảnh vượt quá

ngưỡng cho phép, hệ thống sẽ tự động tóm tắt lịch sử hội thoại cũ nhưng vẫn ưu tiên bảo lưu các thông tin bảo mật quan trọng như lỗ hổng phát hiện được, thông tin xác thực, phiên bản dịch vụ hay các payload thử nghiệm thất bại/thành công. Phương pháp này đảm bảo tính liên tục của mạch làm việc trong một phiên, tuy nhiên, về bản chất đây vẫn chỉ là bộ nhớ ngắn hạn. Strix chưa chú trọng xây dựng lớp lưu trữ có khả năng truy vấn dài hạn (episodic recall), dẫn đến việc kết quả kiểm thử thường chỉ tồn tại dưới dạng các báo cáo rời rạc thay vì hình thành một kho tri thức có cấu trúc và có thể tái sử dụng.

Về khía cạnh quản lý tri thức, phiên bản Strix được khảo sát chưa tích hợp các cơ chế RAG hay cơ sở dữ liệu véc-tơ để khai thác nguồn tri thức nội bộ. Thay vào đó, hệ thống định hướng dạy agent thông qua các mẫu câu lệnh (prompt template) và thay đổi linh hoạt các mẫu câu lệnh đó trong từng môi trường làm việc cụ thể. Mặc dù cách tiếp cận này mang lại sự linh hoạt cao cho các kịch bản pentest, nhưng nó lại bộc lộ hạn chế lớn trong việc cập nhật tri thức từ tài liệu kỹ thuật, đồng thời làm giảm sự sáng tạo của mô hình ngôn ngữ lớn do bị gò bó vào những kịch bản có sẵn.

Xét về khả năng vận hành an toàn và ngoại tuyến, Strix ghi điểm nhờ cơ chế Docker sandbox và khả năng tích hợp linh hoạt các LLM cục bộ thông qua LiteLLM. Tuy nhiên, tính chất offline của hệ thống này chưa thực sự triệt để mà phụ thuộc nhiều vào cấu hình và các công cụ đi kèm.

Nhìn nhận tổng thể, dù Strix sở hữu kiến trúc luồng xử lý rõ ràng và cơ chế cách ly tốt, nó vẫn bộc lộ những khoảng trống lớn so với mục tiêu của đề tài. Cụ thể là sự thiếu hụt kiến trúc RAG cho dữ liệu nội bộ, không có bộ nhớ dài hạn phân tầng và chưa chú trọng xây dựng hệ thống truy vết nhằm phục vụ cho công tác hậu kiểm sau này.

d, Nhận xét tổng hợp

Kết quả phân tích từ ba hệ thống PentestAgent, HexStrike AI và Strix đã phác họa rõ nét các khuynh hướng kiến trúc khác nhau trong việc ứng dụng trí tuệ nhân tạo vào kiểm thử bảo mật. Trong khi PentestAgent theo đuổi mô hình multi-agent với thế mạnh về điều phối đa tác tử và cơ quản lý tri thức thông qua đồ thị, thì HexStrike AI lại tập trung vào việc tối ưu tầng điều phối công cụ, chú trọng vào sự bền vững và khả năng tự phục hồi lỗi trong quá trình vận hành và thực thi công cụ. Ở một hướng tiếp cận khác, Strix lựa chọn mô phỏng hành vi tấn công thực tế thông qua kiến trúc agent loop và ưu tiên cơ chế cách ly môi trường thực thi bằng công nghệ Docker.

Mặc dù mỗi nền tảng đều sở hữu những ưu điểm công nghệ riêng, song khi đối

chiều với các mục tiêu cốt lõi của đề tài, vẫn tồn tại những khoảng trống kiến trúc đáng kể chưa được giải quyết. Cụ thể, các giải pháp này chưa mặc định hỗ trợ chế độ offline, thiếu bộ nhớ dài hạn cũng như cơ chế RAG để khai thác tri thức nội bộ. Khả năng truy vết lỗi và hậu kiểm cũng là một rào cản, do hệ thống log hiện tại chưa được chú trọng. Chính những thách thức công nghệ này đã tạo tiền đề quan trọng để đề án đề xuất kiến trúc “Local Pentest AI agent”, với định hướng tập trung vào khả năng vận hành hoàn toàn cục bộ, tích lũy tri thức bền vững theo thời gian, nâng cao năng lực truy xuất thông tin kết hợp vận hành công cụ đảm bảo an toàn bảo mật trong môi trường kiểm thử nội bộ.

2.2 Tổng quan chức năng

Nội dung phần này tập trung phác thảo bức tranh tổng thể về các nhóm chức năng chính của hệ thống LPAA, nổi bật với các công nghệ như RAG, MCP và long-term memory theo hướng MemGPT. Hệ thống chức năng này được định hình dựa trên kết quả của quá trình tổng hợp từ nhu cầu thực tiễn, phân tích bài học từ các giải pháp đi trước, và đối chiếu với những yêu cầu an toàn đặc thù của môi trường pentest nội bộ. Mục tiêu trọng tâm của phần này là định hình rõ nhiệm vụ và phạm vi hoạt động của các chức năng ở mức tổng quát, đồng thời làm rõ vai trò của chúng trong hệ thống để từ đó tạo cơ sở định hướng cho việc đặc tả chi tiết các ca sử dụng sẽ được trình bày trong mục 2.3.

2.2.1 Biểu đồ ca sử dụng tổng quát

Trước khi phân tích chi tiết từng ca sử dụng, cần xác định rõ các tác nhân chính của hệ thống và vai trò của chúng trong quá trình vận hành.

Pentester là tác nhân bên ngoài khởi tạo hầu hết các tương tác với hệ thống, bao gồm đặt câu hỏi, yêu cầu phân tích, nạp tài liệu, truy vấn bộ nhớ hoặc yêu cầu thực thi các công cụ pentest. Mọi yêu cầu từ pentester đều được tiếp nhận và xử lý ban đầu bởi Core Orchestrator, đóng vai trò là trung tâm điều phối của hệ thống.

Core Orchestrator chịu trách nhiệm phân tích yêu cầu đầu vào, chuẩn hóa ngữ cảnh và phân rã nhiệm vụ thành các tác vụ con phù hợp, sau đó phân phối các tác vụ này cho các tác nhân chuyên biệt. Core Orchestrator không trực tiếp thực hiện truy xuất tri thức, quản lý bộ nhớ hay chạy công cụ kỹ thuật, mà đóng vai trò điều phối luồng xử lý, tổng hợp kết quả từ các agent con và tạo phản hồi cuối cùng cho người dùng.

Memory Agent chịu trách nhiệm quản lý toàn bộ bộ nhớ của hệ thống, bao gồm cả core memory (bộ nhớ dài hạn, mang tính ổn định và chiến lược) và recall memory (bộ nhớ ngắn hạn theo phiên làm việc). Memory Agent tiếp nhận các yêu cầu lưu hoặc truy vấn bộ nhớ từ Core Orchestrator, thực hiện các thao tác tìm kiếm,

cập nhật, tóm tắt hoặc đồng bộ dữ liệu với Kho trí nhớ nhằm đảm bảo tính liên tục và nhất quán của ngữ cảnh.

RAG Agent là tác nhân chuyên dụng cho tác vụ truy xuất tri thức. Agent này tiếp nhận yêu cầu từ Core Orchestrator liên quan đến tìm kiếm tri thức bên ngoài, thực hiện véc-tơ hóa nội dung bằng mô hình nhúng (mô hình nhúng), truy vấn Kho tri thức, lọc và tổng hợp các thông tin liên quan, sau đó trả về kết quả đã được chất lọc phục vụ cho quá trình suy luận và trả lời.

Pentest Agent là tác nhân chịu trách nhiệm thực thi các nhiệm vụ pentest. Khi nhận được yêu cầu từ Core Orchestrator, Pentest Agent xác định công cụ phù hợp, chuẩn bị tham số và gửi yêu cầu thực thi tới MCP Server. Tác nhân này cũng đóng vai trò như một MCP Client, tiếp nhận kết quả trả về từ MCP Server và báo cáo lại cho Core Orchestrator.

MCP Server đóng vai trò là cầu nối giữa hệ thống AI agent và các công cụ pentest thực tế. MCP Server tiếp nhận các yêu cầu kỹ thuật từ Pentest Agent, thực thi các công cụ pentest như Nmap, Metasploit trong môi trường sandbox, và trả kết quả của công cụ trở về.

Pentest Database được sử dụng để lưu trữ lịch sử hội thoại, kết quả phân tích, đầu ra công cụ và các thông tin vận hành khác nhằm phục vụ giao diện người dùng, truy vấn hậu kiểm và đánh giá quá trình pentest.

Ngoài ra, hệ thống còn bao gồm Kho tri thức dùng để lưu trữ các tài liệu tĩnh (CVE, write-up, tài liệu kỹ thuật) được nạp vào bởi Pentester và Kho trí nhớ dùng để lưu trữ các trí nhớ ngắn hạn và dài hạn phát sinh trong quá trình tương tác với người dùng.

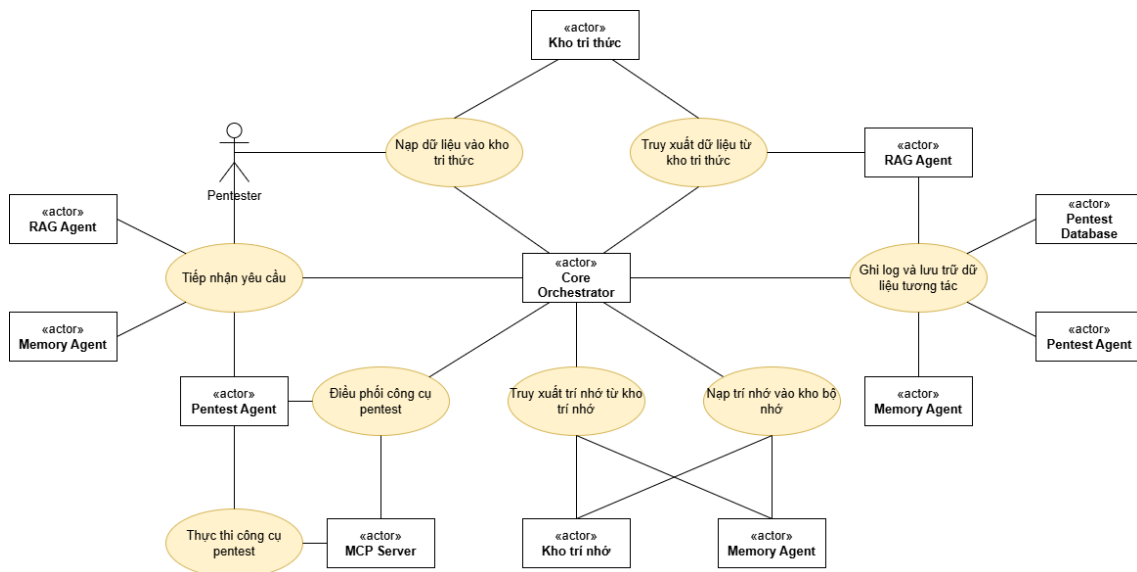
Từ các tác nhân trên, tầm cao sử dụng mức cao của hệ thống được xác định như sau:

1. Tiếp nhận yêu cầu - Core Orchestrator tiếp nhận yêu cầu từ pentester cùng với ngữ cảnh hiện có của hệ thống (core memory, recall memory và message queue), từ đó phân tích và phân chia nhiệm vụ cho các agent phù hợp hoặc trực tiếp tạo phản hồi.
2. Nạp dữ liệu vào kho tri thức - cho phép pentester cung cấp tài liệu kèm siêu dữ liệu (metadata), hệ thống thực hiện chuyển đổi, phân đoạn dựa trên ranh giới ngữ nghĩa (semantic chunking), sinh véc-tơ nhúng (véc-tơ embedding) và nạp dữ liệu vào kho tri thức.
3. Truy xuất dữ liệu từ kho tri thức - RAG Agent thực hiện truy xuất dữ liệu từ kho tri thức thông qua cơ chế RAG kết hợp hybrid search và reranking, rồi sau

đó dựa trên các đoạn thông tin tìm được sẽ tổng hợp các thông tin liên quan.

4. Nạp thông tin vào kho trí nhớ - Memory Agent thực hiện tóm tắt các thông điệp bị loại khỏi message queue để lưu vào recall memory, đồng thời tiếp nhận yêu cầu lưu hoặc cập nhật core memory từ Core Orchestrator.
5. Truy xuất thông tin từ kho trí nhớ - Memory Agent truy xuất recall memory theo yêu cầu của Core Orchestrator nhằm khôi phục ngữ cảnh làm việc.
6. Điều phối công cụ pentest - Core Orchestrator đưa chỉ dẫn thực thi cho Pentest Agent để từ đó agent này diễn giải thành các lời gọi công cụ cụ thể.
7. Thực thi công cụ pentest - MCP Server tiếp nhận lệnh từ Pentest Agent, thực thi các công cụ pentest trong môi trường sandbox và trả kết quả theo luồng.
8. Ghi log và lưu trữ dữ liệu tương tác - Core Orchestrator cùng các tác nhân ghi log hội thoại, kết quả công cụ, kết quả truy xuất RAG và toàn bộ hoạt động phát sinh vào Pentest Database.

Các mối quan hệ này được thể hiện trong sơ đồ ca sử dụng tổng quát dưới đây.

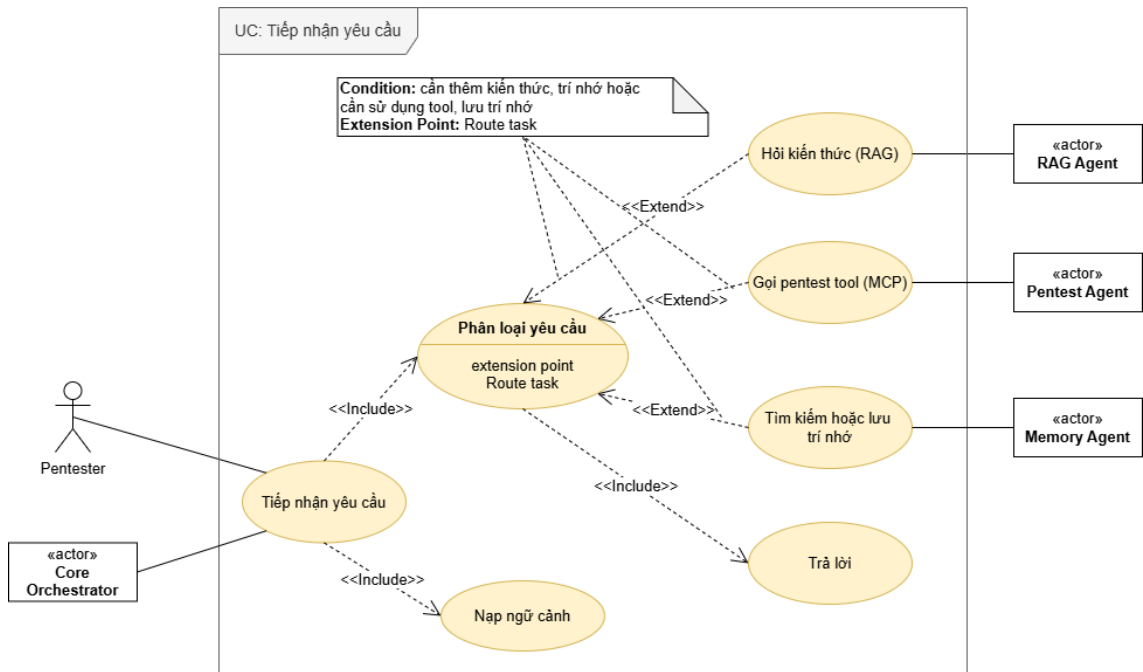


Hình 2.4: Biểu đồ ca sử dụng tổng quát

2.2.2 Biểu đồ ca sử dụng phân rã Tiếp nhận yêu cầu

Ca sử dụng “Tiếp nhận yêu cầu” mô tả quá trình Core Orchestrator tiếp nhận thông tin đầu vào từ Pentester cùng với ngữ cảnh cuộc hội thoại hiện tại từ hệ thống. Ngữ cảnh này bao gồm core memory, recall memory (ở đây chỉ nạp vào bản tóm tắt của các tin nhắn đã bị xóa khỏi message queue) và message queue chứa các thông điệp gần nhất. Dựa trên nội dung yêu cầu và ngữ cảnh, Core Orchestrator thực hiện phân tích để xác định mục đích của yêu cầu.

Nếu yêu cầu liên quan đến truy vấn tri thức, Core Orchestrator kích hoạt RAG Agent; nếu yêu cầu liên quan đến thao tác kỹ thuật, Pentest Agent sẽ được sử dụng để thực thi công cụ thông qua MCP Server; trong trường hợp cần khôi phục hoặc cập nhật ngữ cảnh, Memory Agent sẽ được gọi. Cơ chế điều phối linh hoạt này là yếu tố then chốt giúp hệ thống xử lý chính xác các yêu cầu đa dạng, đồng thời duy trì mạch tư duy và tính nhất quán xuyên suốt phiên làm việc.

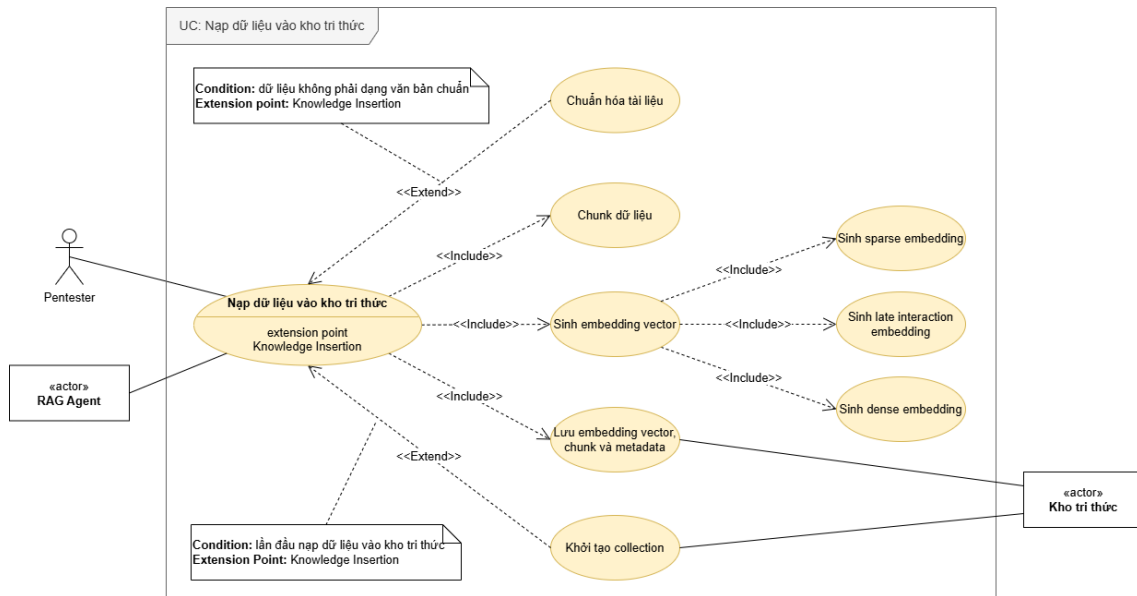


Hình 2.5: Biểu đồ ca sử dụng Tiếp nhận yêu cầu

2.2.3 Biểu đồ ca sử dụng phân rã Nạp dữ liệu vào kho tri thức

Ca sử dụng “Nạp dữ liệu vào kho tri thức” mô tả quy trình hệ thống tiếp nhận một nguồn tài liệu mới từ Pentester, xử lý nội dung và cập nhật vào Kho tri thức để phục vụ cho các truy vấn RAG. Khi tài liệu được nạp, hệ thống thực hiện chuyển đổi tài liệu sang dạng văn bản chuẩn nếu cần thiết, sau đó áp dụng semantic chunking để phân đoạn nội dung theo ngữ nghĩa.

Đối với mỗi đoạn nội dung, hệ thống kích hoạt mô hình nhúng để chuyển đổi từng phân đoạn văn bản thành các dạng biểu diễn véc-tơ đa chiều, bao gồm véc-tơ dày (dense), véc-tơ thưa (sparse) và véc-tơ cho từng token phục vụ cơ chế tương tác muộn (late interaction) cho hoạt động reranking sau này. Toàn bộ dữ liệu véc-tơ này, đi kèm với các metadata tương ứng, cuối cùng sẽ được lưu trữ trong Kho tri thức để đảm bảo khả năng truy xuất chính xác và hiệu quả.

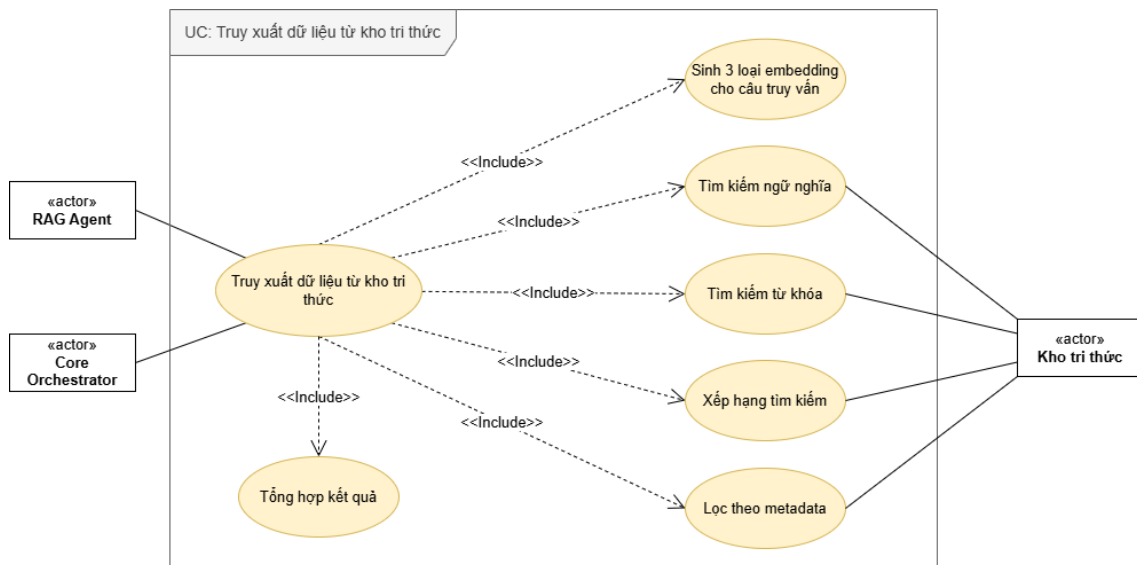


Hình 2.6: Biểu đồ ca sử dụng Nạp dữ liệu vào kho tri thức

2.2.4 Biểu đồ ca sử dụng phân rã Truy xuất dữ liệu từ kho tri thức

Ca sử dụng “Truy xuất dữ liệu từ kho tri thức” mô tả quá trình RAG Agent tìm kiếm và tổng hợp tri thức để phản hồi yêu cầu của Core Orchestrator. Khi được Core Orchestrator kích hoạt, RAG Agent thực hiện truy xuất tri thức thông qua cơ chế hybrid search, trước hết lọc theo metadata ra các dữ liệu liên quan rồi kết hợp tìm kiếm ngữ nghĩa và tìm kiếm từ khóa.

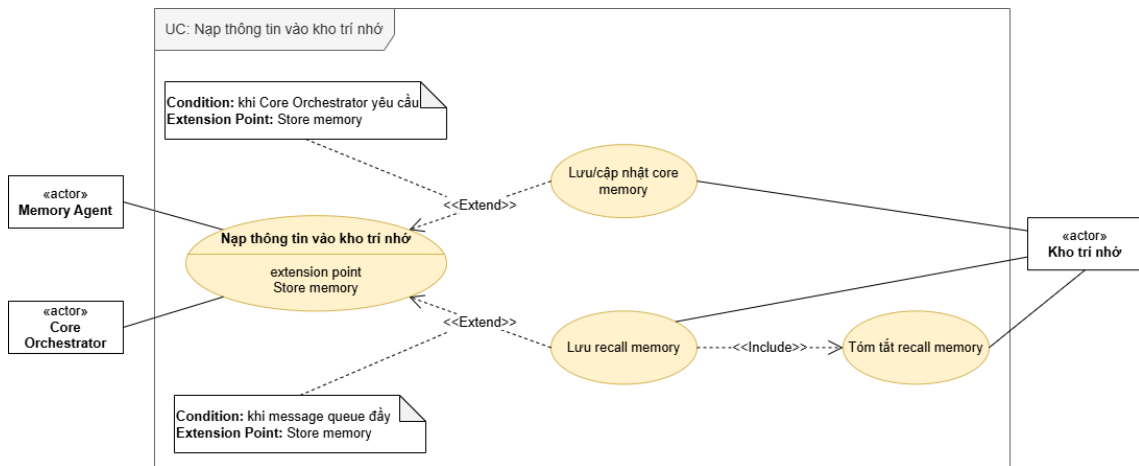
Các kết quả truy xuất sau đó được đưa qua bước reranking để sắp xếp lại các đoạn tri thức phù hợp nhất. Cuối cùng, các mảnh tri thức rời rạc này được RAG Agent tổng hợp thành một cấu trúc thông tin hoàn chỉnh trước khi trả về cho Core Orchestrator để phục vụ các bước xử lý tiếp theo.



Hình 2.7: Biểu đồ ca sử dụng Truy xuất dữ liệu từ kho tri thức

2.2.5 Biểu đồ ca sử dụng phân rã Nạp thông tin vào kho trí nhớ

Ca sử dụng “Nạp thông tin vào kho trí nhớ” đặc tả cơ chế tự động hóa quy trình duy trì và cập nhật trạng thái ngữ cảnh của Memory Agent. Hoạt động này được kích hoạt khi message queue chạm ngưỡng giới hạn về dung lượng, buộc hệ thống phải thực hiện quy trình lưu trữ: các thông điệp cũ được chuyển vào Kho trí nhớ để lưu trữ lâu dài, đồng thời nội dung của chúng được cô đọng thành các bản tóm tắt súc tích để tích hợp vào recall memory. Song song với quy trình tự động này, Memory Agent còn chủ động tiếp nhận các chỉ thị từ Core Orchestrator để cập nhật hoặc bổ sung vào core memory những thông tin mang tính chiến lược và dài hạn.

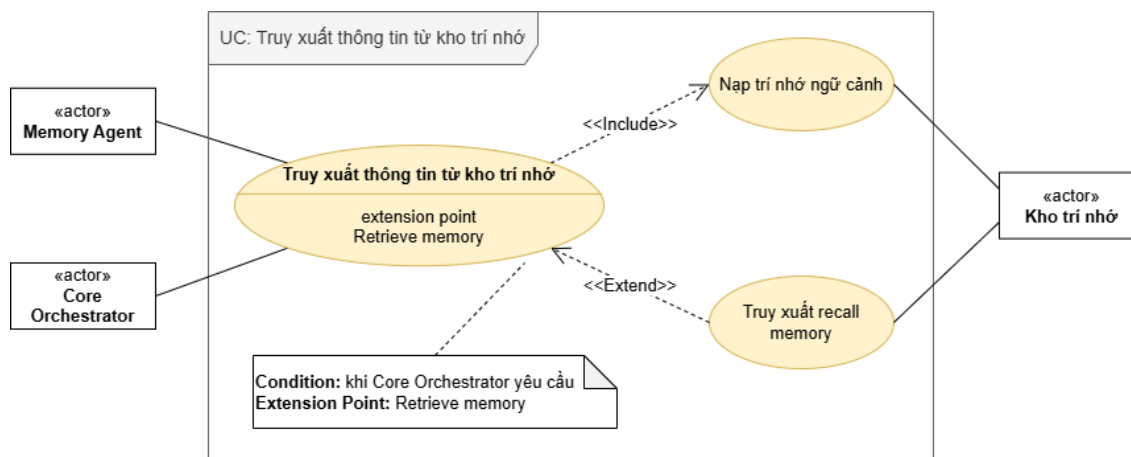


Hình 2.8: Biểu đồ ca sử dụng Nạp thông tin vào kho trí nhớ

2.2.6 Biểu đồ ca sử dụng phân rã Truy xuất thông tin từ kho trí nhớ

Ca sử dụng “Truy xuất thông tin từ kho trí nhớ” mô tả quá trình Memory Agent truy xuất ngữ cảnh khi Core Orchestrator yêu cầu. Memory Agent thực hiện tìm kiếm trên kho trí nhớ để lấy ra các mảnh ký ức liên quan, sau đó tổng hợp lại thành ngữ cảnh thống nhất.

Kết quả của quá trình này được cung cấp ngược lại cho Core Orchestrator, đóng vai trò là nguồn dữ liệu nền tảng hỗ trợ cho các quyết định tiếp theo như truy vấn tri thức chuyên sâu hay điều phối công cụ, qua đó đảm bảo sự nhất quán về mặt logic và duy trì tính liên tục của mạch xử lý xuyên suốt các phiên làm việc kéo dài.

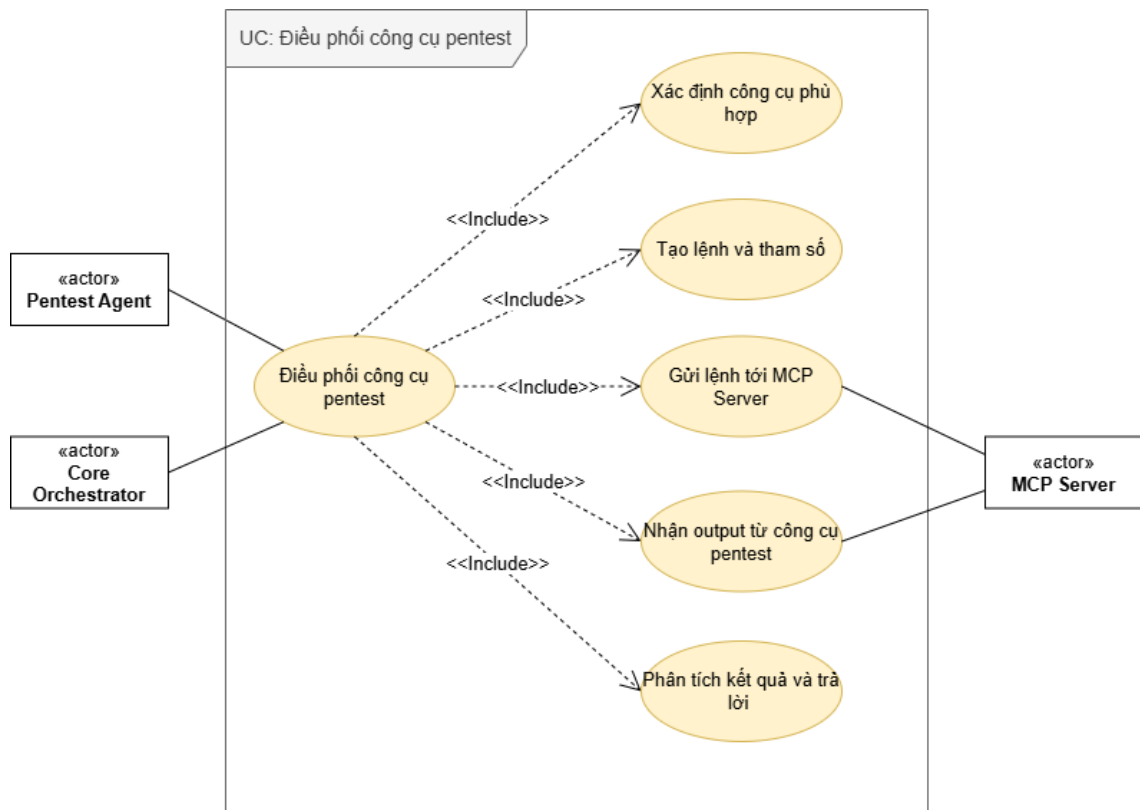


Hình 2.9: Biểu đồ ca sử dụng Truy xuất thông tin từ kho trí nhớ

2.2.7 Biểu đồ ca sử dụng phân rã Điều phối công cụ pentest

Ca sử dụng “Điều phối công cụ pentest” mô tả quá trình Core Orchestrator và Pentest Agent phối hợp để thực thi các thao tác kỹ thuật. Core Orchestrator đưa ra chỉ dẫn ở mức tổng quát về mục tiêu và loại công cụ cần sử dụng, Pentest Agent diễn giải các chỉ dẫn này để xác định công cụ cụ thể và tham số phù hợp.

Đóng vai trò là một MCP Client trong mô hình kiến trúc MCP, Pentest Agent thiết lập kênh giao tiếp thông qua giao thức HTTP streamable để gửi yêu cầu thực thi tới MCP Server, đồng thời đảm nhận vai trò tiếp nhận, phân tích luồng dữ liệu và tổng hợp kết quả trả về để phản hồi lại cho Core Orchestrator.

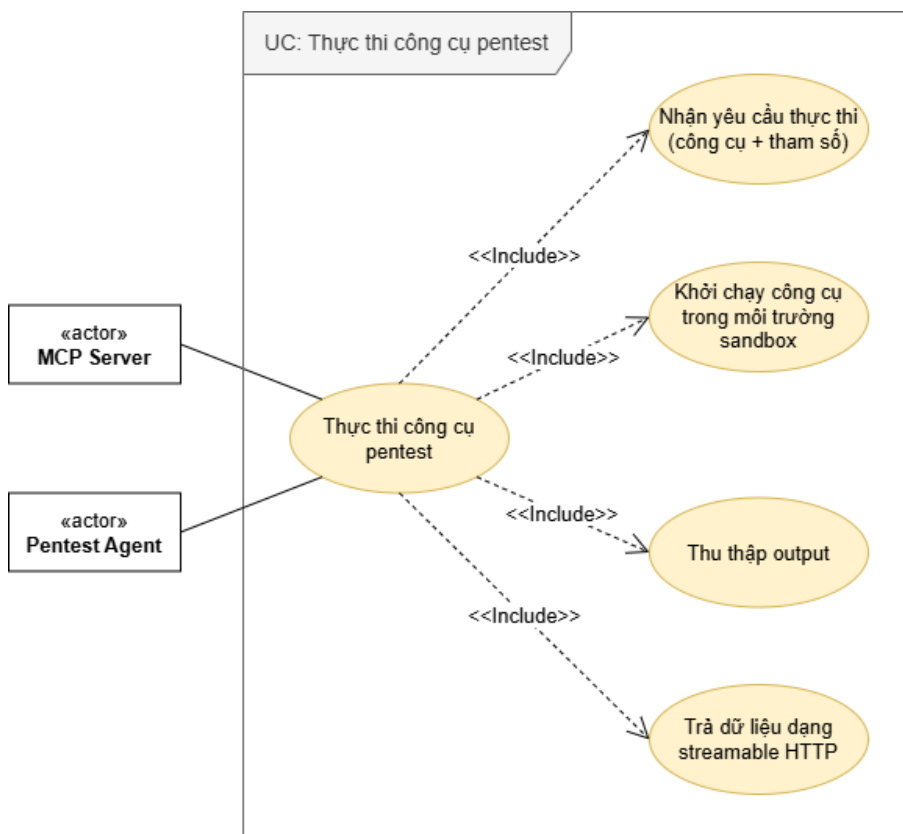


Hình 2.10: Biểu đồ ca sử dụng Điều phối công cụ pentest

2.2.8 Biểu đồ ca sử dụng phân rã Thực thi công cụ pentest

Ca sử dụng “Thực thi công cụ pentest” mô tả quá trình xử lý bên phía MCP Server khi nhận lệnh từ Pentest Agent. MCP Server khởi chạy công cụ trong môi trường sandbox để đảm bảo an toàn và cách ly, đồng thời thu thập dữ liệu từ các luồng output trong suốt quá trình thực thi.

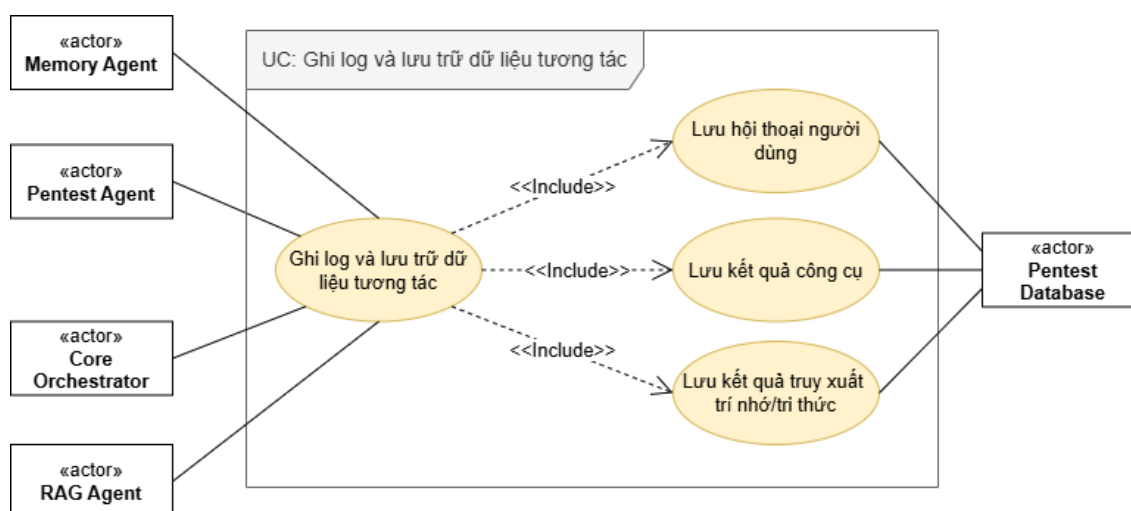
Các dữ liệu này được truyền lại cho Pentest Agent dưới dạng streamable HTTP response, cho phép agent theo dõi tiến trình và kết quả theo thời gian thực.



Hình 2.11: Biểu đồ ca sử dụng Thực thi công cụ pentest

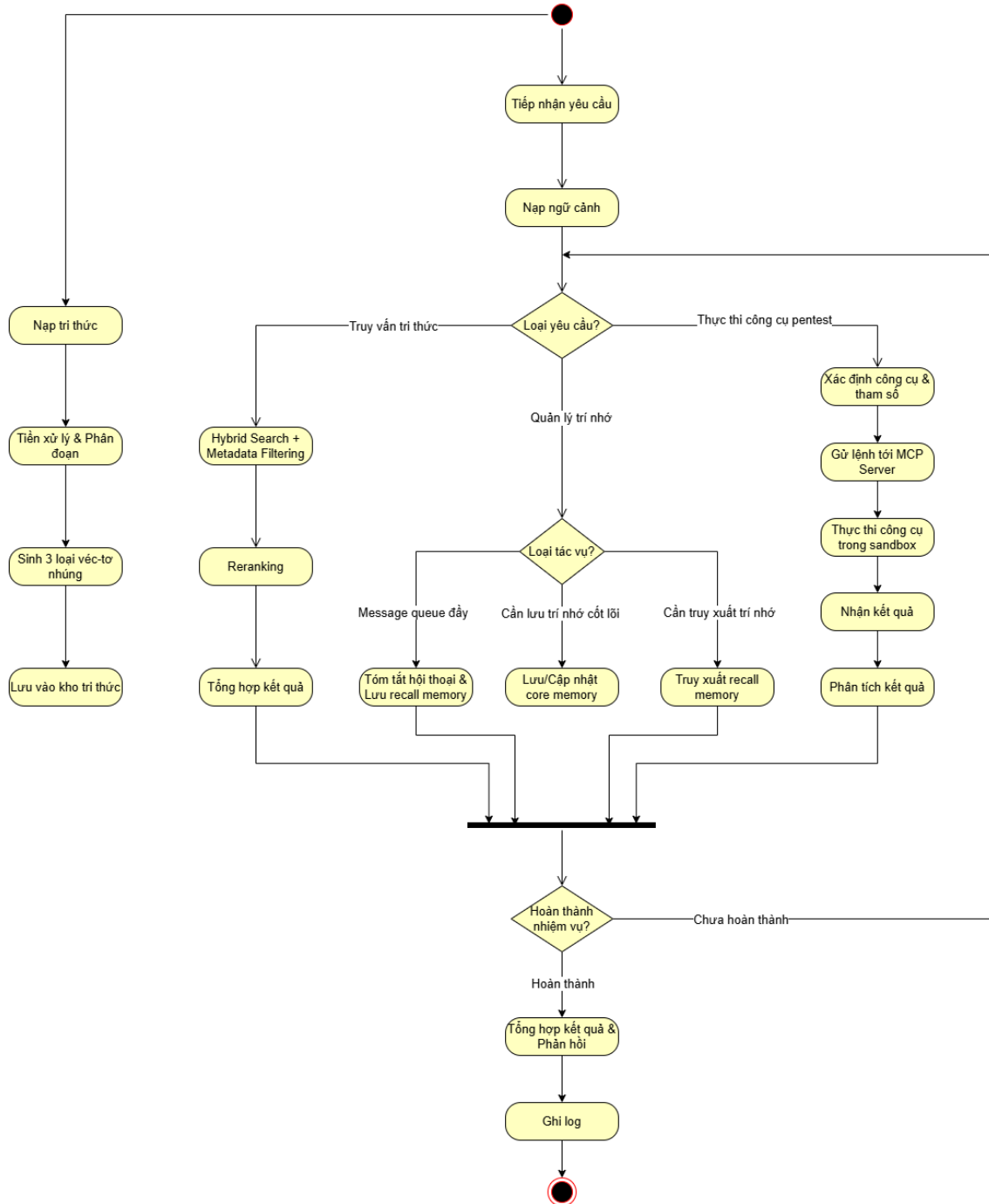
2.2.9 Biểu đồ ca sử dụng phân rã Ghi log và lưu trữ dữ liệu tương tác

Ca sử dụng “Ghi log và lưu trữ dữ liệu tương tác” mô tả cách hệ thống ghi nhận toàn bộ hoạt động phát sinh trong phiên pentest. Core Orchestrator cùng các tác nhân ghi lại lịch sử hội thoại, kết quả truy xuất tri thức, dữ liệu bộ nhớ và đầu ra công cụ vào Pentest Database. Các dữ liệu này được lưu trữ kèm mốc thời gian để phục vụ cho việc truy vết, đánh giá và phân tích sau này.



Hình 2.12: Biểu đồ ca sử dụng Ghi log và lưu trữ dữ liệu tương tác

2.2.10 Quy trình nghiệp vụ



Hình 2.13: Biểu đồ hoạt động Quy trình nghiệp vụ của hệ thống LPAA

Quy trình nghiệp vụ của hệ thống bắt đầu khi Pentester gửi một yêu cầu mới thông qua giao diện người dùng. Yêu cầu này cùng với phần ngữ cảnh được hệ thống trích xuất bao gồm core memory, recall memory (đại diện bởi bản tóm tắt các thông điệp đã bị loại khỏi message queue) và message queue chứa các thông điệp gần nhất sẽ được Core Orchestrator tiếp nhận và phân tích. Trên cơ sở đó, Core Orchestrator phân tích mục tiêu của yêu cầu và quyết định hướng xử lý phù hợp: (i) truy xuất tri thức từ kho tri thức, (ii) nạp/truy xuất bộ nhớ, hoặc (iii) điều

phối và thực thi công cụ pentest. Cách tổ chức này đảm bảo mọi quyết định điều phối đều nhất quán với ngữ cảnh hiện có và tránh việc các agent tự xử lý công việc ngoài phạm vi vai trò của mình.

Trước khi có thể truy xuất tri thức từ kho tri thức, hệ thống cần được nạp thêm dữ liệu từ pentester kèm metadata để phục vụ cho các tác vụ chuyên biệt. Khi tiếp nhận bất cứ tài liệu nào, hệ thống sẽ thực hiện quá trình tiền xử lý (bao gồm chuyển đổi về văn bản chuẩn và phân đoạn ngữ nghĩa) rồi sau đó sinh véc-tơ nhúng (dense, sparse, late interaction) trước khi nạp chúng cùng với đoạn chunk và metadata tương ứng vào kho tri thức.

Nếu yêu cầu là truy xuất tri thức, Core Orchestrator kích hoạt RAG Agent để thực hiện RAG theo cơ chế hybrid search kết hợp reranking, sau đó RAG Agent tổng hợp các đoạn tri thức phù hợp và trả kết quả về cho Core Orchestrator. Nếu yêu cầu liên quan đến bộ nhớ, Core Orchestrator gọi Memory Agent để (a) lưu/cập nhật core memory khi phát sinh thông tin mang tính chiến lược, hoặc (b) truy xuất các trí nhớ liên quan nhằm khôi phục ngữ cảnh. Ngoài ra, hệ thống cũng tự động gọi Memory Agent để tóm tắt và lưu các thông điệp bị loại khỏi message queue vào kho trí nhớ khi message queue đầy. Trong trường hợp yêu cầu là thực thi kỹ thuật, Core Orchestrator chuyển chỉ dẫn về mục tiêu và công cụ cho Pentest Agent; Pentest Agent diễn giải chỉ dẫn thành các lời gọi công cụ để gửi yêu cầu tới MCP Server. Tại đây, MCP Server sẽ thực thi các công cụ pentest trong môi trường sandbox và trả kết quả về cho Pentest Agent phân tích và tổng hợp. Sau khi nhận kết quả từ các agent, Core Orchestrator hợp nhất thông tin, đánh giá mức độ đáp ứng yêu cầu và tạo phản hồi cuối cùng cho Pentester; đồng thời, toàn bộ hội thoại, kết quả truy xuất tri thức và lời gọi công cụ đều được ghi log một cách đầy đủ vào Pentest Database nhằm phục vụ truy vết, hậu kiểm và báo cáo sau này.

2.3 Đặc tả chức năng

2.3.1 Đặc tả ca sử dụng Tiếp nhận yêu cầu

Trường	Nội dung
Tên ca sử dụng	Tiếp nhận yêu cầu
Mục tiêu	Tiếp nhận yêu cầu từ Pentester, khôi phục và chuẩn hóa ngữ cảnh cho phiên làm việc (core memory, recall memory và message queue), sau đó phân tích và điều phối yêu cầu tới các agent chuyên biệt hoặc tạo phản hồi trực tiếp.
Tác nhân	• Pentester: tác nhân bên ngoài gửi yêu cầu. • Core Orchestrator: tiếp nhận yêu cầu, tiếp nhận ngữ cảnh và điều phối tác vụ. • Memory Agent: được gọi khi cần truy xuất hoặc cập nhật bộ nhớ. • RAG Agent: được gọi khi yêu cầu cần truy xuất tri thức từ kho tri thức. • Pentest Agent: được gọi khi yêu cầu cần thực thi công cụ pentest.
Tiền điều kiện	• Phiên làm việc của Pentester đã được khởi tạo.
Hậu điều kiện	• Yêu cầu được gắn với ngữ cảnh hội thoại phù hợp. • Yêu cầu được phân loại chính xác và điều phối tới đúng agent. • Hệ thống tạo được phản hồi sơ bộ hoặc phản hồi cuối cùng cho Pentester.
Luồng sự kiện chính	1. Pentester gửi yêu cầu tới hệ thống. 2. Core Orchestrator tiếp nhận yêu cầu và tải ngữ cảnh hiện có, bao gồm core memory, recall memory (bản tóm tắt) và message queue. 3. Core Orchestrator phân tích nội dung yêu cầu và xác định loại tác vụ cần thực hiện. 4. Core Orchestrator tạo phản hồi gửi lại Pentester. 5. Toàn bộ quá trình được ghi log vào Pentest Database.
Luồng sự kiện phụ	4a. Tri thức không đủ: Core Orchestrator yêu cầu RAG Agent truy xuất tri thức. 4b. Ngữ cảnh không đầy đủ: Core Orchestrator yêu cầu Memory Agent truy xuất trí nhớ. 4c. Cần lưu core memory: Core Orchestrator yêu cầu Memory Agent lưu/cập nhật core memory. 4d. Cần thực thi công cụ: Core Orchestrator yêu cầu Pentest Agent thực thi công cụ pentest.

Bảng 2.1: Bảng đặc tả ca sử dụng Tiếp nhận yêu cầu

2.3.2 Đặc tả ca sử dụng Nạp dữ liệu vào kho tri thức

Trường	Nội dung
Tên ca sử dụng	Nạp dữ liệu vào kho tri thức
Mục tiêu	Tiếp nhận tài liệu từ Pentester, thực hiện tiền xử lý, phân đoạn ngữ nghĩa và sinh véc-tơ nhúng để lưu trữ vào kho tri thức, phục vụ cho các truy vấn RAG trong tương lai.
Tác nhân	• Pentester: cung cấp tài liệu và metadata. • RAG Agent: xử lý tài liệu, sinh embedding và nạp dữ liệu. • Kho tri thức: lưu trữ dữ liệu tri thức đã được véc-tơ hóa. • Pentest Database: lưu thông tin các tài liệu được nạp vào kho tri thức.
Tiền điều kiện	• Phiên làm việc của Pentester đã được khởi tạo. • Kho tri thức được kết nối và đã khởi tạo collection knowledge_base. • RAG Agent sẵn sàng để sinh véc-tơ embedding.
Hậu điều kiện	• Tài liệu được chuyển đổi sang dạng văn bản chuẩn. • Nội dung được chia nhỏ theo phân đoạn ngữ nghĩa. • Các đoạn chunk được sinh véc-tơ nhúng (dense, sparse, late interaction) và lưu vào kho tri thức kèm metadata. • Kết quả nạp tài liệu được ghi log để phục vụ cho công tác quản lý tri thức của agent.
Luồng sự kiện chính	1. Pentester gửi tài liệu cùng metadata tới hệ thống. 2. RAG Agent thực hiện chuyển đổi tài liệu sang dạng văn bản chuẩn nếu cần. 3. RAG Agent áp dụng phân đoạn ngữ nghĩa để chia nội dung theo ranh giới ngữ nghĩa. 4. RAG Agent sinh 3 loại biểu diễn véc-tơ nhúng cho từng đoạn chunk. 5. Dữ liệu được lưu vào kho tri thức kèm metadata tương ứng. 6. Hệ thống ghi log thông tin nạp tài liệu vào Pentest Database.
Luồng sự kiện phụ	2a. Tài liệu không hợp lệ hoặc không hỗ trợ định dạng: hệ thống từ chối nạp và trả thông báo lỗi cho Pentester.

Bảng 2.2: Bảng đặc tả ca sử dụng Nạp dữ liệu vào kho tri thức

2.3.3 Đặc tả ca sử dụng Nạp thông tin vào kho trí nhớ

Trường	Nội dung
Tên ca sử dụng	Nạp thông tin vào kho trí nhớ
Mục tiêu	Lưu trữ và cập nhật trí nhớ của hệ thống, bao gồm recall memory (tóm tắt hội thoại và các thông điệp bị loại khỏi message queue khi đầy) và core memory (thông tin dài hạn, chiến lược), nhằm duy trì ngữ cảnh xuyên suốt các phiên làm việc.
Tác nhân	- Core Orchestrator: gửi yêu cầu lưu hoặc cập nhật trí nhớ. - Memory Agent: xử lý, tóm tắt và quyết định vị trí lưu trí nhớ. - Kho trí nhớ: lưu trữ core memory và recall memory. - Pentest Database: lưu thống kê core memory cho phiên làm việc.
Tiền điều kiện	- Phiên làm việc đang hoạt động. - Message queue đã đạt ngưỡng hoặc xuất hiện thông tin cần lưu vào bộ nhớ dài hạn. - Kho trí nhớ sẵn sàng tiếp nhận dữ liệu mới.
Hậu điều kiện	- Recall memory được cập nhật từ các thông điệp bị loại khỏi message queue. - Các thông điệp bị loại khỏi message queue được lưu vào kho trí nhớ. - Core memory được thêm mới hoặc cập nhật nếu có yêu cầu. - Ngữ cảnh hệ thống được duy trì nhất quán cho các bước xử lý tiếp theo. - Thống kê về mức độ sử dụng bộ nhớ của core memory được lưu vào Pentest Database.
Luồng sự kiện chính	- Memory Agent phát hiện message queue đạt giới hạn dung lượng. - Các thông điệp cũ được loại bỏ khỏi message queue. - Memory Agent tóm tắt nội dung các thông điệp bị loại bỏ. - Nội dung tóm tắt và các thông điệp bị loại bỏ được lưu vào trong kho trí nhớ. - Core Orchestrator gửi yêu cầu lưu hoặc cập nhật core memory. - Memory Agent cập nhật core memory trong kho trí nhớ. - Hệ thống tính toán mức độ sử dụng bộ nhớ của core memory đối với phiên làm việc và lưu vào Pentest Database.
Luồng sự kiện phụ	5a. Khi được yêu cầu lưu core memory mới nhưng core memory đã đạt giới hạn về token: Memory Agent quyết định cập nhật core memory liên quan nhất với core memory cần lưu.

Bảng 2.3: Bảng đặc tả ca sử dụng Nạp thông tin vào kho trí nhớ

2.3.4 Đặc tả ca sử dụng Truy xuất dữ liệu từ kho tri thức

Trường	Nội dung
Tên ca sử dụng	Truy xuất dữ liệu từ kho tri thức
Mục tiêu	Truy xuất và tổng hợp các tài liệu tri thức liên quan từ Kho tri thức nhằm hỗ trợ trả lời câu hỏi, phân tích hoặc suy luận cho Core Orchestrator thông qua cơ chế RAG.
Tác nhân	• Core Orchestrator: gửi yêu cầu truy vấn tri thức. • RAG Agent: thực hiện truy xuất, lọc và tổng hợp dữ liệu. • Kho tri thức: lưu trữ các véc-tơ nhúng của tài liệu tĩnh đã được xử lý. • Pentest Database: lưu trữ thông tin truy xuất tài liệu tri thức.
Tiền điều kiện	• Core Orchestrator đã xác định yêu cầu cần truy xuất tri thức. • Kho tri thức đã có dữ liệu được xử lý trước đó. • RAG Agent ở trạng thái sẵn sàng.
Hậu điều kiện	• Các đoạn tri thức phù hợp được truy xuất và trả về cho Core Orchestrator. • Kết quả truy vấn được ghi log phục vụ truy vết và đánh giá.
Luồng sự kiện chính	1. Core Orchestrator gửi yêu cầu truy xuất tri thức kèm metadata liên quan. 2. RAG Agent tiếp nhận yêu cầu và chuẩn hóa truy vấn. 3. RAG Agent thực hiện hybrid search trên kho tri thức (kết hợp tìm kiếm ngữ nghĩa và tìm kiếm từ khóa). 4. Các kết quả truy xuất được đưa qua bước reranking để xếp hạng mức độ liên quan. 5. RAG Agent tổng hợp các đoạn tri thức phù hợp nhất và trả kết quả cho Core Orchestrator. 6. Hệ thống ghi log truy vấn và kết quả các đoạn truy vấn trích xuất được vào Pentest Database.
Luồng sự kiện phụ	3a. Không tìm thấy tài liệu phù hợp: RAG Agent trả về kết quả rỗng cho Core Orchestrator.

Bảng 2.4: Bảng đặc tả ca sử dụng Truy xuất dữ liệu từ kho tri thức

2.3.5 Đặc tả ca sử dụng Truy xuất thông tin từ kho trí nhớ

Trường	Nội dung
Tên ca sử dụng	Truy xuất thông tin từ kho trí nhớ
Mục tiêu	Truy xuất các trí nhớ liên quan (recall memory) nhằm khôi phục lại trí nhớ cần thiết cho yêu cầu của Orchestrator mà ngữ cảnh hiện tại (core memory, bản tóm tắt của recall memory và message queue) không có.
Tác nhân	- Core Orchestrator: gửi yêu cầu truy xuất trí nhớ. - Memory Agent: thực hiện tìm kiếm và tổng hợp trí nhớ. - Kho trí nhớ: lưu trữ recall memory.
Tiền điều kiện	- Core Orchestrator yêu cầu khôi phục ngữ cảnh. - Kho trí nhớ chứa các dữ liệu liên quan tương ứng với phiên làm việc cần truy xuất. - Memory Agent sẵn sàng xử lý yêu cầu.
Hậu điều kiện	Ngữ cảnh liên quan được truy xuất và tổng hợp về cho Core Orchestrator.
Luồng sự kiện chính	- Core Orchestrator gửi yêu cầu truy xuất trí nhớ. - Memory Agent tiếp nhận yêu cầu và chuẩn hóa câu truy vấn phục vụ cho quá trình RAG. - Memory Agent thực hiện RAG đơn giản (lọc theo phiên làm việc) trên kho trí nhớ để lấy danh sách trí nhớ liên quan. - Các trí nhớ được tổng hợp và chuẩn hóa thành ngữ cảnh thống nhất. - Memory Agent trả ngữ cảnh đã tổng hợp cho Core Orchestrator.
Luồng sự kiện phụ	3a. Không tìm thấy trí nhớ phù hợp: Memory Agent trả về ngữ cảnh rỗng cho Core Orchestrator.

Bảng 2.5: Bảng đặc tả ca sử dụng Truy xuất thông tin từ kho trí nhớ

2.3.6 Đặc tả ca sử dụng Điều phối công cụ pentest

Trường	Nội dung
Tên ca sử dụng	Điều phối công cụ pentest
Mục tiêu	Điều phối quá trình lựa chọn, cấu hình và thực thi công cụ pentest phù hợp với yêu cầu và ngữ cảnh hiện tại của hệ thống.
Tác nhân	- Core Orchestrator: đưa ra chỉ dẫn thực thi ở mức tổng quát và tiếp nhận kết quả. - Pentest Agent: diễn giải chỉ dẫn, lựa chọn công cụ và tham số phù hợp. - MCP Server: tiếp nhận yêu cầu thực thi công cụ và trả về output.
Tiền điều kiện	- Core Orchestrator yêu cầu thực thi công cụ pentest. - Pentest Agent ở trạng thái sẵn sàng. - MCP Server đang hoạt động và có thể nhận yêu cầu.
Hậu điều kiện	- Công cụ pentest được thực thi hoặc bị từ chối với thông báo lỗi phù hợp. - Kết quả thực thi được phân tích và trả về cho Core Orchestrator.
Luồng sự kiện chính	- Core Orchestrator gửi chỉ dẫn thực thi công cụ cùng với ngữ cảnh hiện có cho Pentest Agent. - Pentest Agent phân tích yêu cầu và ngữ cảnh. - Pentest Agent xác định công cụ pentest phù hợp từ kho công cụ được đăng ký. - Pentest Agent xây dựng lệnh thực thi cùng các tham số kỹ thuật cần thiết. - Pentest Agent gửi yêu cầu thực thi tới MCP Server thông qua giao thức HTTP streamable. - Pentest Agent nhận output trả về từ MCP Server trong suốt quá trình thực thi. - Pentest Agent phân tích kết quả sơ bộ và gửi lại cho Core Orchestrator.
Luồng sự kiện phụ	3a. Không xác định được công cụ phù hợp: Pentest Agent trả về thông báo lỗi và đề xuất các công cụ thay thế cho Core Orchestrator. 4a. Tham số không hợp lệ: Pentest Agent điều chỉnh tham số hoặc yêu cầu Core Orchestrator bổ sung thông tin. 6a. MCP Server không phản hồi hoặc kết nối thất bại: hệ thống trả thông báo lỗi, ghi log chi tiết và dừng luồng xử lý.

Bảng 2.6: Bảng đặc tả ca sử dụng Điều phối công cụ pentest

2.3.7 Đặc tả ca sử dụng Thực thi công cụ pentest

Trường	Nội dung
Tên ca sử dụng	Thực thi công cụ pentest
Mục tiêu	Thực thi công cụ pentest theo yêu cầu từ Pentest Agent trong môi trường sandbox và trả kết quả thực thi về cho hệ thống.
Tác nhân	• MCP Server: tiếp nhận yêu cầu thực thi, quản lý sandbox và điều phối tiến trình công cụ. • Pentest Tools: các công cụ pentest như Nmap, Metasploit, lệnh hệ thống. • Pentest Agent: nhận kết quả thực thi từ MCP Server.
Tiền điều kiện	• MCP Server đã được khởi tạo và sẵn sàng nhận yêu cầu từ Pentest Agent. • Các công cụ pentest cần thiết đã được cài đặt trong môi trường sandbox. • Yêu cầu thực thi đã được chuẩn hóa và xác thực (thông qua authentication token).
Hậu điều kiện	• Công cụ pentest được thực thi thành công hoặc bị dừng với thông báo lỗi phù hợp. • Output của công cụ được truyền về cho Pentest Agent theo dạng luồng.
Luồng sự kiện chính	1. MCP Server nhận yêu cầu thực thi từ Pentest Agent và kiểm tra tính hợp lệ của tên công cụ và tham số. 2. MCP Server chạy công cụ trong môi trường sandbox. 3. MCP Server gửi các dữ liệu output về cho Pentest Agent thông qua giao thức HTTP streamable ngay khi dữ liệu được sinh ra. 4. Khi công cụ hoàn tất, MCP Server gửi kết quả cuối cùng, đóng luồng truyền và ghi nhận trạng thái thực thi.
Luồng sự kiện phụ	1a. Công cụ không tồn tại trong kho công cụ đã đăng ký: MCP Server trả về lỗi và không khởi chạy tiến trình. 1b. Tham số không hợp lệ hoặc thiếu: MCP Server trả thông báo lỗi mô tả chi tiết và kết thúc ca sử dụng. 2a. Công cụ vượt quá thời gian cho phép (timeout): MCP Server dừng tiến trình, đánh dấu trạng thái timeout và đóng luồng truyền.

Bảng 2.7: Bảng đặc tả ca sử dụng Thực thi công cụ pentest

2.4 Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống LPAA cần đáp ứng các yêu cầu phi chức năng nhằm bảo đảm hệ thống có thể vận hành ổn định, an toàn và hiệu quả trong môi trường pentest nội bộ. Về hiệu năng, hệ thống phải đảm bảo độ trễ thấp trong các truy vấn tri thức, đặc biệt là các truy vấn yêu cầu sự làm việc của hệ thống RAG. Bên cạnh đó, cơ chế xử lý dữ liệu đầu ra từ công cụ pentest cần được tối ưu theo dạng luồng (streaming). Điều này giúp hệ thống vận hành mượt mà, không chỉ giúp pentester theo dõi quá trình thực thi công cụ mà còn cải thiện đáng kể trải nghiệm của người dùng.

Về độ tin cậy và tính ổn định, hệ thống cần duy trì hoạt động liên tục trong các phiên pentest kéo dài. Khi dung lượng bộ nhớ tăng lên hoặc số lượng yêu cầu xử lý nhiều hơn, hệ thống vẫn phải hoạt động ổn định và không bị treo. Trong trường hợp xảy ra lỗi ở một tác nhân cụ thể, chẳng hạn như Pentest Agent hoặc MCP Server, hệ thống cần xử lý lỗi một cách an toàn, không làm ảnh hưởng đến các thành phần còn lại và cung cấp thông báo rõ ràng cho người dùng. Việc thực thi các công cụ pentest phải được thực hiện trong môi trường sandbox nhằm hạn chế rủi ro ảnh hưởng đến tiến trình chạy agent.

Bảo mật là yêu cầu bắt buộc đối với hệ thống. Toàn bộ dữ liệu, bao gồm tài liệu, tri thức và bộ nhớ hội thoại, phải được xử lý và lưu trữ hoàn toàn cục bộ. Các thành phần trong hệ thống chỉ được phép giao tiếp trong phạm vi nội bộ và phải được giới hạn quyền truy cập rõ ràng. Việc thực thi công cụ pentest cũng cần được kiểm soát chặt chẽ để tránh nguy cơ lạm dụng hoặc gây mất an toàn.

Tính khả dụng là yêu cầu then chốt để đảm bảo hiệu quả hỗ trợ trực tiếp cho các pentester. Hệ thống cần cho phép tương tác bằng ngôn ngữ tự nhiên, giúp người dùng vận hành dễ dàng mà không cần kiến thức quá chuyên sâu về AI. Kết quả đầu ra, đặc biệt là dữ liệu từ công cụ, phải được cấu trúc hóa và trình bày trực quan. Ngoài ra, việc nạp tài liệu vào hệ thống cũng cần đơn giản, hỗ trợ các định dạng file phổ biến để tiết kiệm thời gian cho người sử dụng.

Về khả năng bảo trì, hệ thống cần được thiết kế theo hướng mô-đun hóa, trong đó các tác nhân như Core Orchestrator, RAG Agent, Memory Agent và Pentest Agent có thể được cập nhật hoặc thay thế mà không ảnh hưởng đến các thành phần khác. Các cấu hình quan trọng như mô hình ngôn ngữ, cơ sở dữ liệu và tham số hệ thống cần được quản lý tập trung và rõ ràng thông qua cấu hình môi trường.

Cuối cùng, hệ thống cần có khả năng mở rộng linh hoạt trong tương lai. Việc bổ sung công cụ pentest hay các agent chuyên biệt mới, thay đổi mô hình ngôn ngữ hoặc nâng cấp hạ tầng lưu trữ phải được thực hiện dễ dàng mà không làm thay đổi

kiến trúc tổng thể. Hệ thống cũng cần hỗ trợ cơ chế giám sát và ghi log theo từng giai đoạn xử lý, giúp đảm bảo tính minh bạch và thuận tiện cho việc đánh giá hiệu quả vận hành, kiểm tra và khắc phục sự cố khi cần thiết.

Kết chương

Tổng kết lại, chương này đã định hình phạm vi và yêu cầu hệ thống thông qua việc phân tích thực tế và xây dựng các biểu đồ nghiệp vụ. Đây là cơ sở để Chương 3 trình bày chi tiết về công nghệ và các mô hình kiến trúc sẽ được sử dụng để triển khai hệ thống.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Dựa trên yêu cầu từ Chương 2, chương này trình bày các giải pháp công nghệ cốt lõi để xây dựng LPAA. Tiêu chí lựa chọn ưu tiên sự phù hợp thực tế hơn độ phổ biến, tập trung giải quyết bốn bài toán trọng yếu: vận hành offline-first, ứng dụng RAG, quản lý bộ nhớ đa tầng (ngắn/dài hạn) và điều phối công cụ an toàn. Mỗi lựa chọn công nghệ đều được đối chiếu với các giải pháp tương đương để từ đó làm rõ lý do đề xuất.

3.1 LangChain - Khung xây dựng tác tử và luồng xử lý ngôn ngữ

3.1.1 Khái quát công nghệ

LangChain [4] được lựa chọn làm khung phát triển cho hệ thống nhờ khả năng cung cấp các lớp trừu tượng hóa mạnh mẽ để xây dựng các luồng xử lý ngôn ngữ và kiến trúc AI agent phức tạp. Về mặt cấu trúc, framework này tập trung vào sáu thành phần chính [5]: khả năng tích hợp mô hình ngôn ngữ lớn (importing language models), mẫu câu lệnh (prompt templates) giúp chuẩn hóa đầu vào, chuỗi xử lý (chains) để liên kết quy trình, chỉ mục (indexes) phục vụ tổ chức dữ liệu, công cụ (tools) cung cấp giao diện tương tác với công cụ ngoại vi, và bộ nhớ (memory) để duy trì ngữ cảnh. Hệ thống các lớp trừu tượng này đóng vai trò quan trọng trong việc giảm thiểu chi phí phát triển hệ thống, đồng thời tối ưu hóa khả năng tái sử dụng mã nguồn trong các kịch bản sử dụng khác nhau.

LangChain cho phép tích hợp hầu như mọi LLM thông qua giao diện chuẩn LLM class. Việc nạp một LLM vào LangChain diễn ra vô cùng đơn giản, chủ yếu chỉ cần API key của nhà cung cấp, nhờ đó hệ thống có thể linh hoạt thay đổi hoặc kết hợp nhiều mô hình ngôn ngữ khác nhau. Bên cạnh các mô hình nền tảng phổ biến (đặc biệt là các mô hình mã nguồn mở có thể truy cập qua các nền tảng như Hugging Face, Ollama), LangChain còn cho phép bọc và tích hợp mô hình một cách tùy ý thông qua các wrapper như CustomLLM, phù hợp với các trường hợp dùng mô hình tự triển khai hoặc đã được tinh chỉnh theo nhu cầu riêng.

Trong khâu xử lý đầu vào, PromptTemplate của LangChain thay thế phương pháp hard-code truyền thống bằng cơ chế tạo mẫu, giúp việc tùy biến theo nghiệp vụ trở nên dễ dàng hơn. Nhờ đó, cấu trúc prompt cũng được tổ chức khoa học hơn, phân tách rõ ràng giữa chỉ dẫn ngữ cảnh, các biến dữ liệu đầu vào và định dạng đầu ra mong muốn. Khi cần tăng độ chính xác, hệ thống cũng có thể thêm vài ví dụ minh họa ngay trong prompt (sử dụng few-shot prompting) để mô hình hiểu rõ nhiệm vụ và kết quả đầu ra cần đạt được. Cách làm này khiến prompt dễ tái sử dụng, dễ chỉnh sửa, và thuận tiện hơn trong việc tích hợp vào các bước xử lý tiếp

theo của luồng thực thi.

Khái niệm Chain được sử dụng để mô hình hóa luồng xử lý dưới dạng chuỗi các bước thực thi, cho phép kết nối LLM với các thành phần khác nhau theo một trật tự xác định. Ứng dụng đơn giản nhất của Chain là việc gắn một PromptTemplate cho một LLM, lúc này luồng thực thi sẽ gồm các bước cơ bản như sau: hệ thống sẽ nhận đầu vào, đưa vào prompt đã định nghĩa, gọi mô hình và trả về kết quả. Từ đây, các chuỗi tuần tự phức tạp hơn (như SimpleSequentialChain) được xây dựng để hiện thực hóa các quy trình đa công đoạn, trong đó kết quả của bước xử lý trước trở thành đầu vào cho bước xử lý sau. Cách làm này cũng gia tăng khả năng tùy biến của hệ thống, khi mà mỗi công đoạn có thể được tùy chỉnh với các công cụ, prompt hay thậm chí là các mô hình ngôn ngữ khác nhau.

Để giải quyết bài toán kết nối tri thức từ các nguồn dữ liệu bên ngoài tập huấn luyện, LangChain cung cấp hệ thống Indexes bao gồm các bộ nạp tài liệu (Document Loaders), bộ phân tách văn bản (Text Splitters) và cơ chế tích hợp với các kho lưu trữ véc-tơ. Quy trình này bắt đầu bằng việc thu thập dữ liệu từ nhiều nguồn khác nhau, sau đó phân chia văn bản thành các đoạn nhỏ nhằm tối ưu hóa tốc độ và chi phí xử lý. Cuối cùng, dữ liệu được lưu trữ dưới dạng véc-tơ nhúng kèm metadata trong các kho lưu trữ véc-tơ để phục vụ quá trình tìm kiếm sau này.

Sau khi dữ liệu được lập chỉ mục, thành phần Retriever đảm nhận vai trò truy vấn và trích xuất các đối tượng Document phù hợp dựa trên ngữ nghĩa của đầu vào. Đây là thành phần then chốt trong mô hình RAG, cho phép hệ thống tổng hợp câu trả lời dựa trên bằng chứng thực tế thay vì chỉ dựa vào tập tri thức được huấn luyện của mô hình. Bên cạnh RAG truyền thống, kiến trúc này còn hỗ trợ agentic RAG, nơi tác tử có khả năng chủ động gọi công cụ để kiểm chứng thông tin sau khi truy xuất.

Nhằm khắc phục các giới hạn cố hữu của mô hình ngôn ngữ về khả năng tính toán chính xác và tương tác với môi trường, thành phần Tools được định nghĩa như các hàm chức năng bên ngoài được mô tả chi tiết qua schema (tên, mô tả, tham số). Về mặt kiến trúc, đây là cầu nối giữa tầng suy luận và tầng thực thi, cho phép chuyển đổi agent từ một mô hình sinh văn bản thuần túy thành một hệ thống có khả năng hành động cụ thể (như chạy công cụ pentest hay truy vấn hệ thống).

Bên cạnh giới hạn kể trên, các mô hình ngôn ngữ lớn còn tồn tại nhược điểm về khả năng lưu trữ trạng thái. Về bản chất, LLM hoạt động dựa trên cơ chế phi trạng thái (stateless), nên nếu muốn mô hình nhớ được lịch sử cuộc hội thoại thì ta phải đưa phần lịch sử đó vào cùng câu truy vấn hiện tại. LangChain giải quyết hạn chế này bằng cách sử dụng các hàm tiện ích để bổ sung bộ nhớ cho hệ thống, cho phép

lựa chọn nhiều mức độ lưu trữ khác nhau: có thể giữ toàn bộ cuộc hội thoại, chỉ giữ bản tóm tắt những gì đã trao đổi, hoặc đơn giản là lưu lại một số lượt trao đổi gần nhất.

3.1.2 Vai trò trong hệ thống

Trong hệ thống đề xuất ở Chương 2, LangChain đóng vai trò là lớp tích hợp giữa LLM và các mô-đun chức năng như RAG, bộ nhớ và công cụ. Cụ thể, LangChain được sử dụng để: (i) chuẩn hóa luồng tiếp nhận yêu cầu và diễn giải ý định thông qua cơ chế prompt và parser, giúp hệ thống chuyển từ yêu cầu tự nhiên sang kế hoạch hành động có cấu trúc; (ii) kết nối mô-đun truy xuất tri thức vào luồng sinh đáp án, phục vụ ca sử dụng “Truy xuất dữ liệu từ kho tri thức” và “Truy xuất trí nhớ từ kho trí nhớ”; (iii) chuẩn hóa giao diện tool-calling ở phía agent, cho phép agent gọi công cụ một cách nhất quán. Đối với hệ thống LPAA, LangChain đã vượt xa vai trò của một thư viện chatbot thông thường để trở thành nền tảng xây dựng kiến trúc luồng xử lý vững chắc cho hệ thống đa tác tử.

3.1.3 Lí do lựa chọn công nghệ

Trong quá trình khảo sát các nền tảng phát triển tương tự, một số hướng tiếp cận thay thế tiềm năng đã được xem xét, bao gồm Haystack [6] với thế mạnh về xây dựng luồng truy xuất và hỏi đáp (QA), và LlamaIndex [7] tập trung chuyên sâu vào khả năng kết nối dữ liệu, tối ưu hóa chỉ mục và hỗ trợ xây dựng các ứng dụng AI phức tạp. Tuy nhiên, trong phạm vi đề án này, LangChain được xác định là lựa chọn tối ưu dựa trên ba đặc điểm kỹ thuật chính. Thứ nhất, hệ sinh thái tích hợp phong phú của nền tảng này giúp giảm thiểu đáng kể thời gian phát triển. Thứ hai, khả năng trừu tượng hóa cao cho phép LangChain vận hành như một “lớp keo” kết nối các thành phần mà không áp đặt bất kỳ khuôn mẫu kiến trúc agent cố định nào lên hệ thống. Quan trọng hơn cả, LangChain đóng vai trò nền tảng hạ tầng thiết yếu để triển khai LangGraph (sẽ được trình bày chi tiết tại Mục 3.2), qua đó đáp ứng trọn vẹn yêu cầu về một hệ thống có khả năng điều phối các quy trình đa bước và mang tính lặp lại.

3.2 LangGraph - Kiến trúc điều phối agent theo đồ thị

3.2.1 Khái quát công nghệ

LangGraph [8] được phát triển như một phần mở rộng chiến lược của hệ sinh thái LangChain, hướng tới việc khắc phục các rào cản cố hữu của mô hình xử lý tuyến tính truyền thống. Đây là giải pháp tối ưu để triển khai các AI agent tinh vi, đặc trưng bởi các luồng công việc lặp lại và sự phụ thuộc chặt chẽ vào trạng thái ngữ cảnh. Thay vì gò bó luồng xử lý trong một chuỗi tuần tự cố định, LangGraph tiếp cận bài toán kiến trúc thông qua mô hình đồ thị trạng thái (stateful graph). Trong

mô hình này, mỗi bước xử lý nghiệp vụ được biểu diễn dưới dạng một nút (node), trong khi luồng điều khiển và logic chuyển trạng thái được định nghĩa thông qua hệ thống các cạnh có điều kiện (conditional edges), cho phép hệ thống vận hành linh hoạt và phi tuyến tính.

LangGraph mô hình hóa toàn bộ hệ thống agent dựa trên nền tảng kiến trúc đồ thị trạng thái (Stateful Graph Architecture), nơi mỗi nút được định nghĩa là một bước tính toán độc lập hoặc một agent con chuyên biệt. Điểm đặc thù của kiến trúc này nằm ở cơ chế quản lý trạng thái toàn cục (global state) được truyền tải xuyên suốt quá trình thực thi, bao gồm đa dạng các loại dữ liệu từ yêu cầu khởi tạo, kết quả trung gian, thông tin truy xuất đến bộ nhớ hội thoại và đầu ra của công cụ. Thông qua khả năng lưu giữ trạng thái này, đồ thị đảm bảo duy trì tính liên tục của ngữ cảnh, đồng thời cho phép thiết lập các luồng xử lý phức tạp trong đó kết quả của các bước sau phụ thuộc chặt chẽ vào dữ liệu đầu ra của các bước trước.

Trong kiến trúc LangGraph, nút được định nghĩa là đơn vị thực thi cơ sở, đại diện cho đa dạng các tác vụ xử lý từ lời gọi mô hình ngôn ngữ (LLM call), bước truy xuất tri thức (RAG), đến các thao tác cập nhật bộ nhớ hoặc thực thi công cụ. Các nút này hoạt động như những agent độc lập với phạm vi trách nhiệm được phân định rõ ràng; ví dụ điển hình là ToolNode chuyên trách quản lý các tương tác công cụ, trong khi các nút suy luận chỉ tập trung vào việc phân tích và ra quyết định.

Song song với đó, hệ thống cạnh (Edges) và cơ chế chuyển trạng thái không đơn thuần là các liên kết tĩnh mà được hiện thực hóa thông qua các hàm logic điều kiện (conditional functions). Dựa trên kết quả đánh giá trạng thái hiện tại của đồ thị, các cạnh này đóng vai trò điều hướng luồng điều khiển và xác định bước xử lý kế tiếp. Cơ chế định tuyến động này cho phép hệ thống hỗ trợ các kịch bản rẽ nhánh phức tạp và thay đổi hành vi của agent linh hoạt theo ngữ cảnh, khắc phục hoàn toàn sự cứng nhắc của các luồng xử lý tuyến tính truyền thống.

Một tính năng ưu việt khác của LangGraph là khả năng hỗ trợ tích hợp cơ chế giám sát và tương tác của con người (human-in-the-loop) trực tiếp vào vòng lặp xử lý. Thay vì vận hành hoàn toàn tự động, hệ thống cho phép thiết lập các điểm ngắt (breakpoints), nơi quy trình thực thi có thể tạm dừng để chờ sự phê duyệt hoặc điều chỉnh tham số từ phía người điều hành. Cơ chế này giúp hiện thực hóa mô hình cộng tác “người - máy”, dung hòa giữa tốc độ suy luận của AI với trực giác và kinh nghiệm của con người.

Về mặt quản trị và vận hành, LangGraph cung cấp hệ thống API toàn diện, cho phép can thiệp sâu vào cấu trúc đồ thị để thực hiện các thao tác như tinh chỉnh nút, kiểm soát luồng điều khiển và truy xuất trạng thái của quy trình theo thời gian thực.

Đặc biệt, giá trị của nền tảng được nâng cao đáng kể thông qua việc tích hợp mạnh mẽ với LangSmith, cung cấp năng lực giám sát chi tiết luồng thực thi, hỗ trợ gỡ lỗi và tối ưu hóa hiệu năng của agent. Sự kết hợp chặt chẽ này đảm bảo tính minh bạch tối đa cho các quy trình nghiệp vụ phức tạp, tạo thuận lợi cho việc phát triển cũng như duy trì sự ổn định trong giai đoạn vận hành thực tế.

3.2.2 Vai trò trong hệ thống

Theo đặc tả ở Chương 2, một phiên pentest thường không kết thúc sau một lần suy luận, mà diễn ra theo một vòng lặp khép kín, từ thu thập thông tin đến phân tích và chạy công cụ, cập nhật phát hiện rồi điều chỉnh chiến lược. Trong vòng lặp này, LangGraph đóng vai trò là điều phối viên trung tâm cho ba kịch bản chính: (i) quản lý vòng lặp suy luận và thực thi trong ca sử dụng “Điều phối công cụ pentest”; (ii) kích hoạt cơ chế lưu trữ tri thức trong ca sử dụng “Nạp thông tin và kho trí nhớ”; và (iii) định tuyến yêu cầu chính xác tới từng agent chuyên trách (RAG, Pentest, Memory) dựa trên ngữ cảnh. Từ đó, LangGraph giúp biến quá trình pentest từ “hành vi tự phát” của một agent thành một quy trình có cấu trúc, dễ theo dõi, dễ gỡ lỗi và dễ mở rộng.

3.2.3 Lí do lựa chọn công nghệ

Hướng tiếp cận thay thế thường gặp là agent loop theo kiểu AutoGPT [9], hoặc sử dụng chuỗi agent tuyến tính trong LangChain (một chain dài). Các cách tiếp cận này tuy đơn giản để khởi tạo nhưng thường gặp hạn chế lớn về khả năng kiểm soát điều kiện dừng và truy vết trạng thái. Ngược lại, LangGraph giải quyết triệt để các vấn đề này nhờ cơ chế quản lý đồ thị tường minh. Trong hệ thống LPAA, việc áp dụng các đặc tính của LangGraph (như đã trình bày tại mục 3.2.1) đã tạo nên cơ sở hạ tầng multi-agent vững chắc, giúp loại bỏ hoàn toàn tính bất định của mô hình single-loop truyền thống, đồng thời đáp ứng trọn vẹn các yêu cầu chức năng đã đề ra tại Chương 2.

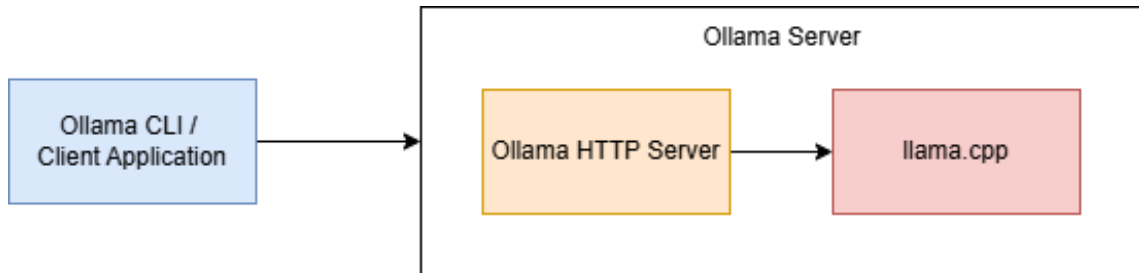
3.3 Ollama - Nền tảng triển khai và vận hành LLM cục bộ

3.3.1 Khái quát công nghệ

Ollama là một nền tảng chạy mô hình ngôn ngữ lớn theo hướng “offline-first”, cho phép người dùng tải về và chạy các LLM trực tiếp trên máy tính cá nhân thay vì phụ thuộc vào các dịch vụ thương mại trên nền tảng đám mây. Hệ thống được thiết kế để vận hành như một ứng dụng chạy nền, cung cấp đồng thời giao diện dòng lệnh (CLI) và hệ thống API nhằm hỗ trợ khả năng tích hợp vào các ứng dụng AI agent chuyên biệt.

Về kiến trúc, Ollama áp dụng mô hình client-server, giao tiếp với nhau qua giao thức HTTP truyền thống. Bên trong Ollama Server, có hai thành phần cốt lõi.

Thành phần thứ nhất là Ollama HTTP Server, chịu trách nhiệm tiếp nhận yêu cầu từ client, điều phối tác vụ và trả kết quả. Thành phần thứ hai là llama.cpp, đóng vai trò là bộ não suy luận (inference engine), giúp nạp mô hình và thực hiện suy luận đối với các yêu cầu được cung cấp. Điểm đáng chú ý là sự giao tiếp giữa Ollama HTTP Server và llama.cpp cũng được thực hiện qua HTTP, thể hiện thiết kế mô-đun hoá, tách rời lớp điều phối khỏi lớp suy luận, qua đó tăng tính linh hoạt khi mở rộng hoặc thay thế các thành phần trong hệ thống.



Hình 3.1: Mô tả kiến trúc tổng quát của Ollama

Để tối ưu hóa hiệu năng trên các cấu hình máy trạm phổ thông, Ollama đã ứng dụng kỹ thuật lượng tử hóa (quantization) vào kiến trúc của mình. Về bản chất, đây là quá trình chuyển đổi trọng số mô hình từ định dạng có độ chính xác cao sang định dạng có độ chính xác thấp hơn. Quá trình này tương tự như nguyên lý sử dụng hằng số π để đại diện cho chuỗi số vô hạn nhằm đơn giản hóa quá trình tính toán. Việc áp dụng kỹ thuật này đã giúp Ollama giảm tải đáng kể nhu cầu về bộ nhớ cũng như năng lực xử lý, cho phép các mô hình phức tạp vận hành mượt mà ngay trên các máy trạm tiêu chuẩn mà không yêu cầu GPU cao cấp. Đặc biệt, sự kết hợp với llama.cpp, động cơ suy luận mã nguồn mở nổi tiếng về tính tương thích đa nền tảng, đã mở rộng khả năng triển khai LLM xuống cả các thiết bị hạn chế tài nguyên như Raspberry Pi, thậm chí còn có thể hoạt động tốt chỉ với CPU.

Việc triển khai LLM cục bộ bằng Ollama mang lại các lợi ích trực tiếp về vận hành và quản trị dữ liệu. Hệ thống hỗ trợ khả năng sử dụng offline, đảm bảo tính sẵn sàng khi không có kết nối Internet. Quan trọng hơn, toàn bộ dữ liệu đầu vào/đầu ra và ngữ cảnh xử lý đều được lưu giữ trên thiết bị người dùng. Điều này đảm bảo được quyền riêng tư và tính bảo mật của dữ liệu, qua đó hoàn toàn phù hợp với các kịch bản xử lý thông tin hoặc các môi trường có yêu cầu nghiêm ngặt về quản trị dữ liệu.

Ngoài khả năng chạy mô hình, Ollama còn cung cấp cơ chế Modelfile như một lớp cấu hình và tinh chỉnh hành vi của LLM. Modelfile là tệp cấu hình dạng văn bản mô tả cách xây dựng và thiết lập một phiên bản mô hình. Với tệp này, người dùng có thể tùy ý lựa chọn mô hình gốc (base model), khai báo chỉ thị hệ thống mặc

định (default system prompt), tích hợp cấu hình tinh chỉnh theo LoRA (Low-Rank Adaptation), và tùy chỉnh các tham số suy luận như temperature, top-P, top-K,...

3.3.2 Vai trò trong hệ thống

Như đã xác định trong phần yêu cầu phi chức năng tại Chương 2, tính bảo mật dữ liệu và khả năng vận hành offline-first là hai yếu tố bắt buộc của hệ thống. Ollama đáp ứng trọn vẹn các tiêu chí này thông qua cơ chế thực thi suy luận LLM hoàn toàn trên môi trường cục bộ, giúp loại bỏ sự phụ thuộc vào bên thứ ba và ngăn chặn nguy cơ rò rỉ thông tin. Trong kiến trúc đề xuất, Ollama đóng vai trò là lớp nền tảng thực thi cốt lõi để: (i) điều phối tác vụ, ra quyết định sử dụng cũng như diễn giải kết quả đầu ra của công cụ; và (ii) thực hiện các tác vụ tóm tắt phục vụ cho cơ chế quản lý bộ nhớ dài hạn/ngắn hạn.

3.3.3 Lí do lựa chọn công nghệ

Ollama được lựa chọn làm nền tảng thực thi LLM cục bộ nhờ đạt được sự cân bằng giữa hiệu năng, tính linh hoạt và khả năng triển khai. Khi đặt lên bàn cân so sánh, mặc dù vLLM [10] sở hữu hiệu suất suy luận vượt trội trên các hệ thống GPU mạnh, nhưng lại đi kèm rào cản lớn về độ phức tạp trong cài đặt và thiếu sự linh hoạt khi cần thay đổi nhanh mô hình. Trái lại, LM Studio tuy thân thiện với giao diện trực quan, nhưng định hướng chính là phục vụ cho người dùng đầu cuối (end user), dẫn đến hạn chế về khả năng tùy biến và khó tích hợp vào các quy trình tự động hóa (headless).

Ollama khắc phục triệt để các nhược điểm trên bằng một quy trình cài đặt tinh gọn và khả năng tương thích với nhiều loại phần cứng, cho phép vận hành trơn tru trên cả CPU, GPU hay kết hợp giữa CPU-GPU. Đặc biệt, với sự hỗ trợ mạnh mẽ cho các mô hình quantization, cơ chế tạo mô hình linh hoạt thông qua Modelfile và khả năng tích hợp sâu với hệ sinh thái LangChain, Ollama được lựa chọn là giải pháp phù hợp nhất để xây dựng các AI agent chạy cục bộ.

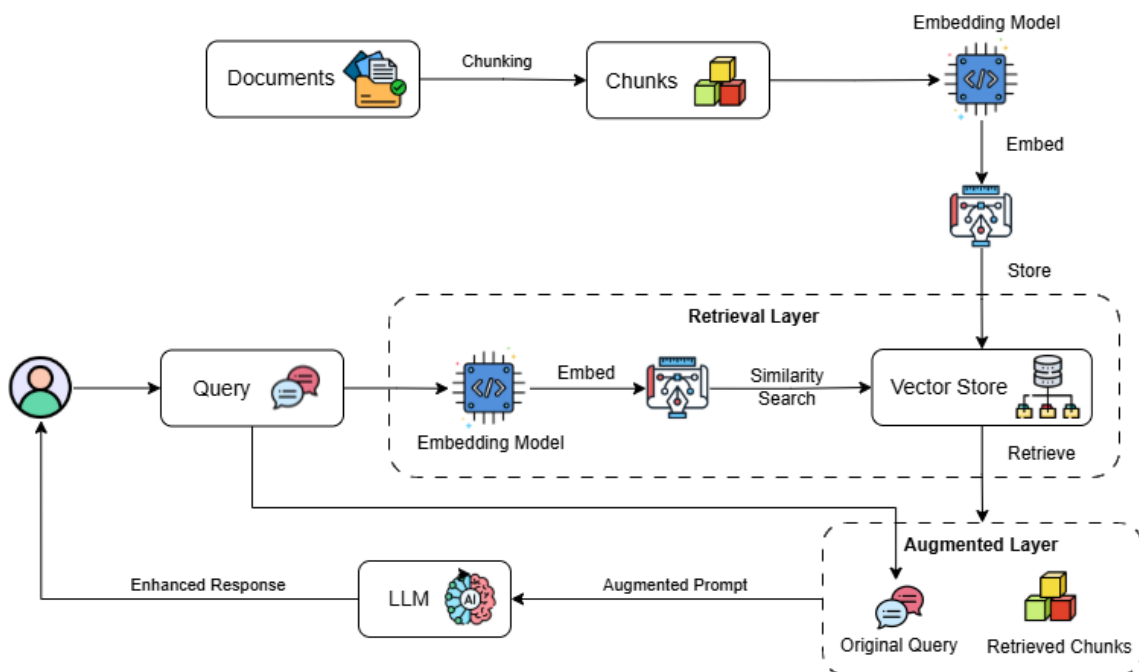
3.4 RAG - Retrieval Augmented Generation

3.4.1 Khái quát kỹ thuật

Retrieval-Augmented Generation (RAG) là một kiến trúc lai ghép, kết hợp sức mạnh của mô hình ngôn ngữ tạo sinh (generative models) với hệ thống truy xuất thông tin nhằm tối ưu hóa độ chính xác và tính kiểm chứng của nội dung được tạo ra. Thay vì phụ thuộc hoàn toàn vào nguồn tri thức tham số hóa (parametric knowledge) vốn bị giới hạn bởi tập dữ liệu tại thời điểm huấn luyện, RAG thiết lập một lớp truy xuất ngoại vi, cho phép hệ thống tham chiếu trực tiếp và tức thời đến các kho tài liệu, cơ sở dữ liệu hoặc ghi chú nội bộ trong quá trình suy luận. Cách tiếp cận này mang lại ý nghĩa thiết thực đối với các lĩnh vực đòi hỏi tri thức chuyên

sâu và cập nhật liên tục như lĩnh vực an toàn thông tin, nơi tính thời sự của dữ liệu đóng vai trò quyết định đến hiệu quả và độ tin cậy của tác vụ.

Xét về khía cạnh kiến trúc, quy trình vận hành của một hệ thống RAG tiêu chuẩn bao gồm một chuỗi các bước xử lý tuần tự và khép kín. Bắt đầu từ việc tiếp nhận câu truy vấn của người dùng, tầng truy xuất thực hiện chuyển đổi yêu cầu sang dạng biểu diễn véc-tơ nhằm xác định và trích xuất các nguồn tri thức liên quan từ kho dữ liệu. Các kết quả này sau đó được chuyển tiếp đến tầng tích hợp để xây dựng câu lệnh tăng cường (augmented prompt), kết hợp chặt chẽ giữa truy vấn gốc và tài liệu vừa thu thập. Cuối cùng, mô hình ngôn ngữ thực hiện quá trình suy luận dựa trên dữ liệu đầu vào để sinh nội dung phản hồi. Toàn bộ chu trình này phản ánh chính xác ba trụ cột nguyên lý cốt lõi của giải pháp: Truy xuất (Retrieve), Tăng cường (Augment) và Tạo sinh (Generate).



Hình 3.2: Mô tả quy trình RAG

Hệ thống RAG vận hành dựa trên một kho tri thức ngoại vi chứa đựng các dữ liệu phi cấu trúc phong phú từ tài liệu kỹ thuật, mã nguồn tới các báo cáo pentest. Để đưa nguồn dữ liệu này vào luồng xử lý, hệ thống áp dụng kỹ thuật nhúng (embedding) nhằm chuyển đổi văn bản thành các véc-tơ trong không gian đa chiều. Tuy nhiên, trước khi thực hiện quá trình nhúng, dữ liệu cần trải qua quy trình phân đoạn (chunking) để đảm bảo tính tương thích với giới hạn cửa sổ ngữ cảnh. Do yêu cầu này, việc xác định kích thước phân đoạn trở thành bài toán đánh đổi then chốt: kích thước quá lớn dễ lẫn thông tin nhiễu gây giảm độ chính xác, trong khi kích thước quá nhỏ lại khiến ngữ nghĩa bị phân mảnh và mất đi ngữ cảnh

cần thiết.

Tầng truy xuất đóng vai trò then chốt trong việc xác định và trích xuất các nguồn dữ liệu có độ tương quan cao nhất từ kho tri thức dựa trên câu truy vấn đầu vào. Cơ chế vận hành của tầng này là kỹ thuật tìm kiếm ngữ nghĩa (semantic search) trong không gian véc-tơ, nơi các truy vấn được véc-tơ hóa và đối sánh với cơ sở dữ liệu thông qua các thuật toán đo lường độ tương đồng như Cosine Similarity hoặc Dot Product. Sự ưu việt của phương pháp này so với tìm kiếm từ khóa (lexical search) truyền thống nằm ở khả năng nắm bắt sắc thái ngữ nghĩa, cho phép hệ thống nhận diện chính xác các mối liên hệ của nội dung ngay cả khi không xuất hiện sự trùng khớp về mặt từ ngữ, qua đó gia tăng đáng kể độ bao phủ và tính linh hoạt của quá trình truy xuất thông tin.

Tầng tích hợp giữ vị trí trung tâm trong kiến trúc RAG, đảm nhiệm chức năng điều phối luồng dữ liệu giao thoa giữa thành phần truy xuất và thành phần tạo sinh. Tại đây, các phân đoạn tài liệu được trích xuất sẽ được hợp nhất một cách có cấu trúc với truy vấn gốc để kiến tạo nên các mẫu câu lệnh tăng cường (augmented prompt). Quá trình này đóng vai trò quyết định trong việc cung cấp một ngữ cảnh đầu vào giàu thông tin, làm tiền đề vững chắc để mô hình ngôn ngữ tạo ra các phản hồi chính xác và sát với thực tế.

Đối với thành phần tạo sinh, hệ thống tận dụng sức mạnh của mô hình ngôn ngữ lớn để tổng hợp tri thức từ ngữ cảnh được cung cấp, từ đó tạo nên câu trả lời hoàn chỉnh. Việc sử dụng RAG để bổ sung dữ liệu thực tế từ kho tri thức giúp mô hình giảm thiểu đáng kể hiện tượng “ảo giác” (hallucination [11]), đảm bảo tính chính xác cao và sự tương thích với dữ liệu đặc thù trong từng trường hợp, qua đó khắc phục hiệu quả các giới hạn về ngưỡng kiến thức (knowledge cutoff) vốn tồn tại trong tập dữ liệu huấn luyện nguyên bản.

Việc triển khai kiến trúc RAG mang lại những giá trị kỹ thuật to lớn, đặc biệt là khả năng cập nhật tri thức động mà không tiêu tốn chi phí tái huấn luyện hoặc tinh chỉnh mô hình, đồng thời giúp giảm thiểu hiện tượng hallucination và nâng cao tính kiểm chứng của dữ liệu đầu ra. Tuy nhiên, hiệu năng thực tế của giải pháp này chịu sự chi phối mạnh mẽ bởi các biến số kỹ thuật, bao gồm chất lượng của kho dữ liệu nguồn, chiến lược phân đoạn (chunking), độ chính xác của mô hình nhúng (embedding) và hiệu quả của thuật toán truy xuất. Bên cạnh đó, sự tích hợp đa tầng của các thành phần chức năng cũng làm gia tăng đáng kể độ phức tạp của hệ thống.

3.4.2 Vai trò trong hệ thống

Trong phạm vi đồ án, RAG đóng vai trò cầu nối giữa LLM cục bộ và nguồn tri thức pentest nội bộ (tài liệu kỹ thuật, lỗ hổng, payload, lịch sử kiểm thử). Cách tiếp cận này giải quyết trực tiếp các yêu cầu đã phân tích tại Chương 2 về khả năng truy xuất thông tin, giảm sự phụ thuộc vào dữ liệu bên ngoài cũng như giảm sự phụ thuộc vào tri thức từ tập dữ liệu được huấn luyện sẵn của LLM. Cụ thể, RAG được áp dụng trong các ca sử dụng “Truy xuất dữ liệu từ kho tri thức” và “Truy xuất thông tin từ kho trí nhớ” nhằm khắc phục hạn chế về kiến thức tĩnh của mô hình, đồng thời tăng cường độ chính xác nhờ khả năng kiểm chứng tài liệu. Qua đó, hệ thống có thể tái sử dụng hiệu quả nguồn tri thức sẵn có và truy xuất ngữ nghĩa chính xác các trí nhớ từ bộ nhớ dài hạn.

3.4.3 Lí do lựa chọn công nghệ

Trong quá trình xác định giải pháp kỹ thuật, các hướng tiếp cận thay thế như phụ thuộc thuần túy vào tri thức tham số của mô hình hoặc thực hiện tinh chỉnh (fine-tuning) đã được cân nhắc nhưng không được lựa chọn. Nguyên nhân chủ yếu xuất phát từ các rào cản kỹ thuật về giới hạn của số ngữ cảnh, tính “tĩnh” của dữ liệu huấn luyện (dẫn đến nguy cơ lạc hậu tri thức) cũng như các ràng buộc đáng kể về tài nguyên tính toán trong khuôn khổ phạm vi đề tài.

Vì vậy, kiến trúc RAG được lựa chọn là giải pháp nền tảng tối ưu. Trong phạm vi của chương, nội dung chỉ tập trung trình bày khái quát về cơ sở lý thuyết cũng như cơ chế hoạt động của mô hình truy vấn tri thức này. Các biến thể kỹ thuật truy xuất chuyên sâu, cùng với các chiến lược tối ưu hóa nâng cao như Reranking, sẽ được phân tích chi tiết và trình bày như một phần đóng góp trọng tâm tại Chương 5.

3.5 Docling kết hợp Semantic Chunking - Chuyển đổi và phân đoạn tài liệu

3.5.1 Khái quát kỹ thuật

Docling [12] là bộ công cụ mã nguồn mở được phát triển bởi IBM Research, đánh dấu bước cải tiến quan trọng từ các phương pháp trích xuất văn bản truyền thống sang hướng tiếp cận “hiểu tài liệu” một cách toàn diện. Khác biệt căn bản so với các kỹ thuật xử lý văn bản như một luồng tuyến tính (linear text stream) hoặc phụ thuộc hoàn toàn vào OCR (kỹ thuật nhận dạng ký tự quang học), Docling ứng dụng các mô hình thị giác máy tính hiện đại để phân tích đồng thời cả bố cục hình học lẫn cấu trúc logic của tài liệu.

Xét về kiến trúc, Docling tích hợp hai nhóm mô hình nòng cốt. Đầu tiên là Mô hình phân tích bố cục (Layout Analysis), được huấn luyện trên các tập dữ liệu quy

mô lớn như DocLayNet mô hình này đảm nhiệm việc nhận diện và phân loại chính xác các thành phần cấu trúc như tiêu đề, đoạn văn, chú thích và hình ảnh. Song song đó là Mô hình tái cấu trúc bảng (Table Structure Recognition), nhờ được tích hợp các kiến trúc chuyên biệt như TableFormer, mô hình này cho phép khôi phục cấu trúc dữ liệu của các bảng biểu phức tạp thay vì chỉ trích xuất nội dung văn bản rời rạc. Nhờ cơ chế này, Docling không chỉ trích xuất dữ liệu mà còn bảo toàn nguyên vẹn thứ bậc thông tin và mối quan hệ ngữ nghĩa giữa các thành phần.

Tiếp nối quá trình trích xuất và chuẩn hóa cấu trúc tài liệu, quá trình phân đoạn dữ liệu (chunking) được xác định là yếu tố then chốt quyết định chất lượng của không gian véc-tơ và hiệu suất truy xuất. Thay vì áp dụng các chiến lược phân đoạn cơ học dựa trên số lượng ký tự hoặc token cố định, vốn dễ dẫn đến việc cắt đứt mối liên hệ ngữ nghĩa của các phân đoạn trong tài liệu, đồ án áp dụng phương pháp phân đoạn dựa trên ngữ nghĩa (Semantic Chunking). Đây là hướng tiếp cận nâng cao dựa trên phân tích ngữ nghĩa, nhằm xác định các điểm cắt tại những ranh giới chuyển đổi ngữ nghĩa trong văn bản.

Về mặt thuật toán, Semantic Chunking vận hành theo quy trình hai bước: trước tiên thực hiện nhúng (embedding) các đơn vị văn bản cơ sở như câu hoặc đoạn ngắn, sau đó tính toán độ tương đồng (thường là Cosine Similarity) giữa các đơn vị văn bản liên kề. Khi hệ thống phát hiện sự sụt giảm đột ngột trong chỉ số tương đồng quá một ngưỡng quy định, vị trí đó được đánh dấu là điểm gãy ngữ nghĩa (semantic breakpoint) để thực hiện tách đoạn. Kết quả là mỗi đoạn (chunk) được tạo ra chứa đựng một đơn vị thông tin hoặc chủ đề hoàn chỉnh, đảm bảo tính mạch lạc cao hơn hẳn so với cách phân đoạn truyền thống. Mặc dù phương pháp này đòi hỏi chi phí tính toán cao hơn do yêu cầu phải nhúng toàn bộ tài liệu trước khi phân đoạn, đây là sự đánh đổi chiến lược có chủ đích nhằm tối ưu hóa chất lượng đầu vào cho mô hình RAG. Các đoạn chunk có tính toàn vẹn ngữ nghĩa cao giúp mô hình trích xuất chính xác các khối tri thức, từ đó nâng cao đáng kể chất lượng suy luận.

3.5.2 Vai trò trong hệ thống

Trong khuôn khổ yêu cầu chức năng của hệ thống đã được đề xuất tại Chương 2, sự kết hợp giữa Docling và kỹ thuật phân đoạn ngữ nghĩa (semantic chunking) đóng vai trò là bước tiền xử lý tri thức ngoại vi, phục vụ trực tiếp cho ca sử dụng “Nạp dữ liệu vào kho tri thức”. Cụ thể, Docling đảm nhiệm chức năng phân tích và bảo toàn cấu trúc nguyên bản của các tài liệu kỹ thuật phức tạp (như PDF, DOCX, PPTX), chuyển đổi chúng thành các biểu diễn dữ liệu có cấu trúc. Tiếp đó, kỹ thuật Semantic Chunking thực hiện phân đoạn tài liệu dựa trên các ranh giới ngữ nghĩa,

thay vì các điểm cắt cơ học, nhằm tạo ra các đơn vị tri thức có tính mạch lạc cao. Các phân đoạn này, sau khi được làm giàu bằng hệ thống metadata, sẽ được véc-tơ hóa và lưu trữ, kiến tạo nên hạ tầng dữ liệu cho các cơ chế truy xuất tri thức và bộ nhớ dài hạn. Nhờ vậy, tầng nạp tài liệu không chỉ đơn thuần phục vụ mục đích lưu trữ, mà còn đóng vai trò quyết định trực tiếp đến chất lượng truy xuất và hiệu quả suy luận tổng thể của toàn bộ hệ thống.

3.5.3 Lí do lựa chọn công nghệ

Trong bài toán tiền xử lý dữ liệu phục vụ ca sử dụng “Nạp dữ liệu vào kho tri thức”, đa dạng các chiến lược phân đoạn đã được khảo sát, bao gồm phân đoạn kích thước cố định (fixed-size), dựa trên câu (sentence-based), cửa sổ trượt (sliding window), đệ quy (recursive) và phân đoạn ngữ nghĩa (semantic chunking). Các phương pháp cơ học như fixed-size hay sentence-based tuy sở hữu ưu thế về tốc độ xử lý và tính đơn giản trong triển khai, song thường bộc lộ hạn chế lớn trong việc duy trì mạch ngữ nghĩa, gây ra hiện tượng đứt gãy thông tin đối với các tài liệu kỹ thuật phức tạp. Trong khi đó, các phương pháp dựa trên cấu trúc hình thức (như paragraph-based, recursive) dù cải thiện phần nào tính mạch lạc nhưng lại phụ thuộc quá nhiều vào định dạng văn bản gốc, dẫn đến sự thiếu ổn định khi xử lý các nguồn dữ liệu không đồng nhất.

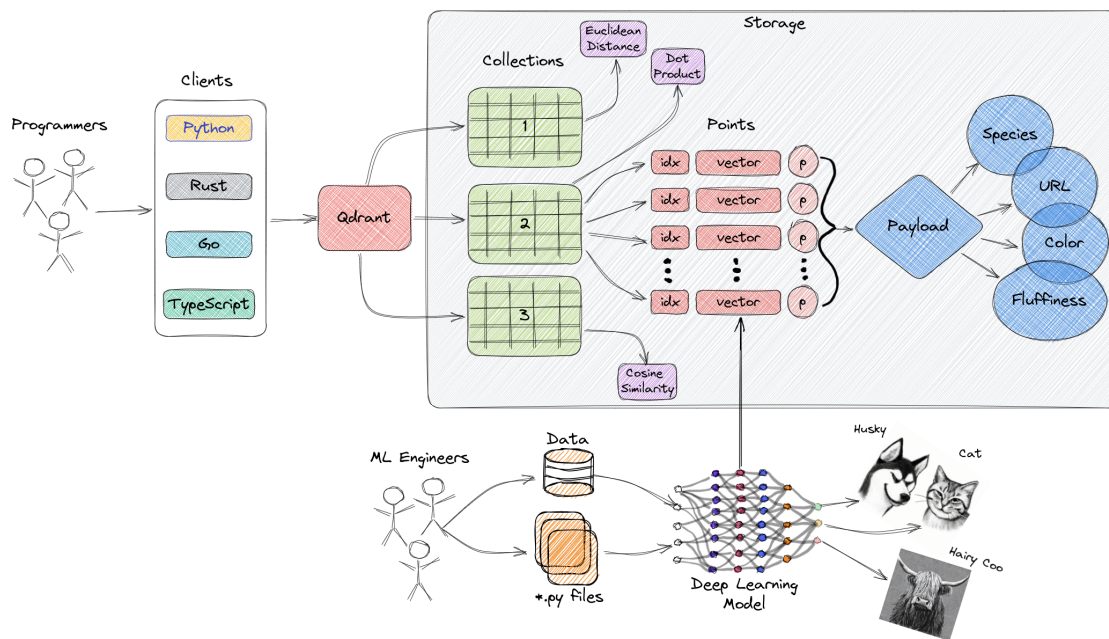
Để khắc phục các nhược điểm trên, Semantic Chunking được lựa chọn là giải pháp tối ưu cho đề án. Mặc dù phương pháp này đòi hỏi chi phí tính toán cao hơn, nhưng giá trị mang lại nằm ở khả năng nhận diện chính xác các ranh giới ngữ nghĩa. Nhờ đó, mỗi phân đoạn được tạo ra đại diện cho một đơn vị tri thức hoàn chỉnh, giúp giảm thiểu thông tin nhiễu và nâng cao đáng kể độ chính xác truy xuất trong hệ thống RAG. Đặc tính này đặc biệt phù hợp với dữ liệu chuyên ngành an ninh mạng (CVE, hướng dẫn khai thác, báo cáo kỹ thuật) - nơi các khái niệm thường được diễn giải xuyên suốt qua nhiều câu và đoạn văn liên kết chặt chẽ.

Về khía cạnh công cụ phân tích tài liệu, các giải pháp truyền thống như PDFMinerLoader [13], Apache Tika [14] hay OCR thuần túy thường xử lý tài liệu dưới dạng luồng văn bản phẳng hoặc hình ảnh, dẫn đến việc đánh mất cấu trúc logic quan trọng như tiêu đề, bảng biểu hay các mối quan hệ phân cấp. Thay vào đó, Docling được lựa chọn nhờ khả năng ứng dụng các mô hình thị giác máy tính để phân tích sâu bố cục và cấu trúc tài liệu. Việc bảo toàn nguyên vẹn ngữ cảnh và sự phân cấp nội dung từ Docling tạo tiền đề vững chắc cho quá trình semantic chunking, đảm bảo chất lượng đầu vào cao nhất cho hệ thống truy xuất tri thức.

3.6 Qdrant và FastEmbed - Hạ tầng truy xuất tri thức dựa trên véc-tơ

3.6.1 Khái quát công nghệ

Trong kiến trúc hệ thống đề xuất, Qdrant [15] được xác định là giải pháp quản trị cơ sở dữ liệu véc-tơ chuyên dụng, đóng vai trò nền tảng cho các tác vụ tìm kiếm tương đồng (similarity search) và truy xuất ngữ nghĩa. Khác biệt căn bản so với các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) truyền thống, vốn tổ chức dữ liệu theo cấu trúc bảng và truy vấn dựa trên sự trùng khớp chính xác của giá trị, Qdrant được tối ưu hóa đặc biệt để lưu trữ và truy vấn các biểu diễn véc-tơ trong không gian đa chiều.



Hình 3.3: Kiến trúc tổng quát của Qdrant [16]

Về phương thức quản trị dữ liệu, Qdrant quản lý thông tin dưới dạng các điểm (points). Mỗi điểm là một thực thể bao gồm ba thành phần cốt lõi: véc-tơ (một dạng biểu diễn đa chiều của tài liệu, hình ảnh, ...) được sinh ra từ mô hình nhúng, định danh duy nhất (ID) và payload định dạng JSON chứa các metadata bổ sung. Kiến trúc này cho phép hệ thống thực hiện cơ chế tìm kiếm lai (hybrid search): vừa tìm kiếm theo độ tương đồng ngữ nghĩa của véc-tơ, vừa kết hợp lọc dữ liệu dựa trên các thuộc tính có cấu trúc trong payload.

Để đảm bảo hiệu năng truy vấn theo thời gian thực trên tập dữ liệu quy mô lớn, Qdrant tích hợp cấu trúc chỉ mục tiên tiến HNSW (Hierarchical Navigable Small World). Thuật toán này cho phép thực hiện tìm kiếm lân cận gần đúng (Approximate Nearest Neighbor - ANN) với độ trễ cực thấp mà vẫn duy trì độ chính xác cao.

Đối với mô-đun sinh véc-tơ nhúng, hệ thống sử dụng FastEmbed [17] – một thư viện Python nhẹ nhưng hiệu năng cao. Khác biệt với các giải pháp truyền thống, FastEmbed không yêu cầu cài đặt PyTorch mà vận hành trực tiếp trên ONNX Runtime. Kiến trúc tinh gọn này không chỉ giúp giảm tải dung lượng hệ thống mà còn tối ưu hóa tốc độ thực thi, đặc biệt phù hợp cho các tác vụ xử lý thời gian thực.

Việc tích hợp FastEmbed mang lại nhiều lợi thế chiến lược cho hệ thống. Thứ nhất, nhờ việc tối giản các gói phụ thuộc (dependencies), việc cài đặt và đóng gói ứng dụng trở nên đơn giản, hạn chế tối đa rủi ro xung đột thư viện. Thứ hai, tận dụng sức mạnh của ONNX, công cụ đảm bảo tốc độ suy luận cao và ổn định trên đa dạng các hạ tầng phần cứng. Thứ ba, về mặt chất lượng, FastEmbed hỗ trợ các mô hình mã nguồn mở đạt thứ hạng cao trên bảng xếp hạng MTEB, đảm bảo độ chính xác vượt trội trong khả năng truy xuất ngữ so với các mô hình cũ như Ada-002. Cuối cùng, do nằm cùng hệ sinh thái với Qdrant, FastEmbed được tận dụng để cùng với cơ sở dữ liệu véc-tơ này triển khai hạ tầng truy xuất một cách dễ dàng và hiệu quả.

3.6.2 Vai trò trong hệ thống

Đối với vai trò hạ tầng dữ liệu, hệ thống sử dụng sự phối hợp giữa Qdrant và FastEmbed để quản lý hai nguồn tài nguyên cốt lõi: tri thức tĩnh (tài liệu, CVE...) và tri thức động (kết quả thực thi, giả định...). Trong khi Qdrant đóng vai trò là nền tảng lưu trữ và tìm kiếm véc-tơ, thì FastEmbed chịu trách nhiệm chuyển đổi dữ liệu thành véc-tơ. Cặp đôi này trực tiếp phục vụ các yêu cầu nghiệp vụ về “Truy xuất tri thức” và “Truy xuất trí nhớ”, đồng thời đóng vai trò xương sống giúp mô hình RAG hoạt động chính xác theo thiết kế đã trình bày ở Mục 3.4.

3.6.3 Lí do lựa chọn công nghệ

Trong quá trình lựa chọn giải pháp lưu trữ véc-tơ, các nền tảng phổ biến như FAISS [18], ChromaDB [19] và Milvus [20] đã được xem xét và đánh giá. FAISS, mặc dù sở hữu tốc độ truy xuất ấn tượng, về bản chất là một thư viện tìm kiếm véc-tơ trong bộ nhớ (in-memory) hơn là một hệ quản trị cơ sở dữ liệu hoàn chỉnh. Điều này đòi hỏi người phát triển phải tự xây dựng lớp quản lý dữ liệu bền vững cùng với cơ chế xử lý metadata phức tạp. ChromaDB, với ưu điểm dễ triển khai, phù hợp cho các giai đoạn thử nghiệm ban đầu nhưng lại bộc lộ hạn chế về khả năng mở rộng và thiếu các tính năng kiểm soát truy vấn nâng cao. Trong khi đó, Milvus tuy mạnh mẽ ở quy mô lớn nhưng đi kèm với chi phí vận hành và độ phức tạp hạ tầng vượt quá yêu cầu của một mô hình triển khai cục bộ. So với các giải pháp trên, Qdrant nổi bật nhờ kiến trúc dịch vụ độc lập, khả năng lưu trữ bền vững (persistence), cơ chế quản lý collection minh bạch và hệ thống API truy vấn linh

hoạt. Đặc biệt, khả năng vận hành dưới dạng Docker container giúp Qdrant dễ dàng tích hợp vào quy trình RAG và đáp ứng tốt các tiêu chuẩn vận hành dài hạn của một hệ thống pentest nội bộ.

3.7 Kiến trúc bộ nhớ dài hạn theo hướng MemGPT

3.7.1 Khái quát hướng tiếp cận

Trong các hệ thống AI agent, bộ nhớ đóng vai trò trung tâm trong việc duy trì ngữ cảnh, tích lũy tri thức và cải thiện khả năng ra quyết định theo thời gian. Trước khi tiếp cận kiến trúc MemGPT, cần phân biệt các loại bộ nhớ tác tử (agentic memory) thường được đề cập trong các nghiên cứu về nhận thức cho AI.

Bộ nhớ ngắn hạn (Short-Term Memory – STM) cho phép agent ghi nhớ các tương tác gần nhất nhằm duy trì tính liên tục trong quá trình suy luận hoặc hội thoại. STM thường được hiện thực dưới dạng context window, nơi các thông điệp gần nhất được sử dụng trực tiếp làm đầu vào cho LLM. Cách tiếp cận này phù hợp với các tương tác ngắn hạn, nhưng bị giới hạn bởi độ dài ngữ cảnh của mô hình và không có khả năng lưu giữ thông tin vượt qua phạm vi một phiên làm việc.

Bộ nhớ dài hạn (Long-Term Memory – LTM) mở rộng khả năng của agent bằng cách lưu trữ thông tin vượt qua ranh giới các phiên làm việc, cho phép tái sử dụng tri thức trong các tương tác sau này. LTM thường được kiến trúc hóa thông qua sự kết hợp giữa các cơ sở dữ liệu, đồ thị tri thức và hệ thống véc-tơ kết hợp truy xuất RAG. Đây là thành phần cốt lõi đối với các hệ thống yêu cầu tính cá nhân hóa cao và khả năng học hỏi liên tục từ lịch sử vận hành.

Bộ nhớ tình huống (Episodic Memory) cho phép agent ghi nhớ các sự kiện hoặc trải nghiệm cụ thể, bao gồm chuỗi hành động và kết quả tương ứng. Dạng bộ nhớ này đặc biệt hữu ích cho suy luận dựa trên trường hợp (case-based reasoning), trong đó agent có thể tham chiếu các tình huống tương tự đã từng xảy ra để đưa ra quyết định tốt hơn. Trong bối cảnh môi trường pentest, episodic memory có thể lưu trữ các phiên pentest trước, các chiến lược đã áp dụng và mức độ thành công của chúng.

Bộ nhớ ngữ nghĩa (Semantic Memory) lưu trữ các tri thức mang tính khái quát như khái niệm, định nghĩa, quy tắc và mối quan hệ giữa các thực thể. Đây là dạng bộ nhớ nền tảng cho khả năng suy luận logic và truy xuất tri thức miền (domain knowledge). Trong hệ thống AI hỗ trợ pentest, semantic memory bao gồm tri thức về lỗ hổng, kỹ thuật khai thác hay các mẫu tấn công phổ biến.

Bộ nhớ thủ tục (Procedural Memory) liên quan đến việc lưu trữ các kỹ năng và quy trình hành động đã được học, cho phép agent thực hiện một chuỗi thao tác

phức tạp mà không cần suy luận lại từ đầu. Dạng bộ nhớ này giúp tối ưu hiệu năng và giảm chi phí tính toán, đặc biệt khi agent phải lặp lại các quy trình quen thuộc như chuỗi các bước do thám, khai thác hoặc hậu khai thác trong quá trình kiểm thử xâm nhập.

Trên cơ sở các phân loại trên, MemGPT được đề xuất như một kiến trúc bộ nhớ phân tầng cho AI agent, nhằm khắc phục hạn chế của việc chỉ dựa vào context window của LLM. Thay vì coi toàn bộ lịch sử tương tác là một khối đồng nhất, MemGPT tách bộ nhớ thành các lớp với vai trò khác nhau, điển hình là core memory (bộ nhớ ổn định, mang tính dài hạn) và recall memory (bộ nhớ truy xuất theo ngữ cảnh), kết hợp với cơ chế truy xuất và tóm tắt tự động để quản lý ngân sách ngữ cảnh. Cách tiếp cận này cho phép agent duy trì ngữ cảnh dài hạn, tích lũy tri thức theo thời gian mà vẫn tuân thủ giới hạn ngữ cảnh của LLM.

Trong phạm vi chương này, MemGPT được xem như một nền tảng lý thuyết cho thiết kế bộ nhớ dài hạn của hệ thống. Chi tiết về cách hiện thực kiến trúc này, bao gồm chiến lược phân tách bộ nhớ, cơ chế tóm tắt, truy xuất và tích hợp với RAG, sẽ được trình bày cụ thể trong Chương 5 như một đóng góp kỹ thuật của đề tài.

3.7.2 Vai trò trong hệ thống

Trong Chương 2, ca sử dụng “Nạp thông tin vào kho trí nhớ” và ca sử dụng “Truy xuất thông tin từ kho trí nhớ” không chỉ yêu cầu hệ thống phải bảo toàn được nguồn thông tin ổn định (mục tiêu, phạm vi, credential hợp lệ), mà còn phải lưu được diễn biến (kết quả công cụ, giả thuyết, nhánh thử nghiệm) để tái sử dụng. Kiến trúc theo hướng MemGPT cho phép: (i) ưu tiên và cố định các thông tin trọng yếu trong core memory (ít thay đổi, giá trị lâu dài); (ii) lưu các sự kiện vào recall memory để truy xuất khi cần; (iii) hạn chế tràn ngữ cảnh và giảm chi phí suy luận trong các phiên pentest kéo dài.

3.7.3 Lí do lựa chọn công nghệ

Trong thực tế triển khai, các phương pháp quản lý ngữ cảnh thông thường như lưu trữ toàn bộ lịch sử hội thoại theo phiên (session-based) hoặc sử dụng kỹ thuật cửa sổ trượt (sliding window) thường được cân nhắc như những giải pháp thay thế. Tuy nhiên, các kỹ thuật này bộc lộ hạn chế rõ rệt trong việc giải quyết bài toán tích lũy tri thức theo thời gian: thông tin trọng yếu dễ bị loại bỏ khỏi cửa sổ ngữ cảnh do giới hạn dung lượng bộ nhớ đệm, đồng thời việc truy xuất lại các phát hiện cũ hay cập nhật ký ức của LLM thường gặp nhiều trở ngại.

Khắc phục các nhược điểm này, kiến trúc bộ nhớ theo hướng MemGPT được lựa chọn làm nền tảng lý thuyết chủ đạo cho hệ thống. Mô hình này cho phép phân tách và quản lý linh hoạt giữa tri thức ngắn hạn và dài hạn, đảm bảo tính liên tục

của ngữ cảnh mà không làm mất đi các dữ liệu quan trọng. Các chi tiết vận hành cụ thể bao gồm thiết kế cấu trúc dữ liệu, cơ chế tóm tắt tự động và chiến lược truy xuất sẽ được trình bày chi tiết hơn tại Chương 5.

3.8 Model Context Protocol - Giao thức ngữ cảnh mô hình

3.8.1 Khái quát công nghệ

Model Context Protocol (MCP) [21] được định nghĩa là một chuẩn giao thức mở, được thiết kế nhằm mục đích chuẩn hóa cơ chế tương tác giữa các AI agents và hệ sinh thái dữ liệu cũng như công cụ bên ngoài. Vận hành dựa trên mô hình kiến trúc client-server, MCP giải quyết các hạn chế của phương pháp tích hợp truyền thống, nơi các LLM thường phải gọi trực tiếp các hàm nội bộ hoặc bị ràng buộc chặt chẽ vào các nền tảng cụ thể. Thay vào đó, giao thức này thiết lập một lớp trung gian trừu tượng hóa, trong đó AI agent đóng vai trò điều phối viên trung tâm, còn các hệ thống ngoại vi được đóng gói thành các MCP Server độc lập, cung cấp ngữ cảnh và chức năng hành động thông qua các giao diện được chuẩn hóa.

Về mặt kiến trúc tổng thể, MCP được cấu thành từ ba thành phần chức năng cốt lõi [22]. Trong đó, MCP Host đóng vai trò là ứng dụng AI trung tâm (ví dụ như Claude Code hoặc Visual Studio Code), chịu trách nhiệm điều phối luồng suy luận và ra quyết định. MCP Client hoạt động như một lớp trung gian, duy trì kết nối với các MCP Server để chuyển tiếp ngữ cảnh hoặc kết quả thực thi về cho Host. Thành phần cuối cùng, MCP Server, là các chương trình độc lập cung cấp ngữ cảnh dữ liệu và khả năng hành động, có thể vận hành cục bộ hoặc từ xa và không phụ thuộc vào mô hình ngôn ngữ. Cách tổ chức phân tán này cho phép một MCP Host có thể thiết lập kết nối đồng thời tới nhiều MCP Server khác nhau, mỗi server được thiết kế chuyên biệt cho một miền chức năng riêng biệt.

Xét về khía cạnh cấu trúc, MCP được phân chia thành hai tầng tách biệt: tầng dữ liệu (data layer) và tầng truyền tải (transport layer). Tầng dữ liệu định nghĩa giao thức trao đổi dựa trên chuẩn JSON-RPC 2.0, bao gồm các cơ chế quản lý vòng đời (lifecycle management) và các nguyên mẫu cốt lõi (core primitives). Các nguyên mẫu này xác định những gì client và server có thể cung cấp cho nhau, bao gồm các loại thông tin ngữ cảnh có thể chia sẻ cho ứng dụng AI và phạm vi các hành động có thể thực hiện. Cụ thể, phía server cung cấp ba nhóm nguyên mẫu chính: tools (các hàm có thể thực thi), resources (nguồn dữ liệu hoặc ngữ cảnh tĩnh), và prompts (các chỉ thị cho LLM). Ở chiều ngược lại, phía client cũng có khả năng cung cấp các nguyên mẫu như sampling (yêu cầu host gọi LLM), elicitation (yêu cầu tương tác với người dùng) và logging (ghi vết). Thiết kế tương tác hai chiều này khẳng định MCP không chỉ đơn thuần là một giao thức gọi hàm, mà là một cơ

chế trao đổi ngữ cảnh linh hoạt giữa AI agent và hệ sinh thái xung quanh nó.

Tầng truyền tải của MCP chịu trách nhiệm hiện thực hóa việc trao đổi dữ liệu một cách an toàn và hiệu quả thông qua hai cơ chế chính. Cơ chế Stdio transport tận dụng luồng nhập/xuất (standard input/output) để thiết lập giao tiếp giữa các tiến trình cục bộ, tối ưu cho các kịch bản nơi MCP Server vận hành trên cùng máy chủ vật lý với Host, đảm bảo yêu cầu độ trễ thấp và bảo mật cao. Ngược lại, cơ chế Streamable HTTP transport được thiết kế chuyên biệt cho các MCP Server vận hành từ xa hoặc phục vụ đa client đồng thời. Phương thức này sử dụng HTTP POST cho các yêu cầu và phản hồi, kết hợp với tùy chọn Server-Sent Events (SSE) để hỗ trợ khả năng streaming dữ liệu và gửi thông báo theo thời gian thực khi cần thiết.

Cụ thể đối với giao thức Streamable HTTP, hệ thống được thiết kế theo kiến trúc client-server dựa trên chuẩn HTTP, trong đó toàn bộ hoạt động giao tiếp diễn ra thông qua một endpoint duy nhất. Kiến trúc này cho phép server hoạt động như một tiến trình độc lập, có khả năng phục vụ nhiều client cùng lúc mà không cần duy trì các kết nối trạng thái dài hạn ở tầng mạng. Tuy nhiên, về mặt logic ứng dụng, hệ thống vẫn đảm bảo tính hướng trạng thái (stateful) thông qua cơ chế quản lý phiên chặt chẽ. Phiên làm việc được thiết lập ngay trong pha khởi tạo bằng việc trao đổi cặp tin `InitializeRequest` và `InitializeResponse`, sử dụng định danh `Mcp-Session-Id` để liên kết các chuỗi tương tác tiếp theo.

Về luồng hoạt động, client gửi các yêu cầu JSON-RPC đến server thông qua các HTTP POST; server có thể phản hồi theo hai cách: trả về một HTTP response đơn thuần cho các tác vụ ngắn, hoặc mở một luồng Server-Sent Events (SSE) để streaming kết quả và thông báo tiến trình cho các tác vụ dài. Ngược lại, client cũng có thể mở một HTTP GET để lắng nghe SSE, cho phép server chủ động gửi yêu cầu hoặc thông báo bất đồng bộ. Cơ chế kết hợp POST cho hành động đơn và GET tích hợp SSE cho sự kiện giúp Streamable HTTP hỗ trợ giao tiếp hai chiều, bất đồng bộ và theo luồng, đồng thời vẫn giữ được tính đơn giản, khả thi và an toàn của kiến trúc HTTP truyền thống.

Từ góc nhìn kiến trúc hệ thống, MCP kết hợp với Streamable HTTP giải quyết trực tiếp bài toán tách biệt giữa suy luận và thực thi. AI agent không cần biết chi tiết cách một công cụ được triển khai, chạy ở đâu hay với quyền hạn nào; mọi tương tác đều thông qua giao diện chuẩn hóa do MCP Server cung cấp. Điều này đặc biệt phù hợp với các hệ thống pentest, nơi các công cụ pentest cần được chạy trong môi trường cách ly và có khả năng giám sát. MCP cho phép mở rộng, thay thế hoặc cô lập các công cụ mà không ảnh hưởng đến logic suy luận của agent.

3.8.2 Vai trò trong hệ thống

Trong Chương 2, ca sử dụng “Điều phối công cụ pentest” và ca sử dụng “Thực thi công cụ pentest” yêu cầu hai đặc tính: (i) mở rộng danh sách công cụ mà không làm tăng độ kết dính với agent; (ii) đảm bảo an toàn khi thực thi công cụ có rủi ro cao. MCP đáp ứng hai yêu cầu này bằng cách: (i) chuẩn hóa giao diện gọi công cụ (schema, tham số, kết quả), giúp agent gọi công cụ theo một cách nhất quán; (ii) tách biệt môi trường thực thi: công cụ được chạy trong Docker container thay vì chạy trực tiếp trong tiến trình chạy agent; (iii) cho phép streaming kết quả, tăng khả năng giám sát tiến trình và hỗ trợ agent suy luận theo thời gian thực.

3.8.3 Lí do lựa chọn công nghệ

Trong quá trình xác định giải pháp tích hợp công cụ, các hướng tiếp cận truyền thống như thực thi trực tiếp qua tiến trình con (subprocess) hoặc phát triển các mô-đun mở rộng (plugin) gắn chặt với framework agent đã được cân nhắc. Tuy nhiên, các phương pháp này thường bộc lộ những hạn chế đáng kể, bao gồm việc gia tăng bề mặt tấn công (attack surface) dẫn đến rủi ro bảo mật cao, cũng như làm giảm tính linh hoạt trong việc mở rộng hoặc triển khai theo kiến trúc mô-đun. Khắc phục những nhược điểm này, Model Context Protocol (MCP) được lựa chọn nhờ khả năng cung cấp một cơ chế giao tiếp chuẩn hóa, dễ dàng mở rộng và đặc biệt tương thích với chiến lược cô lập các công cụ thực thi trong môi trường sandbox. Các khía cạnh kỹ thuật cụ thể về triển khai sandbox, cơ chế phân quyền và tổ chức máy chủ công cụ (tool server) sẽ được trình bày chi tiết tại Chương 4 và Chương 5.

Tổng kết lại, sự kết hợp giữa MCP và giao thức Streamable HTTP kiến tạo nên một nền tảng giao tiếp có cấu trúc chặt chẽ, khả năng mở rộng cao và tương thích tốt với hạ tầng hệ thống AI agent. Trong phạm vi đề án, công nghệ này không chỉ là yếu tố then chốt để hiện thực hóa các ca sử dụng điều phối và thực thi công cụ pentest, mà còn đặt nền móng kỹ thuật để xây dựng cơ chế an toàn, bảo mật cho hệ thống.

3.9 Containerization & Docker - Đóng gói hệ thống & sandbox thực thi công cụ

3.9.1 Khái quát công nghệ

Containerization là giải pháp ảo hóa ở cấp độ hệ điều hành nhằm giải quyết triệt để vấn đề bất đồng bộ giữa môi trường phát triển (development) và môi trường triển khai (production). Khác với máy ảo truyền thống phải ảo hóa toàn bộ phần cứng và chạy hệ điều hành khách riêng biệt, container đóng gói ứng dụng cùng toàn bộ thư viện và cấu hình phụ thuộc vào một đơn vị độc lập, vận hành dựa trên cơ chế chia sẻ nhân (kernel) của hệ điều hành chủ. Cách tiếp cận này không chỉ đảm bảo

ứng dụng hoạt động nhất quán trên mọi hạ tầng mà còn giúp tối ưu hóa tài nguyên phần cứng và tăng tốc độ khởi động đáng kể.

Docker là nền tảng mã nguồn mở phổ biến nhất hiện nay để hiện thực hóa công nghệ này, cung cấp giao diện giúp đơn giản hóa việc quản lý vòng đời ứng dụng. Kiến trúc của Docker vận hành theo mô hình Client-Server, trong đó Docker Daemon là trung tâm điều phối chạy trên máy chủ, chịu trách nhiệm quản lý các đối tượng như Docker Image (bản thiết kế bất biến của ứng dụng) và Docker Containers (phiên bản thực thi của Docker Image). Người dùng tương tác với hệ thống thông qua Docker CLI hoặc API, trong khi Docker Hub đóng vai trò là kho lưu trữ tập trung (registry) để chia sẻ và phân phối các Docker Image.

Về cơ chế hoạt động bên trong, Docker tận dụng các tính năng tiên tiến của Linux kernel để kiến tạo môi trường thực thi. Cụ thể, nó sử dụng Namespaces để cô lập các không gian tài nguyên (như PID, mạng, hệ thống tệp) và Control Groups (cgroups) để giới hạn mức tiêu thụ phần cứng (CPU, RAM) của từng container. Trên môi trường Windows, Docker sử dụng các lớp ảo hóa như Hyper-V hoặc tích hợp với WSL 2 (Windows Subsystem for Linux 2) để cung cấp nhân Linux cần thiết, đảm bảo các container Linux có thể vận hành với hiệu năng cao, tương thích tốt ngay trên nền tảng Windows.

3.9.2 Vai trò trong hệ thống

Trong yêu cầu tổng thể của hệ thống LPAA được đề xuất tại Chương 2, Docker đóng vai trò là hạ tầng nền tảng chiến lược, đảm nhiệm việc hiện thực hóa các ca sử dụng cốt lõi liên quan đến điều phối và thực thi công cụ pentest. Cụ thể, đối với ca sử dụng “Thực thi công cụ pentest”, công nghệ này được sử dụng để thiết lập môi trường sandbox cách ly nghiêm ngặt các công cụ pentest với tiến trình chạy agent. Cơ chế này giúp giới hạn phạm vi ảnh hưởng của các câu lệnh pentest, giảm thiểu rủi ro an ninh đối với hạ tầng mạng và đảm bảo tính an toàn khi thực thi các công cụ này.

Song song với chức năng cách ly, Docker hỗ trợ đắc lực cho ca sử dụng “Điều phối công cụ pentest” thông qua việc cung cấp một môi trường thực thi nhất quán và có khả năng tái lập cao. Bằng cách đóng gói công cụ cùng toàn bộ các thư viện phụ thuộc vào container, hệ thống trừu tượng hóa các chi tiết cấu hình phức tạp, cho phép AI agent tập trung hoàn toàn vào logic suy luận và điều phối chiến lược. Agent có thể gọi các công cụ thông qua giao thức MCP mà không cần quan tâm đến sự khác biệt của môi trường vận hành bên dưới.

Hơn nữa, việc ứng dụng Docker đáp ứng trực tiếp các yêu cầu phi chức năng quan trọng đã được phân tích tại 2.4, bao gồm tính an toàn và khả năng kiểm soát

tài nguyên. Thông qua cơ chế cô lập tài nguyên, khả năng thu hồi container tự động theo phiên làm việc và quy trình lưu trữ log thực thi có cấu trúc, Docker tạo ra một lớp bảo vệ vững chắc, đảm bảo hệ thống vận hành ổn định và minh bạch trong suốt quá trình kiểm thử bảo mật.

3.9.3 Lí do lựa chọn công nghệ

Trong quá trình xác định giải pháp cho một môi trường thực thi an toàn, các phương pháp tiếp cận thay thế đã được cân nhắc. Một lựa chọn phổ biến là thực thi trực tiếp trên hệ điều hành chủ, nơi các công cụ được gọi thông qua các tiến trình hệ thống tiêu chuẩn. Mặc dù phương pháp này sở hữu ưu thế về hiệu suất và sự đơn giản trong triển khai kỹ thuật, song lại tồn tại các rủi ro bảo mật nghiêm trọng. Việc thiếu cơ chế cô lập khiến các lệnh pentest có khả năng tác động tiêu cực trực tiếp đến hệ thống máy chủ, gây khó khăn trong việc khoanh vùng phạm vi ảnh hưởng và không đáp ứng được các tiêu chuẩn an toàn của môi trường sandbox.

Một hướng tiếp cận khác là sử dụng công nghệ ảo hóa toàn phần (Virtual Machine - VM) như KVM hoặc VirtualBox để thiết lập môi trường cách ly. Tuy nhiên, rào cản chính của giải pháp này nằm ở chi phí tài nguyên lớn (resource overhead) và độ trễ khởi tạo cao, làm giảm đáng kể tính linh hoạt của các kịch bản pentest. Hơn nữa, sự phức tạp trong quản lý vòng đời của VM tạo ra những thách thức không nhỏ đối với khả năng tự động hóa quy trình điều phối từ phía AI agent.

Dựa trên kết quả so sánh đối chiếu, Docker được xác định là giải pháp tối ưu nhờ khả năng cân bằng giữa mức độ cách ly an toàn, hiệu năng xử lý và tính thuận tiện cho quy trình tự động hóa. Thông qua công nghệ Containerization, giải pháp này cho phép đóng gói trọn vẹn công cụ cùng các thư viện phụ thuộc, đảm bảo khả năng khởi tạo tức thời theo từng phiên làm việc và tích hợp liền mạch với máy chủ MCP trong mạng nội bộ. Đặc biệt, kiến trúc của Docker hoàn toàn tương thích với định hướng thiết kế offline-first, qua đó đáp ứng trọn vẹn các yêu cầu phi chức năng khắt khe đã được xác lập tại Chương 2.4.

Kết chương

Chương này đã trình bày các công nghệ và hướng tiếp cận kỹ thuật chủ đạo được sử dụng trong đồ án, đồng thời làm rõ vai trò của từng công nghệ trong việc đáp ứng các yêu cầu đã được phân tích ở Chương 2. Trên cơ sở các lựa chọn công nghệ này, Chương 4 sẽ tập trung vào hiện thực hóa chúng trong một kiến trúc hệ thống cụ thể, bao gồm thiết kế kiến trúc, thiết kế chi tiết, quá trình xây dựng ứng dụng, kiểm thử và triển khai thử nghiệm.

CHƯƠNG 4. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này tập trung trình bày quá trình thiết kế và hiện thực hệ thống. Nội dung chương bao gồm việc áp dụng kiến trúc multi-agent theo mô hình sub-agents, thiết kế chi tiết các agent thành phần và cơ chế phối hợp giữa chúng. Bên cạnh đó, chương mô tả thiết kế cơ sở dữ liệu kết hợp PostgreSQL và Qdrant, minh họa các chức năng chính và kết quả đạt được, xây dựng các test case kiểm thử khả năng hoạt động của hệ thống, và trình bày quá trình triển khai hệ thống trong thực tế.

4.1 Lựa chọn kiến trúc hệ thống

4.1.1 Khái niệm kiến trúc multi-agent theo kiểu subagents

Kiến trúc multi-agent [23] là cách tổ chức hệ thống AI theo hướng phối hợp nhiều thành phần chuyên biệt để xử lý các luồng công việc phức tạp một cách hiệu quả; dù vậy, không phải mọi nhiệm vụ phức tạp đều bắt buộc cần multi-agent, vì trong nhiều trường hợp một agent đơn với prompt tốt và bộ công cụ phù hợp vẫn có thể cho kết quả tương đương. Tuy nhiên, multi-agent thường được lựa chọn khi hệ thống cần: (i) quản trị ngữ cảnh tốt hơn (context management), tức là cung cấp tri thức và năng lực chuyên sâu theo nhu cầu mà không làm quá tải context window của mô hình, (ii) phát triển phân tán (distributed development) theo mô-đun, cho phép các nhóm/khối chức năng xây dựng và bảo trì độc lập rồi ghép lại một cách rõ ràng, và/hoặc (iii) song song hóa quá trình xử lý (parallelization), tức tách các tác vụ con cho các “worker” chuyên biệt chạy đồng thời để giảm độ trễ. Kiến trúc multi-agent vì vậy đặc biệt hữu ích trong các bài toán phức tạp đòi hỏi tri thức chuyên sâu, quản lý ngữ cảnh dài và vận hành đa dạng công cụ. Quan trọng hơn, mô hình này cung cấp khả năng kiểm soát luồng xử lý linh hoạt: cho phép áp đặt các ràng buộc để đảm bảo một số tác vụ chỉ được thực thi sau khi các điều kiện tiên quyết đã được thỏa mãn.

Kiểu kiến trúc subagents [24] là một biến thể của multi-agent trong đó tồn tại một agent trung tâm (main agent) đóng vai trò là tác tử điều phối (supervisor), và các subagent được đóng gói như các công cụ để main agent gọi khi cần. Main agent chịu trách nhiệm quyết định subagent nào phù hợp với nhiệm vụ nào, từ đó truyền đầu vào cần thiết và tổng hợp các kết quả trả về để tạo ra câu trả lời hoặc quyết định bước xử lý tiếp theo. Điểm quan trọng của mẫu kiến trúc này là subagent thường không ghi nhớ tương tác trước đó (stateless); toàn bộ bộ nhớ hội thoại và ngữ cảnh của phiên làm việc được duy trì ở main agent, qua đó tạo ra sự cách ly ngữ cảnh (context isolation), mỗi lần gọi subagent diễn ra trong một context window riêng biệt, giúp tránh phình to ngữ cảnh của hệ thống.

Về đặc tính vận hành, kiến trúc multi-agent theo kiểu subagents nhấn mạnh khả năng điều phối tập trung (mọi quyết định định tuyến đi qua main agent), subagent không tương tác trực tiếp với người dùng mà trả kết quả về cho main agent, và main agent có thể gọi nhiều subagent trong cùng một lượt để tận dụng khả năng song song hóa khi phù hợp. Đồng thời, supervisor trong mẫu subagents khác với “router” thuần túy ở chỗ supervisor là một agent đầy đủ có thể duy trì ngữ cảnh hội thoại và ra quyết định qua nhiều lượt, trong khi router thường chỉ là một bước phân loại/điều hướng phi trạng thái.

4.1.2 Lí do lựa chọn kiến trúc multi-agent theo kiểu subagents

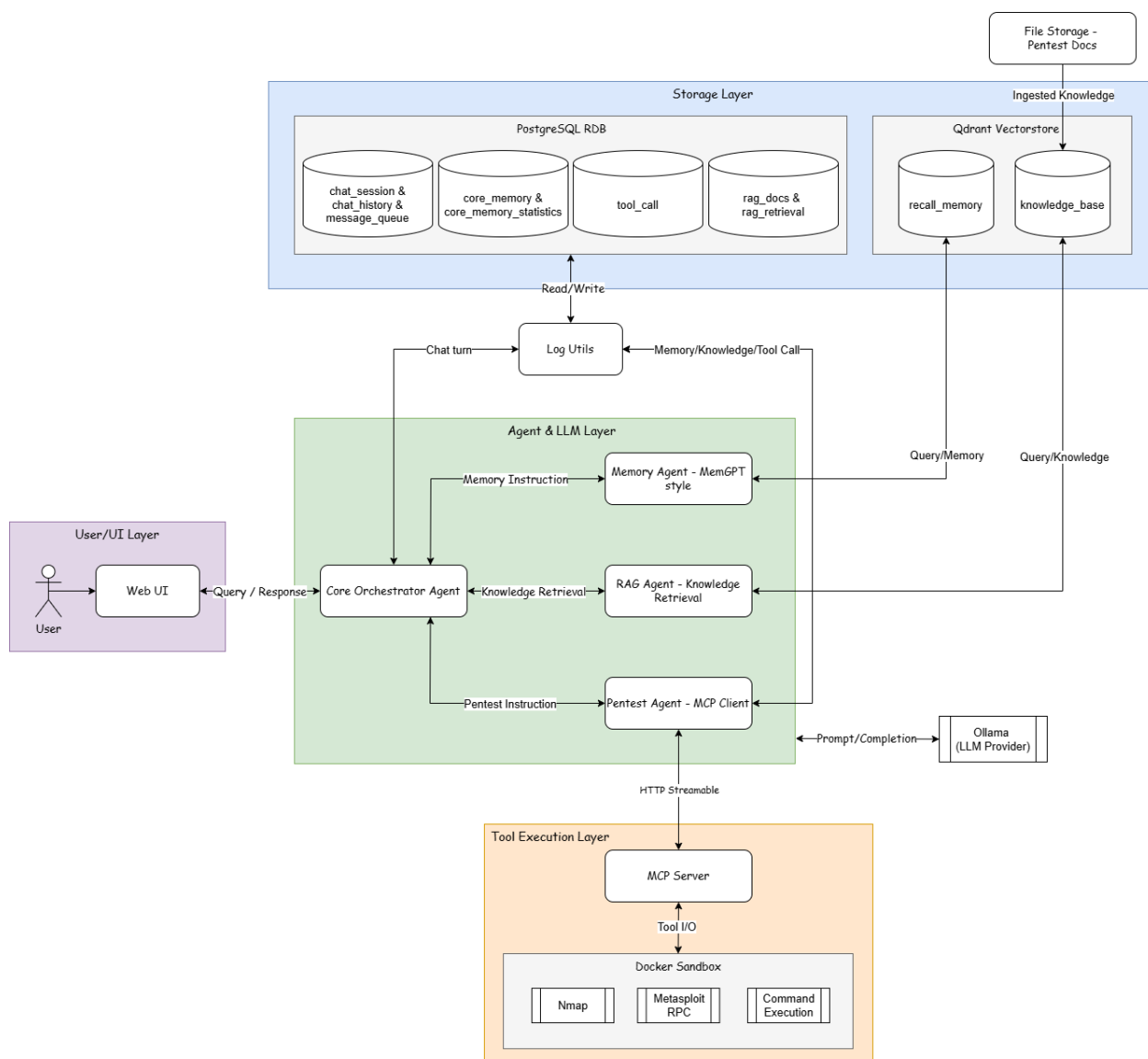
Đề tài lựa chọn kiến trúc multi-agent theo kiểu subagents vì nó giải quyết trực tiếp ba nhu cầu cốt lõi của một hệ thống trợ lý pentest cục bộ: (i) quản lý ngữ cảnh hiệu quả, (ii) tách rời phát triển theo mô-đun, và (iii) khai thác hiệu quả tài nguyên cho các tác vụ độc lập. Theo định hướng context management nêu ở mục 4.1.1, kiến trúc multi-agent cho phép hệ thống truyền đúng thông tin cần thiết vào đúng tác tử, thay vì dồn toàn bộ tri thức và công cụ vào một prompt lớn (dễ vượt giới hạn của sổ ngữ cảnh và làm suy giảm chất lượng ra quyết định). Đồng thời, việc chia thành các thành phần chuyên biệt tạo ra ranh giới rõ ràng cho quá trình phát triển và bảo trì sau này.

Trong hệ thống LPAA, main agent (Core Orchestrator) giữ vai trò điều phối viên trung tâm: tiếp nhận yêu cầu người dùng, phân rã nhiệm vụ và gọi các subagent dưới dạng công cụ theo đúng năng lực mà nó quảng bá. Cụ thể, RAG Agent được gọi khi cần truy xuất tri thức và tài liệu; Memory Agent được gọi khi cần đọc/ghi/tóm tắt bộ nhớ dài hạn hoặc truy xuất trí nhớ theo phiên; và Pentest Agent được gọi khi cần thực thi các công cụ pentest thông qua lớp MCP. Cách tổ chức này bám sát mẫu thiết kế subagents: main agent gọi subagent như một công cụ, trong đó main agent giữ trạng thái hội thoại và chỉ nhận về kết quả đã được cô đọng để tiếp tục điều phối hoặc trả lời người dùng.

Nhờ đặc tính phi trạng thái của các subagent, mỗi lần gọi agent chuyên biệt đều diễn ra trong một ngữ cảnh riêng, giảm nguy cơ tràn ngữ cảnh (hay context bloat như được định nghĩa trong [24]) ở luồng điều phối và hạn chế việc LLM bị quá tải bởi quá nhiều công cụ hay tri thức không liên quan tại thời điểm suy luận. Điều này đặc biệt phù hợp với bài toán pentest, nơi cùng một phiên làm việc có thể xen kẽ nhiều loại thao tác và việc trộn tất cả vào một ngữ cảnh dễ làm giảm độ chính xác cũng như tăng rủi ro thao tác.

4.2 Kiến trúc tổng quan của hệ thống

4.2.1 Các thành phần chính



Hình 4.1: Kiến trúc hệ thống LPAA

Kiến trúc hệ thống được tách ra thành bốn lớp (layer) riêng biệt, trong đó lớp logic trung tâm là lớp Agent & LLM được thiết kế theo kiến trúc multi-agent kiểu subagents. Như đã phân tích ở 4.1.2, kiến trúc này phân tách các năng lực chuyên biệt và đóng gói chúng thành các subagent để Core Orchestrator gọi theo nhu cầu. Ở lớp giao diện người dùng (User/UI Layer), Web UI là điểm vào chính của hệ thống, cho phép pentester khởi tạo phiên làm việc, gửi yêu cầu và nhận phản hồi; lớp này không chứa logic nghiệp vụ phức tạp mà tập trung vào quản lý vòng đời của phiên hội thoại và hiển thị kết quả một cách trực quan, đồng thời đóng vai trò cầu nối giữa người dùng và bộ xử lý trung tâm.

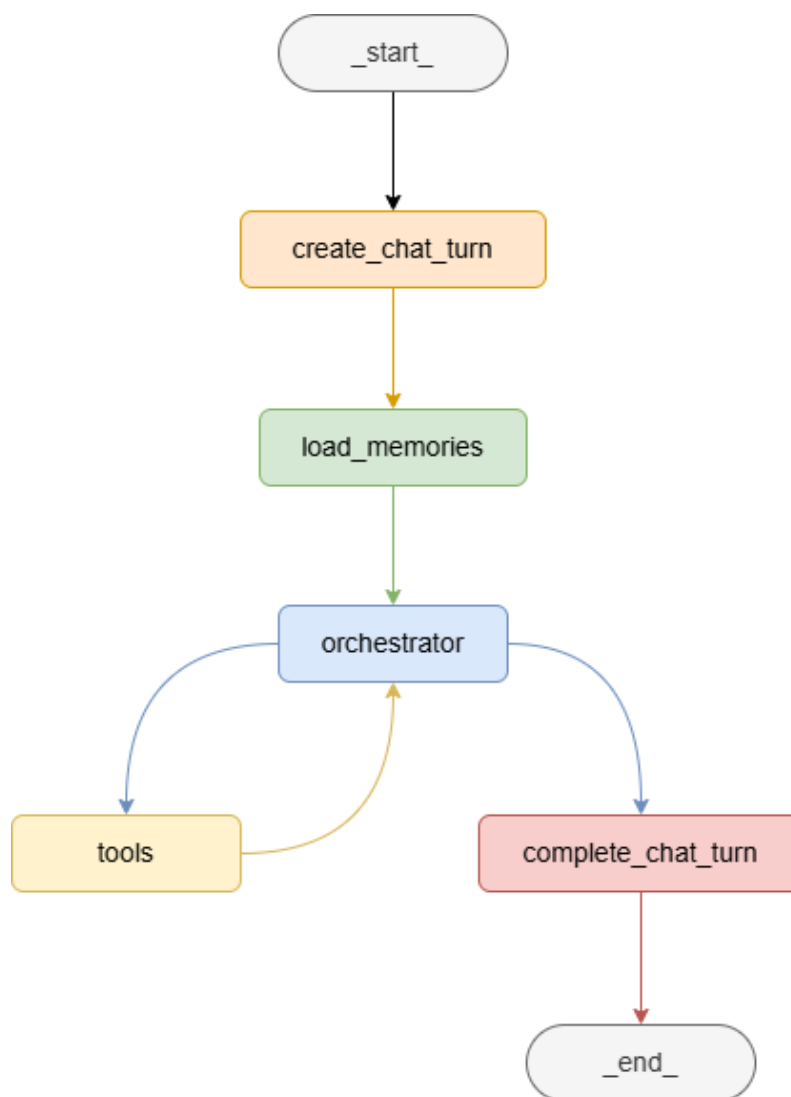
Lớp Agent và LLM (Agent & LLM Layer) là trung tâm xử lý nghiệp vụ, được

mô hình hóa bằng Conversation Graph dựa trên LangGraph để điều khiển luồng hội thoại dưới dạng các nút và các cạnh, nhờ đó kiểm soát chặt chẽ trình tự thực thi, điều kiện chuyển trạng thái và điểm hội tụ kết quả. Trong lớp này, Core Orchestrator đóng vai trò main agent/supervisor của kiến trúc subagents: agent này tiếp nhận yêu cầu từ lớp giao diện người dùng, kết hợp ngữ cảnh đã nạp, ra quyết định định tuyến và gọi các subagent dưới dạng công cụ. Theo đúng tinh thần subagents, Core Orchestrator không trực tiếp thực hiện tác vụ chuyên sâu mà chịu trách nhiệm phân rã nhiệm vụ, lựa chọn đúng tác tử, truyền đầu vào tối thiểu cần thiết và tổng hợp các đầu ra để tạo phản hồi cuối cùng cho người dùng, qua đó duy trì được sự cách ly ngữ cảnh và hạn chế phình to của sổ ngữ cảnh ở luồng điều phối.

Các thành phần Memory Agent, RAG Agent và Pentest Agent trong thiết kế này chính là các subagent như trong kiến trúc đã đề cập. Memory Agent đảm nhiệm quản lý bộ nhớ theo hướng MemGPT, bao gồm truy xuất và cập nhật core memory và recall memory, và được Core Orchestrator gọi khi cần nạp bối cảnh, lưu kết luận quan trọng hoặc tóm tắt hội thoại để kiểm soát ngân sách token. RAG Agent chịu trách nhiệm truy xuất dữ liệu từ kho tri thức bằng cơ chế Hybrid Search kết hợp Reranking, sau đó tổng hợp và chắt lọc thông tin phù hợp với câu hỏi để cung cấp tri thức phục vụ cho quá trình suy luận của Core Orchestrator. Pentest Agent thực thi các nhiệm vụ kỹ thuật và được triển khai như một MCP Client, chỉ được gọi khi cần thao tác với bộ công cụ pentest, đồng thời trả về kết quả đã được cô đọng để Core Orchestrator ra quyết định bước tiếp theo hoặc kết luận.

Về mặt hạ tầng, hệ thống được xây dựng trên ba trụ cột công nghệ vững chắc. Đầu tiên, lớp lưu trữ (Storage Layer) đóng vai trò là nơi lưu trữ dữ liệu, kết hợp PostgreSQL cho các bản ghi cấu trúc và Qdrant cho các truy vấn véc-tơ có ngữ nghĩa. Kế đến là lớp thực thi công cụ (Tool Execution Layer), nơi tích hợp MCP Server và Docker Sandbox để thiết lập một cơ chế an toàn: mọi tác vụ kỹ thuật từ Pentest Agent đều bị cô lập trong môi trường Sandbox trước khi kết quả được trả về cho Core Orchestrator. Cuối cùng, để đảm bảo an toàn thông tin và khả năng chạy mượt mà các mô hình ngôn ngữ lớn, hệ thống sử dụng Ollama làm nền tảng runtime cho LLM, giữ cho mọi dữ liệu nhạy cảm không rời khỏi môi trường nội bộ.

4.2.2 Luồng xử lý tổng quát



Hình 4.2: Luồng xử lý của Conversation Graph

Luồng xử lý của hệ thống được xây dựng dựa trên từng lượt hội thoại và được điều khiển bởi Conversation Graph. Khi pentester gửi một yêu cầu mới, hệ thống khởi tạo một lượt hội thoại thông qua bước `create_chat_turn`. Tại bước này, thông tin cơ bản của lượt làm việc được ghi nhận, đồng thời chuẩn bị các cấu trúc cần thiết cho việc ghi log và theo dõi trạng thái xử lý.

Sau khi lượt hội thoại được khởi tạo, hệ thống tiến hành nạp ngữ cảnh thông qua bước `load_memories`. Memory Agent được kích hoạt để truy xuất các core memory tương ứng với phiên làm việc, message queue chứa các lượt hội thoại gần nhất và recall memory đã được tóm tắt từ các lượt hội thoại đã bị loại bỏ khỏi hàng đợi thông điệp. Việc nạp ngữ cảnh này giúp đảm bảo rằng mọi quyết định ở các bước sau đều được đặt trong ngữ cảnh hiện tại của phiên làm việc.

Tiếp theo, Core Orchestrator ở nút `orchestrator` tiếp nhận yêu cầu cùng với ngữ

cảnh đã được nạp và thực hiện phân tích. Dựa trên nội dung yêu cầu, Core Orchestrator quyết định liệu có cần gọi các subagent chuyên trách hay không. Nếu yêu cầu chỉ mang tính giải thích hoặc tổng hợp từ ngữ cảnh hiện có, hệ thống có thể tạo phản hồi trực tiếp và chuyển thẳng tới nút `complete_chat_turn` để kết thúc lượt hội thoại. Trong trường hợp cần truy xuất tri thức, tìm kiếm, lưu trữ trí nhớ hay thực thi công cụ, Core Orchestrator sẽ chuyển giao nhiệm vụ tới nút `tools` - nơi mà RAG Agent, Memory Agent hoặc Pentest Agent sẽ thực hiện công việc được giao. Quá trình này có thể lặp lại nhiều lần theo mô hình vòng lặp công cụ (tool loop) cho đến khi yêu cầu được giải quyết.

Khi không còn tác vụ nào cần thực hiện thêm, hệ thống chuyển sang bước hoàn tất lượt hội thoại - nút `complete_chat_turn`. Tại đây, phản hồi cuối cùng được tổng hợp và gửi về cho người dùng và được ghi lại vào hệ thống lưu trữ. Đồng thời, quá trình xử lý message queue cũng được thực hiện tại đây để lọc bỏ những cuộc hội thoại cũ làm vượt quá ngưỡng token cho phép. Luồng xử lý kết thúc khi trạng thái hội thoại được đánh dấu hoàn tất, sẵn sàng cho lượt tương tác tiếp theo.

4.3 Thiết kế agent

4.3.1 Danh mục các agent

Core Orchestrator là agent trung tâm chịu trách nhiệm điều phối toàn bộ luồng xử lý. Agent này được triển khai dưới dạng một nút trong Conversation Graph và thực thi thông qua hàm `orchestrator(...)`. Core Orchestrator tiếp nhận yêu cầu từ người dùng, kết hợp với trạng thái hiện tại của cuộc hội thoại, sau đó quyết định gọi các subagent thông qua cơ chế tool call của LangChain. Việc định tuyến được thực hiện thông qua hàm `route_to_agents(...)`, cho phép chuyển luồng xử lý sang các subagent phù hợp - Memory Agent, RAG Agent hoặc Pentest Agent tùy theo yêu cầu.

Memory Agent chịu trách nhiệm quản lý bộ nhớ hội thoại của hệ thống, bao gồm cả bộ nhớ dài hạn và bộ nhớ theo phiên làm việc. Agent này được cung cấp cho Core Orchestrator dưới dạng tool thông qua hàm `memory_agent_tool(...)`. Memory Agent đảm bảo rằng các thông tin quan trọng trong quá trình pentest đều được lưu trữ lâu dài, đồng thời duy trì sự liên tục của ngữ cảnh bằng cách tóm tắt và lưu trữ các thông điệp cũ khi hàng đợi thông điệp (message queue) vượt quá giới hạn về token.

RAG Agent là tác tử chuyên trách truy xuất dữ liệu từ kho tri thức kỹ thuật. Agent này được gọi thông qua hàm `rag_agent_tool(...)` khi Core Orchestrator xác định rằng yêu cầu cần bổ sung thông tin từ các tài liệu ngoại vi. RAG Agent thực hiện tìm kiếm trên kho tri thức, tổng hợp và rút gọn nội dung để cung cấp cho

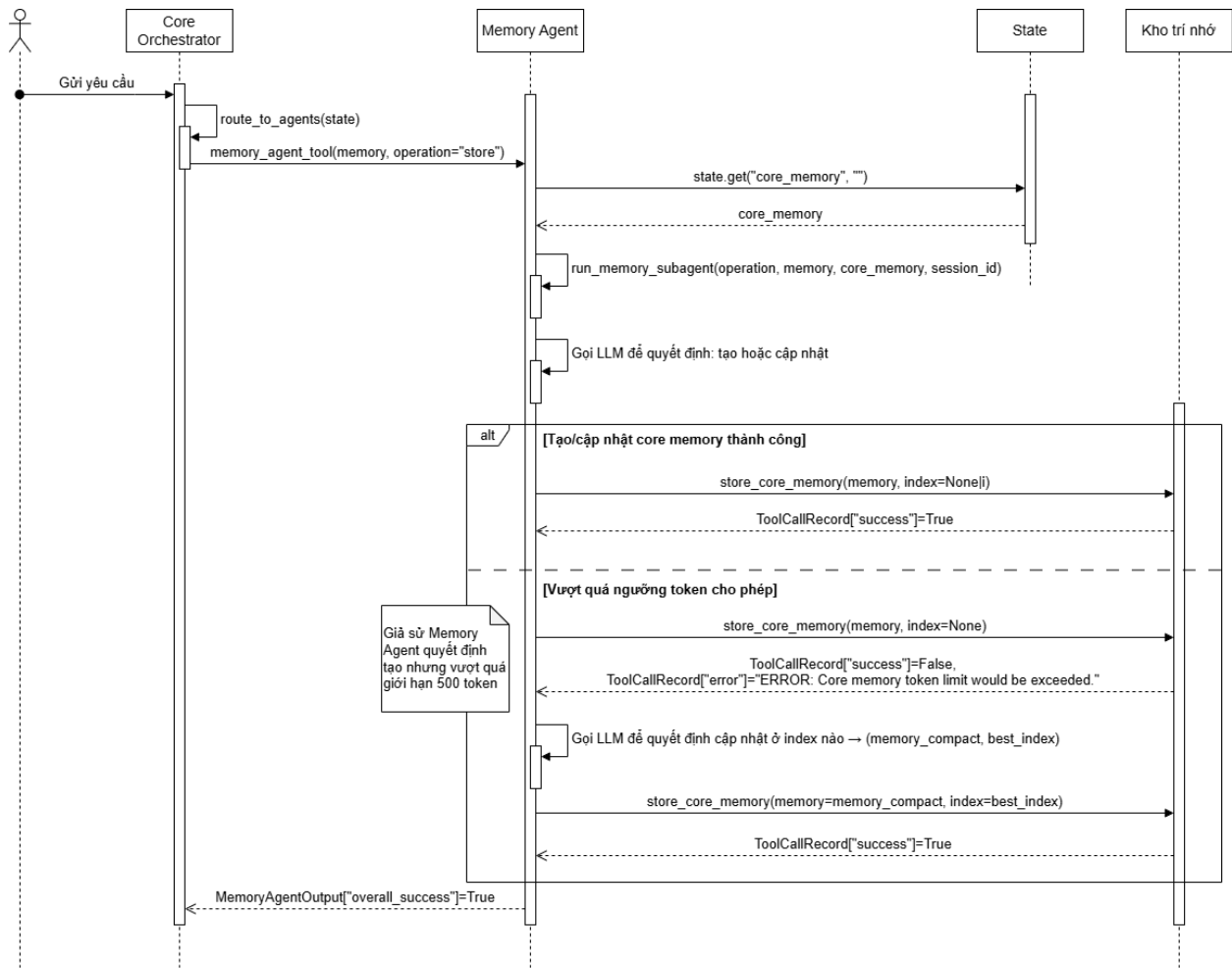
Core Orchestrator các thông tin phù hợp phục vụ cho quá trình suy luận.

Pentest Agent chịu trách nhiệm thực thi các thao tác kiểm thử bảo mật. Agent này được cung cấp cho Core Orchestrator dưới dạng tool thông qua hàm `pentest_agent_tool(...)`. Pentest Agent diễn giải chỉ dẫn của Core Orchestrator thành các hành động kỹ thuật cụ thể và thực hiện chúng thông qua MCP Server. Trong kiến trúc, Pentest Agent đóng vai trò như một MCP client, bảo đảm mọi hành động pentest đều được thực thi trong môi trường an toàn và biệt lập.

4.3.2 Chi tiết các hàm gọi Agent

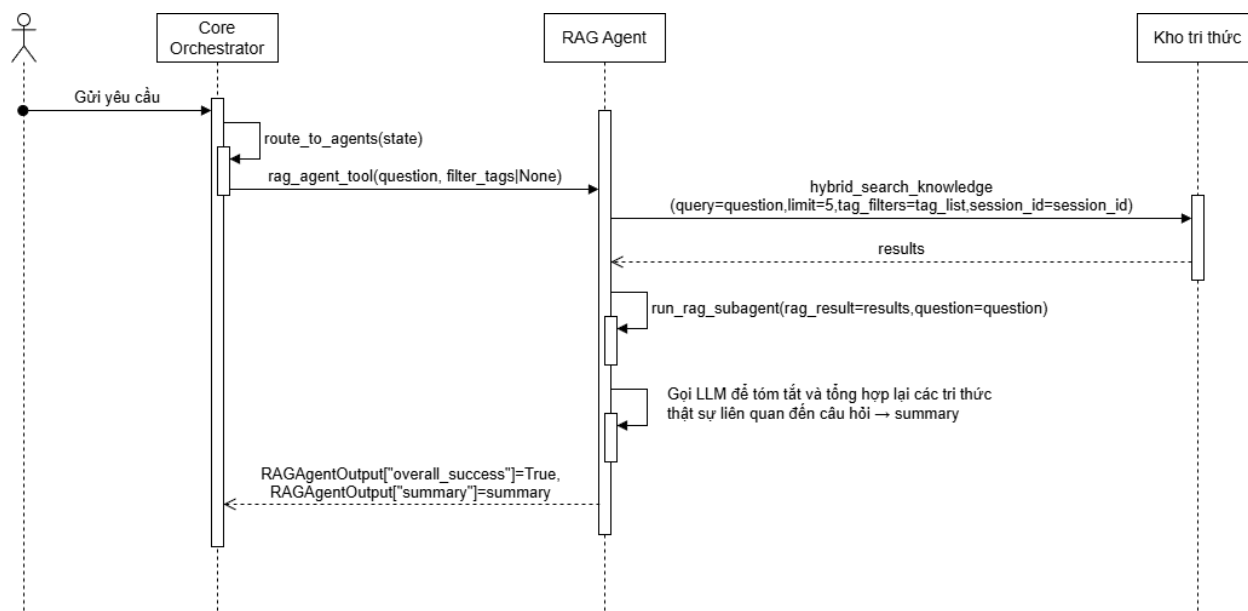
Để đảm bảo tính mô-đun hóa và khả năng thay thế linh hoạt, mỗi agent trong hệ thống được định nghĩa dưới dạng một tool với hợp đồng giao tiếp rõ ràng. Core Orchestrator chỉ tương tác với các agent thông qua chữ ký hàm (function signature), schema tham số đầu vào và cấu trúc dữ liệu trả về, không phụ thuộc vào logic triển khai nội bộ.

Memory Agent được định nghĩa thông qua hàm `memory_agent_tool(operation, memory, config, state)`. Tham số `operation` xác định loại thao tác cần thực hiện, bao gồm tìm kiếm trí nhớ (`search`) hoặc lưu trữ/cập nhật trí nhớ (`store`). Tham số `memory` chứa nội dung cần truy xuất hoặc lưu trữ, trong khi `config` và `state` cung cấp thông tin phiên làm việc và trạng thái hội thoại. Kết quả trả về tuân theo cấu trúc `MemoryAgentOutput`, bao gồm trường `summary` mô tả kết quả xử lý, danh sách `tool_calls` (nếu có), cờ `overall_success` thông báo trạng thái hoàn thành nhiệm vụ và trường `error` trong trường hợp xảy ra lỗi.



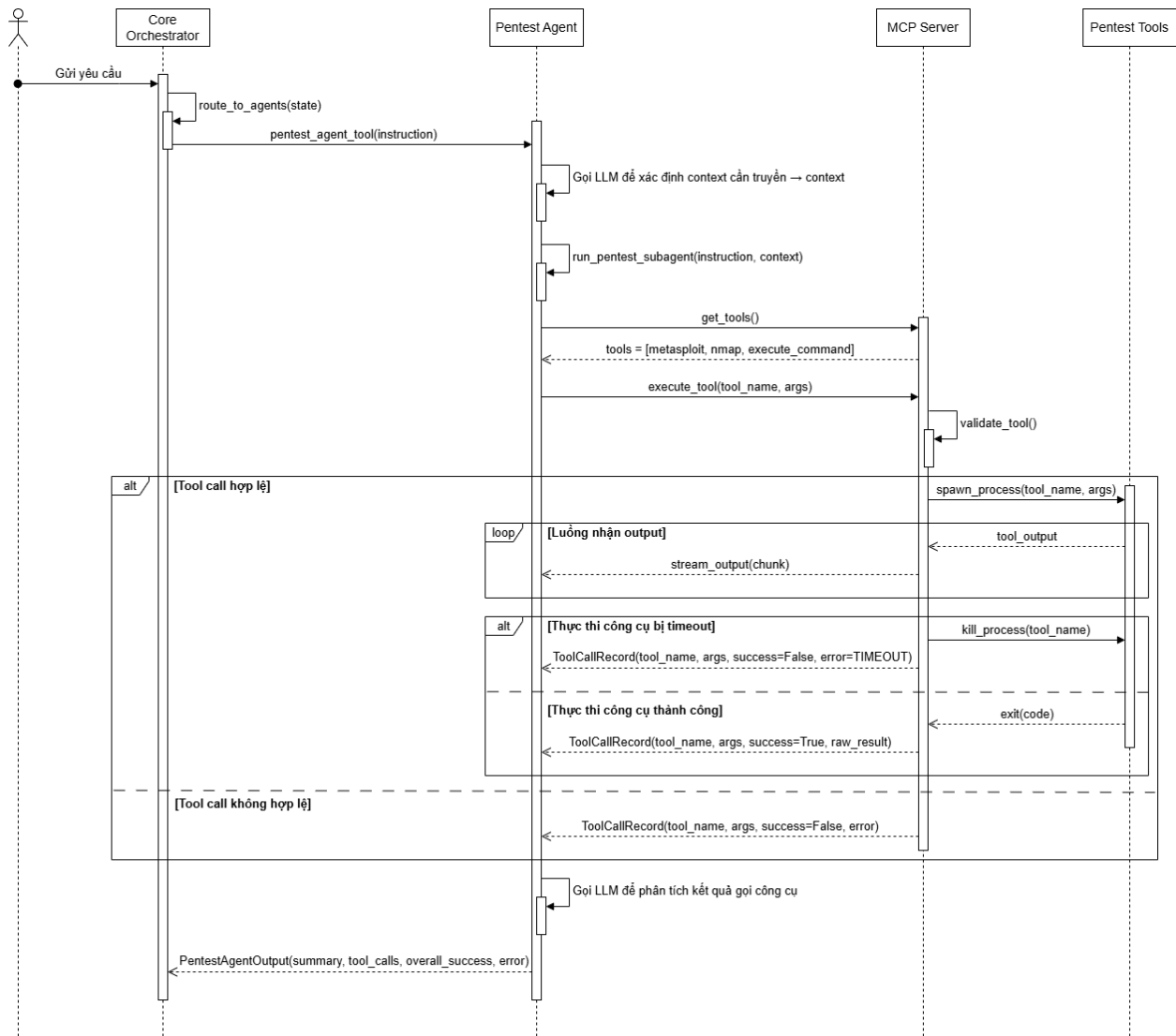
Hình 4.3: Biểu đồ trình tự cho quá trình lưu/cập nhật core memory

RAG Agent được cung cấp dưới dạng hàm `rag_agent_tool (question, config, state, filter_tags)`. Tham số `question` đại diện cho câu truy vấn tri thức, `filter_tags` cho phép giới hạn phạm vi tìm kiếm theo metadata, trong khi `config` và `state` đảm bảo truy xuất tri thức của đúng phiên làm việc hiện tại. Kết quả trả về của RAG Agent tuân theo cấu trúc `RagAgentOutput`, bao gồm trường `summary` là bản tóm tắt tri thức đã tổng hợp, `overall_success` nêu lên trạng thái thành công và `error` thông báo thông tin lỗi nếu truy xuất thất bại.



Hình 4.4: Biểu đồ trình tự cho quá trình Truy xuất dữ liệu từ kho tri thức

Pentest Agent được cung cấp thông qua hàm `pentest_agent_tool (instruction, config, state)`. Tham số `instruction` chứa chỉ dẫn thực thi bằng ngôn ngữ tự nhiên, có thể bao gồm thông tin về mục tiêu, phạm vi và kỹ thuật cần sử dụng. Pentest Agent sử dụng thông tin ngữ cảnh từ `state` để hỗ trợ quá trình thực thi. Kết quả trả về tuân theo cấu trúc `PentestAgentOutput`, bao gồm `summary` là bản tóm tắt kết quả thực thi công cụ, danh sách `tool_calls` đã được thực hiện, trạng thái thành công `overall_success` và `error` cho thông tin lỗi nếu quá trình thực thi gặp sự cố.



Hình 4.5: Biểu đồ trình tự quá trình Điều phối và Thực thi công cụ pentest

Trong trường hợp xảy ra lỗi, tất cả các agent đều tuân theo cùng một nguyên tắc xử lý là không phát sinh exception trực tiếp ra ngoài. Thay vào đó, lỗi được đóng gói vào trường `error` trong cấu trúc đầu ra và đánh dấu `overall_success` là `false`. Cách tiếp cận này cho phép Core Orchestrator xử lý lỗi một cách thống nhất, duy trì tính ổn định của hệ thống và cung cấp phản hồi phù hợp cho người dùng.

4.3.3 Thiết kế System Prompt cho Core Orchestrator Agent

Core Orchestrator đóng vai trò là một quản lý trung tâm với ba nhiệm vụ chính: phân tích yêu cầu, điều phối agent phù hợp và tổng hợp phản hồi cuối cùng. Thiết kế prompt sẽ tập trung vào nguyên tắc điều phối các agent và cách sử dụng hiệu quả ngữ cảnh được cung cấp.

Để không bị giới hạn bởi các nguyên tắc an toàn vốn có, system prompt của Core Orchestrator (`CORE_ORCHESTRATOR_PROMPT`) xác nhận mọi hành động nằm trong phạm vi cho phép của dự án, giúp agent hoạt động một cách có chủ

đích mà không bị chịu gò bó bởi những ràng buộc quá khắt khe. Không những thế, system prompt cũng định hình rõ vai trò quản lý của Core Orchestrator so với vai trò chuyên gia của các subagent, đảm bảo việc điều phối đúng người đúng việc. Hệ thống áp dụng giới hạn chi phí bằng cách cho phép tối đa ba lần gọi công cụ mỗi lượt, đóng vai trò như cơ chế kiểm soát chi phí và ngăn chặn vòng lặp vô hạn.

Các quy định về giới hạn của công cụ được mô tả chi tiết như một dạng tài liệu API tích hợp trong system prompt. Trước hết, prompt định nghĩa rõ đầu vào bắt buộc và kiểu đầu ra mà từng agent sẽ phản hồi. Tiếp theo, Core Orchestrator sẽ được hướng dẫn cách sử dụng các agent dựa trên context mà system prompt cung cấp, bao gồm core memory, recall memory và message queue. Các quy tắc ra quyết định được phân loại cụ thể như sau: đối với các tác vụ tương tác trực tiếp với mục tiêu được định tuyến sang Pentest Agent; đối với các truy vấn về kiến thức, kỹ thuật hoặc CVE được xử lý bởi RAG Agent; trong khi các yêu cầu truy hồi ngữ cảnh hoặc cập nhật tri thức mới sẽ do Memory Agent đảm nhận. Cuối cùng, yêu cầu đầu ra bắt buộc tính chính xác tuyệt đối đối với các thông số kỹ thuật và cấm việc tự ý bịa đặt thông tin.

Cách tiếp cận này giúp tách biệt rõ ràng giữa suy luận và thực thi, giảm thiểu rủi ro ảo giác khi thiếu ngữ cảnh. Các ràng buộc về số lần gọi công cụ và nguyên tắc chỉ gọi khi cần thiết giúp hệ thống vận hành ổn định và hiệu quả trong các kịch bản kiểm thử phức tạp.

4.3.4 Thiết kế System Prompt cho RAG Agent

RAG Agent được thiết kế chuyên biệt cho nhiệm vụ truy xuất tri thức từ kho tài liệu cục bộ, phục vụ các câu hỏi mang tính tra cứu hoặc giải thích kỹ thuật. system prompt của RAG Agent được thiết kế để giảm thiểu rủi ro hallucination thông qua một cơ chế ràng buộc chặt chẽ, yêu cầu mô hình chỉ xây dựng câu trả lời dựa trên ngữ cảnh đã được truy xuất.

Về đầu vào, hệ thống yêu cầu câu hỏi phải rõ ràng và chứa các thông tin định danh quan trọng như tên sản phẩm, phiên bản hoặc mã lỗi để tối ưu hóa quá trình tìm kiếm. Về đầu ra, agent chỉ được phép trả lời dựa trên các trích đoạn hoặc tóm tắt từ cơ sở tri thức thay vì tự suy luận. Một ràng buộc quan trọng được đặt ra tại tầng tổng hợp là cấm tuyệt đối việc bịa đặt thông tin; nếu không tìm thấy tài liệu phù hợp, hệ thống phải thừa nhận là không biết.

Để tối ưu hóa chất lượng đầu ra, hệ thống tích hợp một đoạn prompt chuyên biệt (RAG_SUMMARY_SYSTEM_PROMPT) cho bước tóm tắt ở giai đoạn hậu xử lý. Đoạn prompt này đóng vai trò như một bộ lọc giúp loại bỏ các ngữ cảnh không liên quan mà quá trình RAG tìm thấy, chỉ giữ lại các thông tin trả lời trực tiếp cho câu

hỏi và ưu tiên các đoạn văn bản có điểm xếp hạng cao. Cơ chế này giúp ngữ cảnh được bổ sung trở nên gọn gàng hơn, từ đó nâng cao độ chính xác và ổn định của câu trả lời cuối cùng.

4.3.5 Thiết kế System Prompt cho Memory Agent

Được thiết kế để hoạt động tách biệt với Core Orchestrator, Memory Agent đảm nhận vai trò quản lý luồng thông tin vào/ra bộ nhớ (lưu trữ và truy xuất). Để đạt hiệu suất tối ưu, system prompt được chia thành hai luồng xử lý riêng biệt: `MEMORY_SEARCH_SYSTEM_PROMPT` phục vụ tìm kiếm, trong khi `MEMORY_STORE_SYSTEM_PROMPT` phục vụ ghi nhớ. Việc chuyên biệt hóa ngữ cảnh cho từng tác vụ giúp loại bỏ sự nhập nhằng, hạn chế các suy đoán không cần thiết và đảm bảo tính nhất quán cao trong quản lý dữ liệu.

Đối với thao tác tìm kiếm, quy trình được thực hiện được prompt chuẩn hóa thành chuỗi các bước như sau: biến đổi yêu cầu thành truy vấn ngắn gọn giàu thông tin, sử dụng công cụ `search_memory(query=<optimized_query>)` để thực hiện tìm kiếm trên cơ sở dữ liệu véc-tơ, lọc kết quả và báo cáo tổng hợp thông tin nhận được. Cách làm này giúp kết quả truy xuất loại bỏ được các thông tin nhiễu không cần thiết và giải quyết được hiệu quả yêu cầu của Core Orchestrator.

Đối với thao tác lưu trữ và cập nhật, nguyên tắc tạo mới hoặc cập nhật được chi rõ nhằm tránh trường hợp trùng lặp dữ liệu. Hệ thống ưu tiên hợp nhất thông tin dựa trên mục tiêu hoặc phát hiện cụ thể để giữ cho core memory luôn cô đọng. Trong trường hợp bộ nhớ vượt quá giới hạn token, tác tử buộc phải chuyển sang chiến lược cập nhật vào các mục có sẵn thay vì tạo mới, đảm bảo tuân thủ ngân sách về token đã được đặt ra.

Song song đó, Memory Agent được trang bị thêm một system prompt chuyên biệt (`SUMMARY_SYSTEM_PROMPT`) nhằm thực hiện nhiệm vụ tóm tắt các thông tin trong message queue. Prompt này nhấn mạnh nhiệm vụ cô đọng lịch sử hội thoại thành các bản tóm tắt súc tích, đồng thời phải bảo toàn tính nguyên vẹn của các thông số kỹ thuật và phát hiện quan trọng. Qua đó, hệ thống đảm bảo duy trì tính liên tục của ngữ cảnh và ngăn chặn triệt để nguy cơ thất thoát thông tin trong các phiên làm việc kéo dài.

4.3.6 Thiết kế System Prompt cho Pentest Agent

Pentest Agent là agent thực thi chịu trách nhiệm tương tác với các công cụ kiểm thử thông qua giao thức MCP. Thiết kế prompt tập trung vào ba yêu cầu: tuân thủ phạm vi, đảm bảo độ chính xác và tối ưu hóa hiệu quả thực thi.

System prompt (`PENTEST_SYSTEM_PROMPT`) của Pentest Agent được thiết

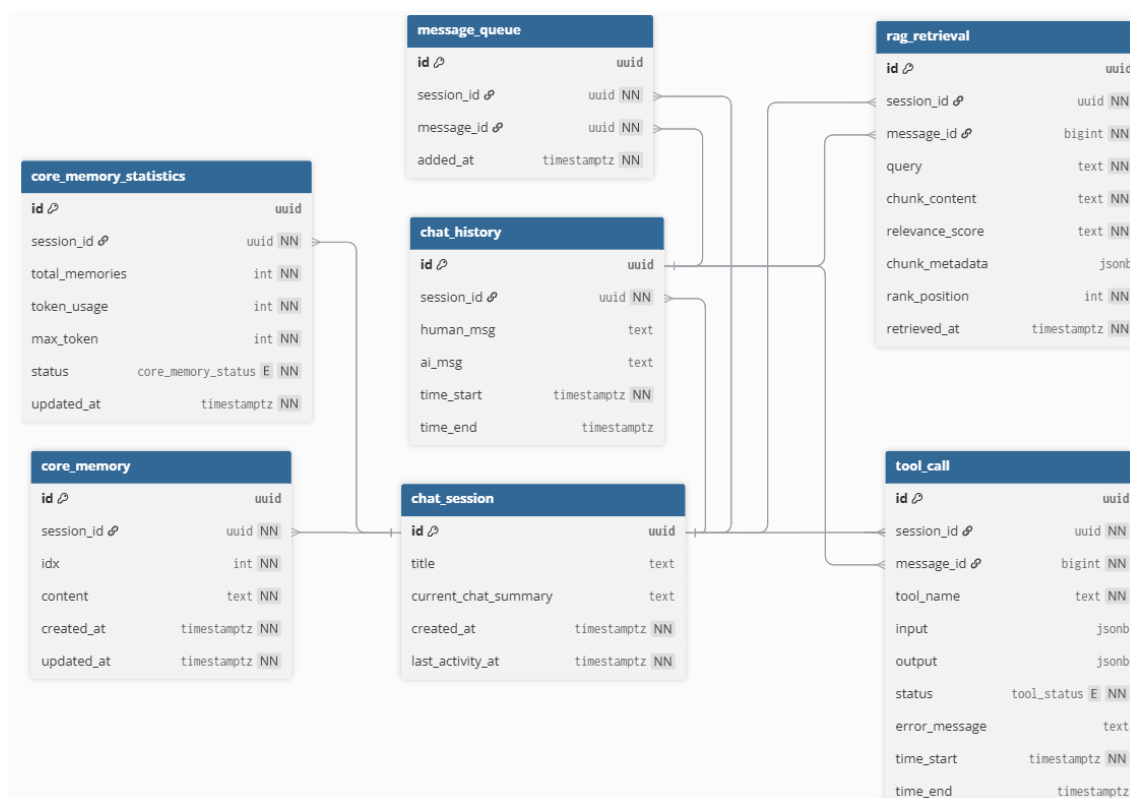
kế để áp dụng các biện pháp kiểm soát chặt chẽ nhất do đây là thành phần trực tiếp đối mặt với các rủi ro về bảo mật. Thứ nhất, phạm vi hoạt động bị giới hạn chỉ trong chỉ thị được tiếp nhận từ Core Orchestrator, ngăn chặn tuyệt đối việc suy đoán các mục tiêu hay lỗ hổng dẫn đến việc tấn công vào các tài nguyên không được cho phép. Thứ hai, các tham số kết nối (như trong kịch bản reverse shell) được truyền vào qua tham số môi trường để duy trì sự kiểm soát và ngăn chặn rủi ro cấu hình sai. Thứ ba, Agent hoạt động tách biệt hoàn toàn với ngữ cảnh hội thoại chính. Điều này buộc Core Orchestrator phải chuyển giao tường minh mọi dữ liệu cần thiết cho mỗi lần gọi, giúp quy trình kiểm thử luôn nằm trong tầm kiểm soát và dễ dàng truy vết.

Ngoài ra, system prompt cũng liệt kê chính xác các công cụ mà agent này được phép sử dụng, đồng thời nghiêm cấm hành vi bịa đặt tên công cụ hay tham số. Logic thực thi quy định rõ cách lựa chọn công cụ theo từng loại tác vụ, giới hạn số lần thử lại và cấm thực hiện các chuỗi hành động không được yêu cầu. Đầu ra được yêu cầu phải ngắn gọn, tập trung vào các phát hiện quan trọng như cổng dịch vụ, lỗ hổng hay thông tin xác thực và phải báo cáo rõ ràng các trường hợp thất bại.

Nhờ cách thiết kế này, Pentest Agent trở thành một bộ phận thực thi đáng tin cậy với hành vi có thể dự đoán được. Điều này hỗ trợ trực tiếp cho mục tiêu an toàn khi triển khai công cụ trong môi trường sandbox, đồng thời tạo thuận lợi cho việc kiểm soát hoạt động của hệ thống.

4.4 Thiết kế cơ sở dữ liệu

4.4.1 Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)



Hình 4.6: Sơ đồ thiết kế cơ sở dữ liệu

Cơ sở dữ liệu quan hệ PostgreSQL đóng vai trò lưu trữ trạng thái vận hành chính của hệ thống, bao gồm lịch sử hội thoại, bộ nhớ dài hạn có cấu trúc, hàng đợi thông điệp và nhật ký thực thi. Kiến trúc dữ liệu được thiết kế theo mô hình tập trung, xoay quanh thực thể phiên làm việc (`chat_session`), đảm bảo tính toàn vẹn khi tham chiếu và khả năng truy vết dữ liệu cho từng phiên kiểm thử xâm nhập cụ thể.

Bảng `chat_session` đóng vai trò trung tâm, đại diện cho một phiên làm việc độc lập của hệ thống. Mỗi bản ghi xác định duy nhất một phiên thông qua `id` (UUID) và là điểm neo để liên kết toàn bộ dữ liệu phát sinh trong quá trình hội thoại và kiểm thử. Các trường `title`, `current_chat_summary` phục vụ hiển thị và duy trì ngữ cảnh rút gọn, trong khi `created_at` và `last_activity_at` hỗ trợ quản lý vòng đời, sắp xếp và theo dõi trạng thái hoạt động của phiên.

Bảng `chat_history` lưu trữ chi tiết từng lượt trao đổi giữa người dùng và hệ thống trong một phiên cụ thể, thông qua khóa ngoại `session_id` tham chiếu đến trường `id` của bảng `chat_session`. Mỗi bản ghi phản ánh một vòng tương tác hoàn chỉnh (người - máy), kèm mốc thời gian bắt đầu và kết thúc để đo lường hiệu năng xử lý. Quan hệ một-nhiều giữa `chat_session` và `chat_history`

cho phép tái dựng đầy đủ cuộc hội thoại theo thứ tự thời gian.

Bảng `message_queue` được sử dụng để quản lý tập con các thông điệp đang nằm trong ngữ cảnh hoạt động của mô hình ngôn ngữ. Mỗi bản ghi liên kết một `message_id` (từ `chat_history`) với một `session_id`, phản ánh việc thông điệp đó đang tồn tại trong hàng đợi ngữ cảnh. Bảng này cho phép tách biệt lịch sử cuộc hội thoại với ngữ cảnh đang sử dụng, gia tăng hiệu năng truy xuất và hỗ trợ kiểm soát ngân sách token của cửa sổ ngữ cảnh.

Bảng `tool_call` ghi nhận toàn bộ các lần thực thi công cụ kỹ thuật trong quá trình pentest, chẳng hạn như quét, khai thác hoặc thu thập thông tin. Mỗi lần gọi công cụ được liên kết với một `session_id` và một `message_id` đã kích hoạt hành động đó. Ngoài ra bảng cũng lưu trữ `input`, `output` của công cụ dưới dạng JSONB cùng trạng thái và thời gian thực thi giúp đảm bảo khả năng truy vết và tái lập các thao tác kỹ thuật trong từng phiên làm việc.

Bảng `core_memory` đảm nhiệm vai trò lưu trữ bộ nhớ dài hạn của hệ thống theo từng phiên, bao gồm các thông tin cốt lõi đã được xác nhận trong quá trình pentest. Mỗi mục bộ nhớ được đánh chỉ số `idx` duy nhất trong phạm vi `session_id`, cho phép cập nhật hoặc hợp nhất nội dung một cách có kiểm soát. Quan hệ trực tiếp với `chat_session` giúp đảm bảo bộ nhớ lõi luôn gắn chặt với bối cảnh phiên tương ứng.

Bảng `core_memory_statistics` lưu trữ thông tin thống kê và trạng thái sử dụng core memory của mỗi phiên theo quan hệ một-một với `chat_session`. Các trường về số lượng core memory, mức tiêu thụ token và giới hạn token tối đa cho phép hệ thống theo dõi sát sao tình trạng ngữ cảnh. Trạng thái `active`, `near_limit` hoặc `at_limit` hỗ trợ Core Orchestrator và Memory Agent đưa ra quyết định lược bỏ hoặc rút gọn bộ nhớ khi cần thiết.

Bảng `rag_retrieval` ghi lại các lần truy xuất tri thức từ kho RAG trong quá trình trả lời người dùng. Mỗi bản ghi liên kết với một `session_id` và một `message_id` để xác định lượt hội thoại đã kích hoạt quá trình truy vấn. Việc lưu nội dung chunk (`chunk_content`), điểm liên quan (`relevance_score`), thứ hạng (`rank_position`) và metadata (`chunk_metadata`) cho phép hệ thống truy vết nguồn tri thức được sử dụng, đồng thời hỗ trợ giải thích và đánh giá chất lượng truy xuất trong từng phiên.

Bảng `rag_docs` lưu trữ thông tin về các tài liệu đã được nạp trong kho tri thức Qdrant (`knowledge_base`), với logic phân quyền dựa trên trạng thái của trường `session_id`. Nếu trường này chứa giá trị cụ thể, tài liệu được coi là dữ liệu riêng tư thuộc về phiên làm việc tương ứng. Ngược lại, giá trị `NULL` đánh dấu tài liệu đó

là tri thức chung (general knowledge), cho phép tất cả các phiên chat đều có quyền truy cập và sử dụng làm ngữ cảnh chung.

4.4.2 Thiết kế cơ sở dữ liệu véc-tơ (Qdrant)

Song song với cơ sở dữ liệu quan hệ, hệ thống sử dụng Qdrant làm cơ sở dữ liệu véc-tơ để phục vụ các tác vụ tìm kiếm ngữ nghĩa và truy xuất dựa trên véc-tơ nhúng. Dữ liệu trong Qdrant tập trung vào hai tập (collection) chính: knowledge base và recall memory.

Tập `knowledge_base` lưu trữ toàn bộ tri thức tĩnh được nạp vào hệ thống, bao gồm tài liệu pentest, CVE, write-up và hướng dẫn kỹ thuật. Mỗi điểm (point) trong collection này được biểu diễn bằng nhiều loại véc-tơ khác nhau (dense vector, sparse vector và late interaction vector), được sử dụng để phục vụ cho quá trình truy xuất RAG. Ngoài véc-tơ nhúng, mỗi điểm còn đi kèm payload chứa nội dung gốc và metadata chứa thẻ phân loại (filter tag), giúp RAG Agent lọc và xếp hạng kết quả chính xác hơn.

Tập `recall_memory` lưu trữ các cuộc hội thoại đã bị loại khỏi message queue. Mỗi bản ghi được gắn với một phiên làm việc cụ thể và được véc-tơ hóa để hỗ trợ truy vấn ngữ nghĩa. Thiết kế này cho phép Memory Agent nhanh chóng khôi phục các thông tin liên quan trong quá khứ, ngay cả khi chúng không còn nằm trong context window của LLM.

Cách tổ chức dữ liệu tách biệt giữa PostgreSQL và Qdrant giúp hệ thống tận dụng được ưu điểm của từng loại cơ sở dữ liệu: PostgreSQL đảm bảo tính nhất quán, truy vết và quản lý trạng thái, trong khi Qdrant tối ưu cho tìm kiếm ngữ nghĩa hiệu quả. Sự kết hợp này đóng vai trò nền tảng cho các ca sử dụng quản lý bộ nhớ và truy xuất tri thức đã được trình bày ở chương 2.

4.5 Xây dựng ứng dụng

4.5.1 Thư viện và công cụ sử dụng

Mục đích	Thư viện (phiên bản)	Địa chỉ URL
Framework xây dựng LLM Agent	langchain $\geq$ 1.0.8	https://pypi.org/project/langchain/
Thư viện mở rộng cho LangChain	langchain-community $\geq$ 0.4.1	https://pypi.org/project/langchain-community/
Tích hợp MCP	langchain-mcp-adapters $\geq$ 0.1.13	https://pypi.org/project/langchain-mcp-adapters/
Kết nối Ollama	langchain-ollama $\geq$ 1.0.0	https://pypi.org/project/langchain-ollama/
Điều phối Agent bằng đồ thị	langgraph $\geq$ 1.0.3	https://pypi.org/project/langgraph/
Thuật toán truy xuất văn bản	rank-bm25 $\geq$ 0.2.2	https://pypi.org/project/rank-bm25/
Tokenizer cho LLM	tiktoken $\geq$ 0.12.0	https://pypi.org/project/tiktoken/
Sinh embedding hiệu năng cao	fastembed $\geq$ 0.4.2	https://pypi.org/project/fastembed/
Cơ sở dữ liệu véc-tơ	qdrant-client $\geq$ 1.12.1	https://pypi.org/project/qdrant-client/
ORM và quản lý cơ sở dữ liệu	sqlalchemy $\geq$ 2.0.23	https://pypi.org/project/SQLAlchemy/
Quản lý biến môi trường	python-dotenv $\geq$ 1.0.0	https://pypi.org/project/python-dotenv/
Xử lý ngôn ngữ tự nhiên	nltk $\geq$ 3.8.1	https://pypi.org/project/nltk/
Embedding ngữ nghĩa	sentence-transformers $\geq$ 5.1.2	https://pypi.org/project/sentence-transformers/
Kết nối PostgreSQL	psycopg2 $\geq$ 2.9.11	https://pypi.org/project/psycopg2/
Xử lý và trích xuất tài liệu	docling $\geq$ 2.65.0	https://pypi.org/project/docling/
Phân tách văn bản cho LLM	langchain-text-splitters $\geq$ 1.0.0	https://pypi.org/project/langchain-text-splitters/
MCP Server	fastmcp $\geq$ 0.4.0	https://pypi.org/project/fastmcp/
Web server ASGI	uvicorn $\geq$ 0.30.0	https://pypi.org/project/uvicorn/
Tương tác với công cụ Metasploit	pymetasploit3 $\geq$ 1.0.6	https://pypi.org/project/pymetasploit3/
Tương tác với công cụ Nmap	python-nmap $\geq$ 0.7.1	https://pypi.org/project/python-nmap/
Giao diện Web	react $\geq$ 19.2.0	https://github.com/facebook/react
Thư viện UI cho React	antd $\geq$ 6.1.4	https://github.com/ant-design/ant-design
Websocket server	websockets $\geq$ 15.0.1	https://pypi.org/project/websockets/
Backend server	fastapi $\geq$ 0.128.0	https://pypi.org/project/fastapi/

Bảng 4.1: Danh sách thư viện và công cụ sử dụng

4.5.2 Kết quả đạt được

Sản phẩm chính của đề án là một hệ thống đa tác tử, bao gồm các thành phần cốt lõi như Core Orchestrator, các agent chuyên biệt (RAG Agent, Memory Agent, Pentest Agent), mô hình ngôn ngữ lớn chạy local, cùng hạ tầng hỗ trợ cho truy xuất tri thức, lưu trữ bộ nhớ dài hạn và thực thi công cụ kiểm thử. Hệ thống được đóng gói để có thể hoạt động theo mô hình client–server, cho phép triển khai linh hoạt trên nhiều máy trạm khác nhau, đồng thời đảm bảo tính cách ly và an toàn khi thực thi các công cụ pentest.

Về mặt ý nghĩa, sản phẩm của đề án góp phần giải quyết bài toán tự động hóa và hỗ trợ tri thức trong kiểm thử xâm nhập, đặc biệt trong các môi trường hạn chế kết nối Internet hoặc yêu cầu bảo mật cao. Việc tích hợp LLM với hệ thống RAG, cơ chế bộ nhớ dài hạn và giao thức MCP giúp giảm đáng kể khối lượng công việc của pentester, đồng thời nâng cao khả năng tổng hợp, phân tích và ghi nhớ thông tin xuyên suốt quá trình kiểm thử.

Bảng dưới đây trình bày thống kê tổng quan về mã nguồn của ứng dụng:

Ngôn ngữ lập trình	Số lượng file	Tổng số dòng code
Python	74	6 132
TypeScript	11	2 175
Tổng cộng	85	8 307

Bảng 4.2: Thống kê khối lượng mã nguồn của hệ thống

Ngoài ra, dung lượng toàn bộ mã nguồn của hệ thống, bao gồm cả các thư viện và phụ thuộc cần thiết cho quá trình triển khai, đạt khoảng 1.52 GB.

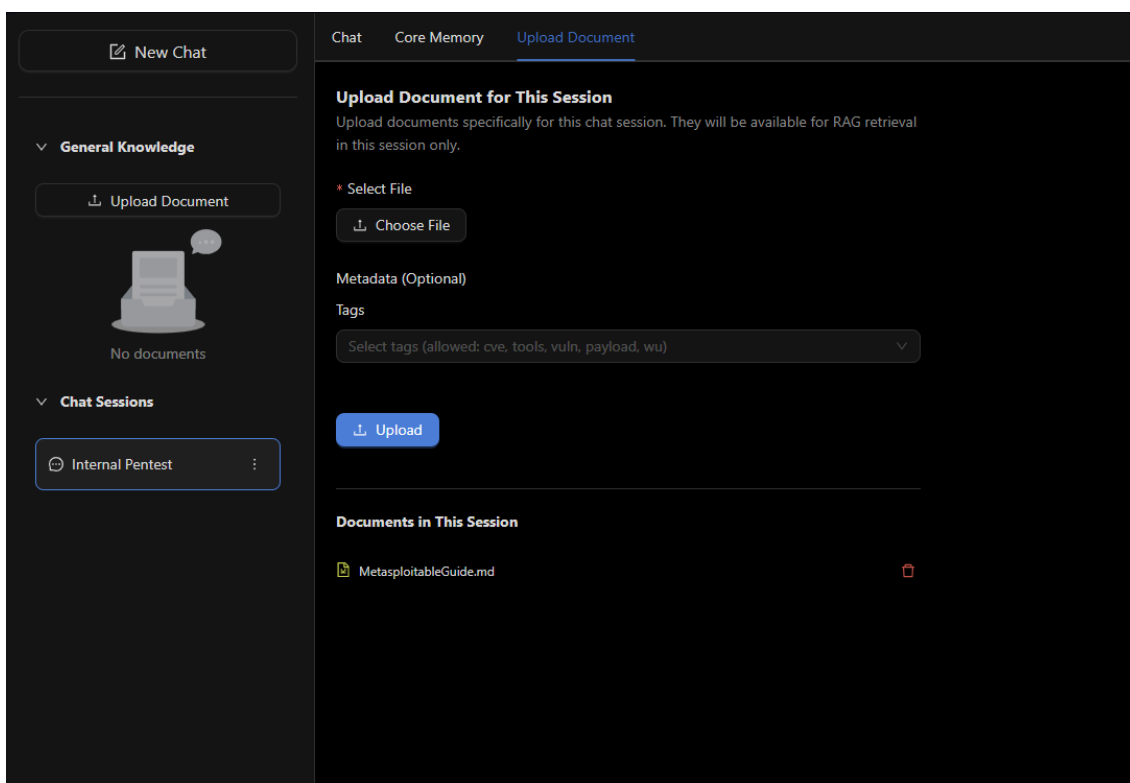
Hệ thống cũng đi kèm ba container Docker độc lập, mỗi container đảm nhiệm một vai trò rõ ràng trong kiến trúc tổng thể. Thông tin chi tiết được trình bày trong bảng sau:

Tên container	Docker image	Vai trò trong hệ thống	Dung lượng
pentest-postgres	postgres:16-alpine	Lưu trữ dữ liệu quan hệ, bộ nhớ dài hạn và log hệ thống	395 MB
pentest-qdrant	qdrant/qdrant:latest	Lưu trữ và truy vấn véc-tơ phục vụ cơ chế truy xuất tri thức tăng cường (RAG)	272 MB
kali-mcp-server	kalilinux/kali-rolling	Thực thi các công cụ pentest trong môi trường cách ly	4.61 GB

Bảng 4.3: Thông kê các container Docker của hệ thống

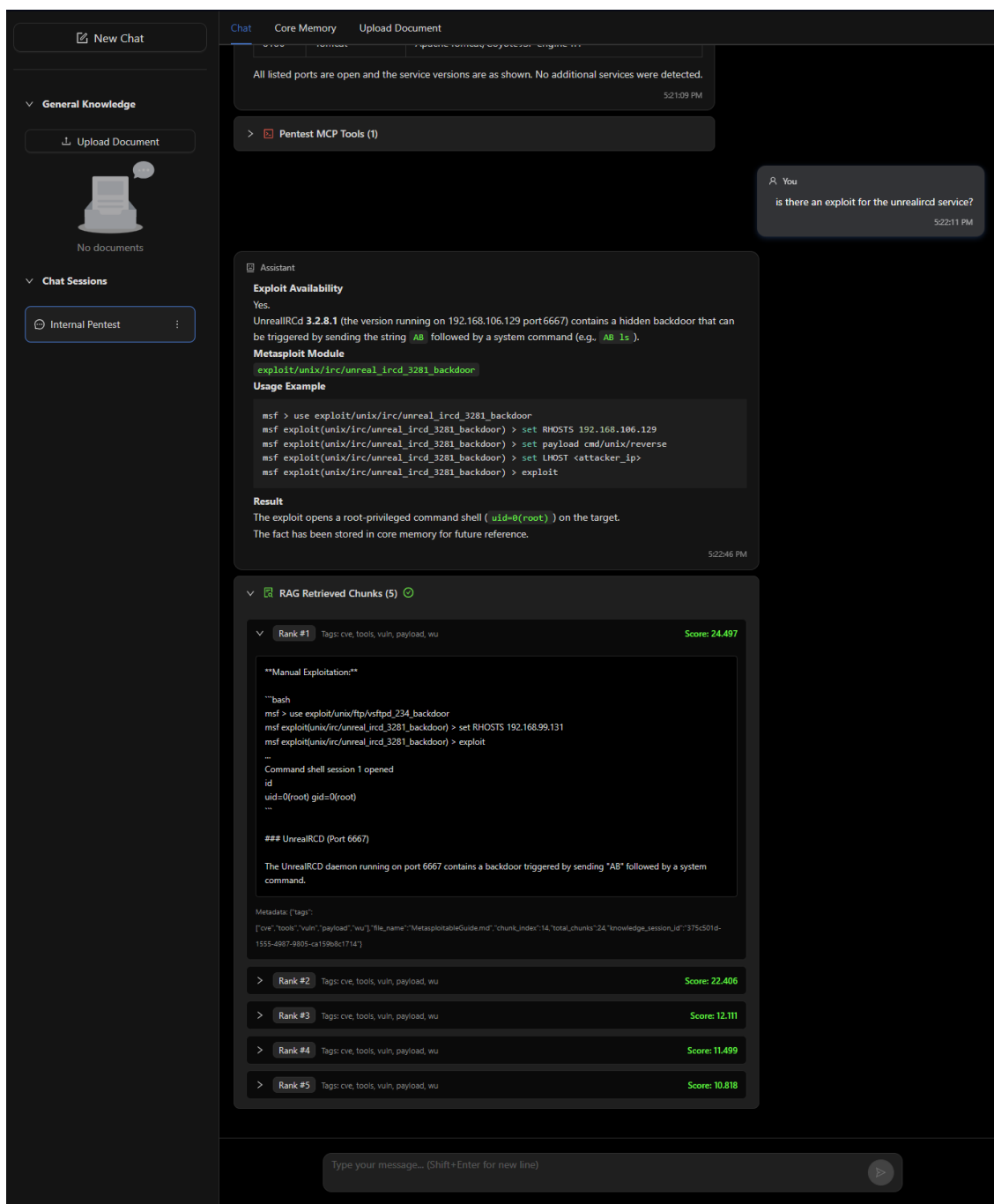
4.5.3 Minh họa các chức năng chính

Hệ thống hỗ trợ chức năng nạp và truy xuất tri thức dựa trên tài liệu nhằm gia tăng khả năng suy luận của LLM. Người dùng có thể tải lên các tài liệu dạng văn bản (như markdown, pdf,...) cho từng phiên làm việc thông qua giao diện “Upload Document” (Hình 4.7). Ngoài ra người dùng cũng có thể cung cấp tri thức chung cho agent thông qua giao diện “Upload Document” trong mục “General Knowledge” trên sidebar. Tài liệu nạp thông qua chức năng này sẽ được LLM sử dụng chung qua nhiều phiên khác nhau.



Hình 4.7: Minh họa kết quả nạp dữ liệu cho Kho tri thức của Agent theo phiên làm việc

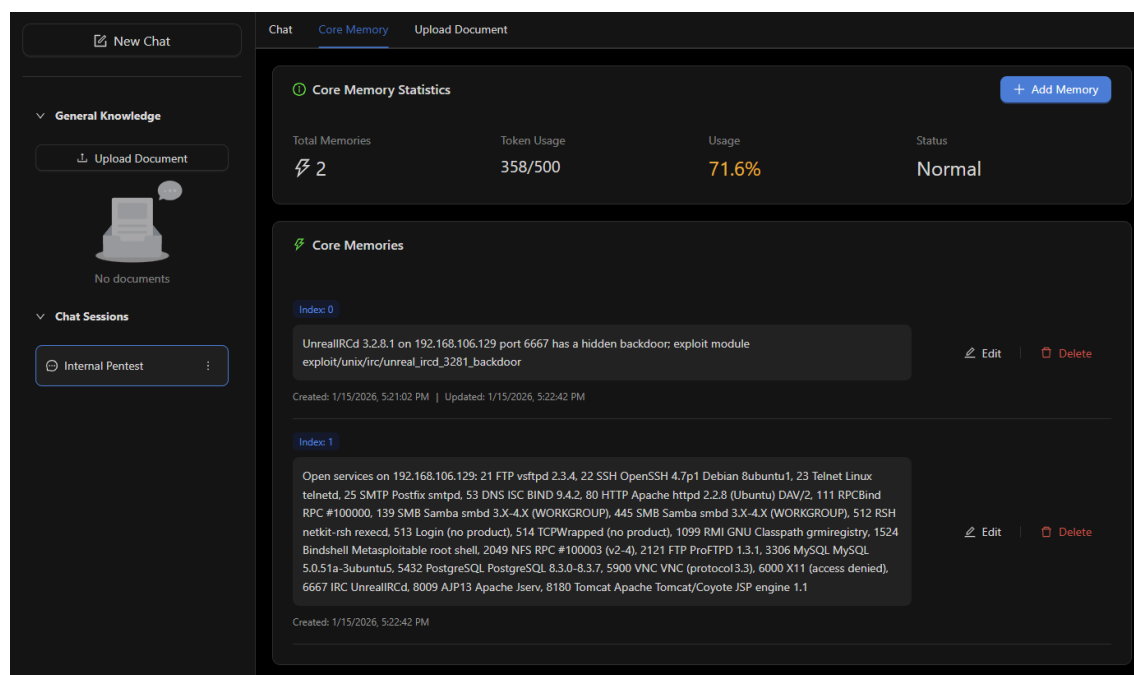
Các tài liệu sau khi được tiền xử lý và véc-tơ hóa sẽ được lưu trữ trong kho tri thức, cho phép agent thực hiện truy vấn ngữ nghĩa dựa trên dữ liệu được nạp vào. Khi truy xuất, hệ thống hiển thị các đoạn nội dung liên quan cùng với điểm xếp hạng mức độ tương đồng (Hình 4.8), giúp đánh giá rõ ràng mức độ phù hợp của từng nguồn tri thức được sử dụng trong quá trình trả lời.



Hình 4.8: Minh họa kết quả truy xuất RAG

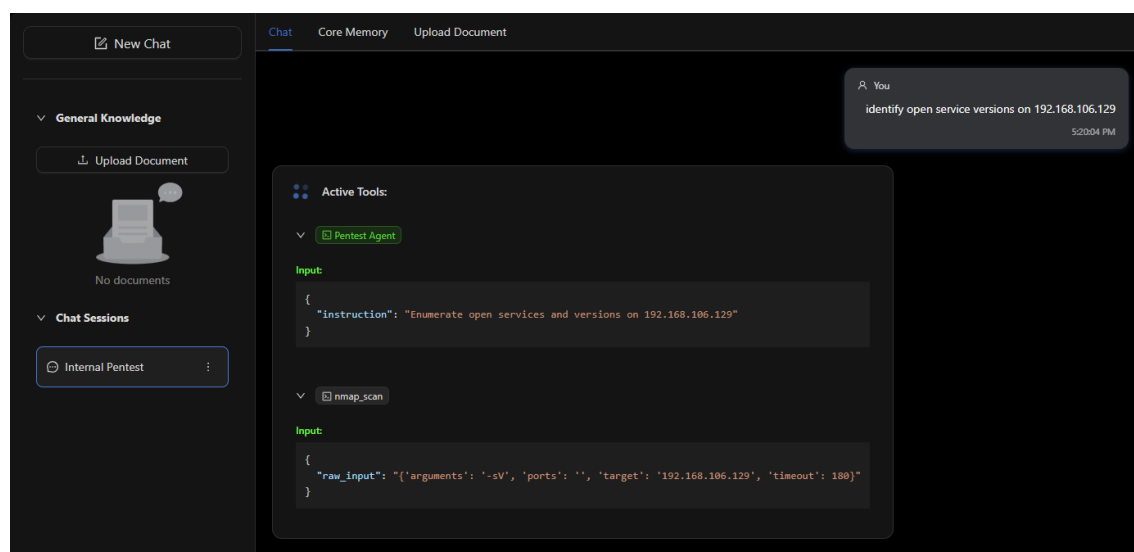
Bên cạnh đó, hệ thống cung cấp cơ chế quản lý Core Memory như một sổ tay ghi nhớ dành riêng cho agent trong suốt phiên làm việc. Thông qua giao diện Core Memory (Hình 4.9), các thông tin quan trọng như kết quả quét dịch vụ, lỗ hổng đã

xác định hoặc exploit khả dụng có thể được thêm mới, chỉnh sửa hoặc xóa. Core Memory đóng vai trò là bộ nhớ dài hạn ở mức phiên, giúp agent duy trì ngữ cảnh, tránh lặp lại các bước phân tích đã thực hiện và đảm bảo tính nhất quán trong toàn bộ quá trình kiểm thử xâm nhập.



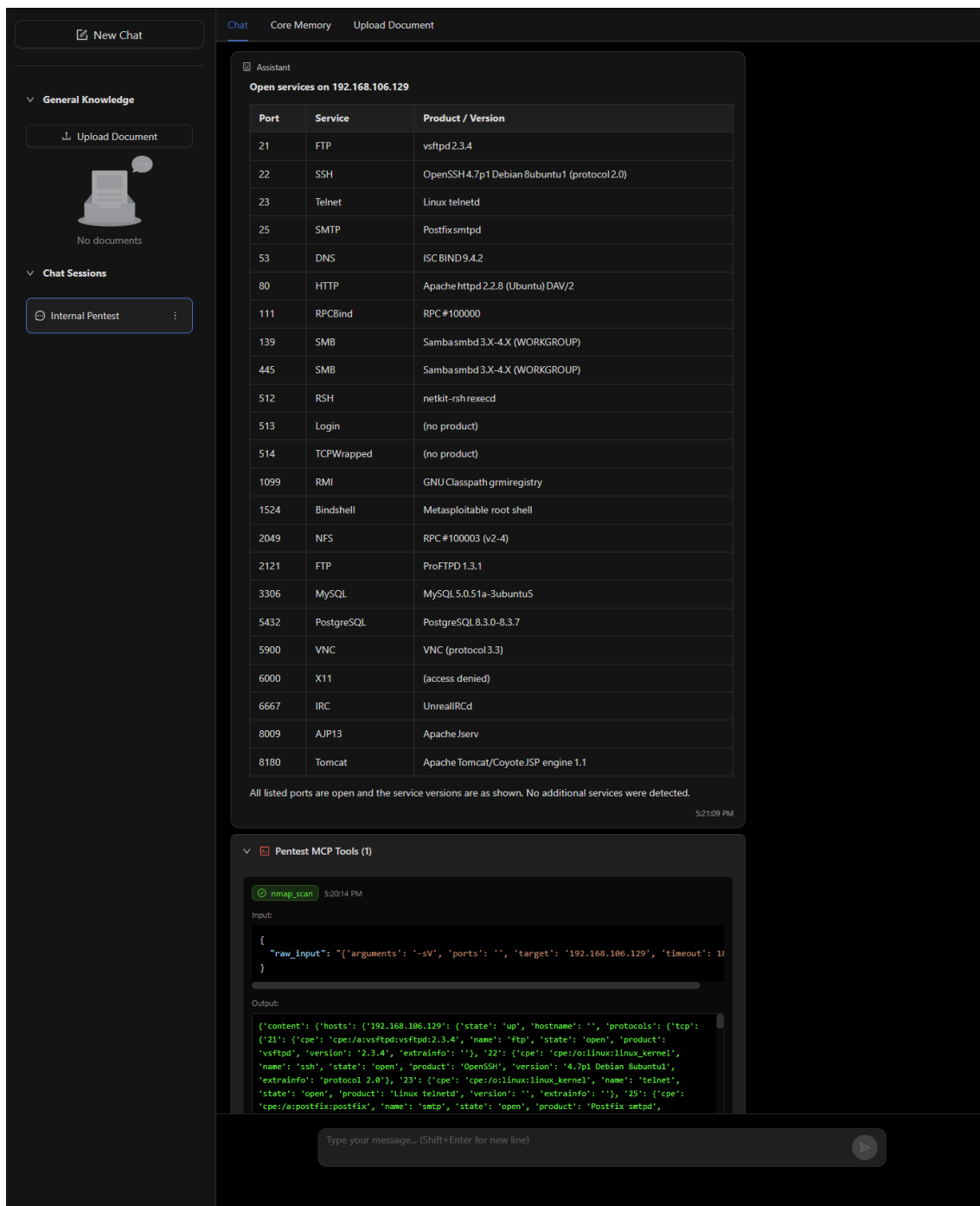
Hình 4.9: Minh họa giao diện quản lý core memory

Ngoài ra, hệ thống cho phép agent tự động thực thi các công cụ kiểm thử xâm nhập thông qua giao thức MCP streamable HTTP. Quá trình thực thi được hiển thị theo thời gian thực (Hình 4.10), cho phép người dùng theo dõi chi tiết hoạt động của agent cũng như các công cụ được gọi.



Hình 4.10: Minh họa giao diện hiển thị lời gọi công cụ theo thời gian thực

Sau khi phản hồi yêu cầu người dùng, hệ thống cũng hiển thị đầy đủ các lời gọi công cụ với thông tin đầu vào và đầu ra tương ứng (Hình 4.11).



Hình 4.11: Minh họa giao diện hiển thị lời gọi công cụ

4.6 Kiểm thử

Mục tiêu của hoạt động kiểm thử trong đồ án này là đánh giá mức độ chính xác, ổn định và khả năng phối hợp giữa các thành phần chính trong hệ thống LPAA. Khác với các phương pháp kiểm thử đơn vị (unit test) hoặc kiểm thử giả lập (mock-based test), đồ án tập trung triển khai kiểm thử tích hợp (integration testing) trên

hạ tầng thật, nhằm phản ánh sát nhất hành vi của hệ thống trong môi trường vận hành thực tế.

4.6.1 Chiến lược và kỹ thuật kiểm thử

Trong phạm vi đồ án, hệ thống áp dụng kỹ thuật kiểm thử tích hợp để đánh giá sự ổn định trong hành vi của các agent. Toàn bộ các trường hợp kiểm thử đều sử dụng: (i) Mô hình ngôn ngữ lớn chạy cục bộ thông qua Ollama, (ii) Cơ sở dữ liệu PostgreSQL thật để lưu trữ session, lịch sử hội thoại và bộ nhớ dài hạn, (iii) Kho véc-tơ Qdrant thật phục vụ cho cơ chế RAG, (iv) MCP Server chạy trên Kali Linux để thực thi các công cụ pentest (nmap, Metasploit), (v) Hệ thống mục tiêu thật trong môi trường lab để phục vụ kiểm thử đầu-cuối.

Việc sử dụng hạ tầng thật giúp đánh giá chính xác khả năng suy luận của LLM, năng lực điều phối của Core Orchestrator, cũng như độ tin cậy của các agent chuyên biệt khi phối hợp trong một quy trình pentest hoàn chỉnh.

4.6.2 Các trường hợp kiểm thử đã thực hiện

Dựa trên kiến trúc hệ thống, tôi lựa chọn bốn chức năng quan trọng nhất tương ứng với bốn agent cốt lõi để thiết kế và triển khai kiểm thử, bao gồm: khả năng điều phối agent, quản lý bộ nhớ dài hạn, truy xuất tri thức (RAG) và thực thi tác vụ pentest.

a, Kiểm thử khả năng điều phối của Core Orchestrator

Trường hợp kiểm thử này được thiết kế nhằm đánh giá khả năng của Core Orchestrator trong việc phân tích yêu cầu người dùng và điều phối đúng agent chuyên biệt tương ứng. Các truy vấn kiểm thử được xây dựng dựa trên ba ngữ cảnh tương ứng với ba agent khác nhau, bao gồm truy vấn mang tính tra cứu tri thức (RAG Agent), yêu cầu lưu trữ thông tin vào bộ nhớ dài hạn (Memory Agent) và yêu cầu thực thi các tác vụ kỹ thuật như quét dịch vụ hoặc thu thập thông tin hệ thống (Pentest Agent).

Trong quá trình kiểm thử, hệ thống lần lượt tiếp nhận các câu lệnh yêu cầu tra cứu kiến thức khai thác lỗ hổng, thực hiện quét mạng và ghi nhớ thông tin mục tiêu. Core Orchestrator sử dụng mô hình ngôn ngữ lớn chạy cục bộ để suy luận ý định của người dùng, từ đó lựa chọn và kích hoạt agent phù hợp (RAG Agent, Memory Agent hoặc Pentest Agent) thông qua các công cụ tương ứng.

```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "Routing" -s

=====
TEST: test_route_to_rag_agent_real_llm
=====

[USER INPUT] Look up your knowledge base on how to exploit UnrealIRCd 3.2.8.1 using Metasploit?
  ✓ Result contains messages
  ✓ Orchestrator returned 1 message(s)
  ✓ Orchestrator made tool call(s)

[LLM DECISION] Orchestrator routed to: ['rag_agent_tool']
  ✓ RAG agent tool was selected

[✓ PASS] RAG agent was correctly selected for knowledge query
=====

.
=====
TEST: test_route_to_pentest_agent_real_llm
=====

[USER INPUT] Identify open services and versions on 192.168.1.100
  ✓ Result contains messages
  ✓ Orchestrator returned 1 message(s)
  ✓ Orchestrator made tool call(s)

[LLM DECISION] Orchestrator routed to: ['pentest_agent_tool']
  ✓ Pentest agent tool was selected

[✓ PASS] Pentest agent was correctly selected for scanning task
=====

.
=====
TEST: test_route_to_memory_agent_real_llm
=====

[USER INPUT] Our target is at 192.168.106.129 and has SSH, FTP open. Remember this.
  ✓ Result contains messages
  ✓ Orchestrator returned 1 message(s)
  ✓ Orchestrator made tool call(s)

[LLM DECISION] Orchestrator routed to: ['memory_agent_tool']
  ✓ Memory agent tool was selected

[✓ PASS] Memory agent was correctly selected for storage request
=====

.
3 passed, 5 deselected in 4.10s
```

Hình 4.12: Kết quả kiểm thử khả năng điều phối của Core Orchestrator

Kết quả kiểm thử cho thấy Core Orchestrator có khả năng nhận diện chính xác ý định của người dùng và điều phối đúng agent trong tất cả các kịch bản đã triển khai. Đặc biệt, trong các tình huống yêu cầu mang tính kỹ thuật cao, hệ thống không đưa ra kết luận dựa trên suy đoán từ tri thức có sẵn mà ưu tiên sử dụng kết quả thực thi thực tế do Pentest Agent cung cấp làm nguồn thông tin đáng tin cậy, ngay cả khi các kết quả này mâu thuẫn với dữ liệu đã được lưu trữ trong bộ nhớ trước đó.

```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "ToolVsMem" -s

=====
TEST: test_tool_result_overrides_outdated_memory
=====

[OUTDATED CORE MEMORY]
[0] Target 192.168.106.129: FTP port 21 is CLOSED (filtered or not running).

[USER QUERY] Is FTP open on 192.168.106.129? Scan it to verify.
✓ Result contains messages
✓ Orchestrator returned 1 message(s)
✓ Orchestrator made tool call (not relying solely on memory)

[LLM DECISION] Orchestrator routed to: ['pentest_agent_tool']
✓ Pentest agent was called for fresh scan
✓ Graph execution completed

[FINAL LLM RESPONSE] Yes - the FTP service is open on **192.168.106.129**.
A quick Nmap scan (`-sV -p 21`) revealed:

- **Port 21** is **open**
- Service: **vsftpd 2.3.4**

The result has been recorded in core memory for future reference. If you'd like to proceed with exploitation or further enumeration, let me know!...
✓ Response references actual scan results
✓ Response contains FTP status from fresh scan

[✓ PASS] LLM prioritized fresh tool results over outdated core memory
=====

.
1 passed, 7 deselected in 26.13s
```

Hình 4.13: Kết quả kiểm thử khả năng ưu tiên kết quả thực thi so với ký ức cũ

b, Kiểm thử khả năng lưu trữ và cập nhật bộ nhớ của Memory Agent

Kịch bản kiểm thử này mô phỏng tình huống hệ thống đã có sẵn một bản ghi bộ nhớ liên quan đến một mục tiêu cụ thể, sau đó tiếp nhận thêm các thông tin mới có liên quan trực tiếp đến cùng mục tiêu đó. Các thông tin mới bao gồm chi tiết về dịch vụ đang chạy, phiên bản phần mềm và khả năng bị khai thác thông qua một mô-đun cụ thể.

Memory Agent được yêu cầu xử lý dữ liệu mới và quyết định cách lưu trữ phù hợp, bao gồm việc lựa chọn giữa tạo bản ghi mới hoặc cập nhật bản ghi hiện có trong bộ nhớ dài hạn. Toàn bộ quá trình được thực hiện với mô hình ngôn ngữ lớn thật và dữ liệu được lưu trữ trực tiếp vào cơ sở dữ liệu PostgreSQL.

```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "MemoryAgent" -s

=====
TEST: test_memory_store_with_real_llm
=====

[EXISTING MEMORY] [0] Target 192.168.1.100 has IRC with unrealircd version
[NEW MEMORY INPUT] Target 192.168.1.100 is vulnerable to unrealircd backdoor exploit via Metasploit
  ✓ Memory operation returned result
  ✓ Result is a dictionary
  ✓ Memory operation succeeded

[LLM RESPONSE] **Core memory entry created successfully.**

[STORED IN DB] Target 192.168.1.100 is vulnerable to unrealircd backdoor exploit via Metasploit
[MEMORY COUNT] 1 entry/entries
  ✓ All required keywords present in memory
  ✓ Target IP found in memory
  ✓ Memory consolidated to single entry

[✓ PASS] Memory consolidated with all required keywords: ['192.168.1.100', 'unrealircd', 'irc', 'backdoor', 'metasploit']
=====

.
1 passed, 7 deselected in 3.32s
```

Hình 4.14: Kết quả kiểm thử khả năng lưu trữ và cập nhật bộ nhớ của Memory Agent

Kết quả kiểm thử cho thấy Memory Agent không tạo ra các bản ghi rời rạc gây trùng lặp mà có khả năng hợp nhất và cập nhật thông tin một cách thông minh thành một bản ghi duy nhất. Bản ghi bộ nhớ sau khi cập nhật bao gồm đầy đủ các thông tin quan trọng (như địa chỉ IP mục tiêu, dịch vụ, phiên bản phần mềm, lỗ hổng tồn tại và phương thức khai thác tương ứng), qua đó chứng minh khả năng duy trì ngữ cảnh dài hạn và hỗ trợ hiệu quả cho quá trình suy luận của hệ thống trong các lượt tương tác tiếp theo.

c, Kiểm thử khả năng truy xuất và tổng hợp tri thức của RAG Agent

Trong trường hợp kiểm thử này, hệ thống tiếp nhận một truy vấn yêu cầu hướng dẫn khai thác một dịch vụ cụ thể dựa trên tri thức đã được nạp sẵn vào kho dữ liệu. Trước khi kiểm thử, các tài liệu kỹ thuật liên quan đến Metasploit và lỗ hổng UnrealIRCd đã được tiền xử lý và lưu trữ trong kho véc-tơ Qdrant.

RAG Agent thực hiện tìm kiếm ngữ nghĩa trên kho véc-tơ để truy xuất các tài liệu liên quan nhất, sau đó sử dụng mô hình ngôn ngữ lớn để tổng hợp và sinh phản hồi dựa trên nội dung đã truy xuất. Quá trình này cho phép hệ thống kết hợp chặt chẽ giữa tri thức bên ngoài và khả năng suy luận của mô hình ngôn ngữ.

```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "RagAgent" -s

=====
TEST: test_rag_summarization_real_llm
=====

[USER QUESTION] How to exploit UnrealIRCd 3.2.8.1?
[VECTOR STORE] Retrieved 3 relevant documents
  ✓ RAG agent returned result
  ✓ Result contains summary

[LLM ANSWER] **Exploiting UnrealIRCd 3.2.8.1 (backdoor "unreal_ircd_3281_backdoor")**

- **Metasploit method**

  ```bash
 msf > use exploit/unix/irc/unreal_ircd_3281_backdoor
 msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS <target_ip>
 msf exploit(unix/irc/unreal_ircd_3281_backdoor) > set paylo...

[VERIFICATION]
 ✓ Contains exploit module: exploit/unix/irc/unreal_ircd_3281_backdoor
 ✓ Contains payload: cmd/unix/reverse
 ✓ Exploit module assertion passed
 ✓ Payload assertion passed

[✓ PASS] RAG provided correct exploit information
=====

 ✓ Exploit module assertion passed
 ✓ Payload assertion passed

[✓ PASS] RAG provided correct exploit information
=====

.
1 passed, 7 deselected in 11.15s
```

**Hình 4.15:** Kết quả kiểm thử khả năng truy xuất và tổng hợp tri thức của RAG Agent

Kết quả kiểm thử cho thấy RAG Agent truy xuất đúng các tài liệu liên quan và tổng hợp chính xác nội dung kỹ thuật cần thiết.

### **d, Kiểm thử khả năng lựa chọn và thực thi công cụ của Pentest Agent**

Trường hợp kiểm thử này tập trung đánh giá khả năng của Pentest Agent trong việc lựa chọn công cụ phù hợp và thực thi các tác vụ an ninh mạng khi chỉ nhận được mô tả nhiệm vụ ở mức khái quát. Hệ thống tiếp nhận yêu cầu xác định các dịch vụ đang mở trên một mục tiêu mạng cụ thể mà không chỉ định trước công cụ cần sử dụng.

Dựa trên yêu cầu đầu vào, Pentest Agent suy luận loại tác vụ cần thực hiện và lựa chọn công cụ nmap thông qua MCP Server để tiến hành quét mạng và thu thập thông tin từ hệ thống mục tiêu. Quá trình thực thi được thực hiện trên hạ tầng thật, cho phép đánh giá chính xác hành vi của agent trong điều kiện vận hành thực tế.



```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "PentestAgent" -s

=====
TEST: test_pentest_tool_selection_real_llm
=====

[USER INSTRUCTION] Identify open services on 192.168.1.100
 ✓ Pentest agent returned result
 ✓ Result is a dictionary
 ✓ Result contains tool calls
 ✓ At least one tool was called

[LLM TOOL SELECTION] 1 tool(s) called
 1. nmap_scan

[VERIFICATION] ✓ nmap tool was called
[LLM SUMMARY] **Open services on 192.168.1.100**

- No TCP ports were reported as open.
- The host is reachable (state "up"), but the scan did not detect any listening services.

If you need a deeper scan (e.g....
 ✓ nmap tool assertion passed

[✓ PASS] Pentest agent correctly selected nmap for scanning
=====

.
1 passed, 7 deselected in 12.66s
```

**Hình 4.16:** Kết quả kiểm thử khả năng lựa chọn và thực thi công cụ của Pentest Agent

Kết quả cho thấy Pentest Agent không chỉ lựa chọn đúng công cụ mà còn có khả năng tổng hợp đầu ra của công cụ thành thông tin có cấu trúc, phục vụ cho các bước suy luận và báo cáo ở các giai đoạn tiếp theo của hệ thống.

### 4.6.3 Kiểm thử quy trình pentest của toàn bộ hệ thống multi-agent

Bên cạnh các trường hợp kiểm thử riêng lẻ cho từng agent, tôi triển khai một kịch bản kiểm thử đầu-cuối nhằm đánh giá khả năng phối hợp tổng thể của toàn bộ hệ thống. Kịch bản này mô phỏng một quy trình kiểm thử xâm nhập hoàn chỉnh, bao gồm các bước từ xác định dịch vụ đang chạy trên mục tiêu, lưu trữ thông tin thu thập được vào bộ nhớ dài hạn, truy xuất tri thức khai thác từ kho dữ liệu cho đến thực thi khai thác thông qua công cụ Metasploit.

```
PS D:\test\DATN\src> uv run pytest -q tests/test_integration_real_llm.py --use-real-llm -k "EndToEnd" -s

[STEP 1] Identifying service on port 6667 at 192.168.106.129...
 ✓ Scan was performed successfully

[STEP 2] Storing core memory about target...
 ✓ Memory was stored successfully

[STEP 3] Querying RAG for exploitation method...
 ✓ RAG provided correct exploit module
 ✓ RAG provided payload information

[STEP 4] Executing exploit and retrieving /etc/hosts...
 ✓ Exploit executed and /etc/hosts retrieved successfully

[LLM RESPONSE] **Exploit Summary**

- **Target:** 192.168.106.129 (UnrealIRCd on port 6667)
- **Module used:** `exploit/unix/irc/unreal_ircd_3281_backdoor`
- **Payload:** `cmd/unix/reverse` (reverse shell to 192.168.1.148:4444)
- **Result:** Root shell obtained (Session 3).

/etc/hosts content

...
127.0.0.1 localhost
127.0.1.1 metasploitable.localdomain metasploitable

The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts
...

The reverse shell was successfully established, and the `/etc/hosts` file was retrieved.
.
1 passed, 7 deselected in 91.50s (0:01:31)
```

**Hình 4.17:** Kết quả kiểm thử quy trình pentest của toàn bộ hệ thống multi-agent

Toàn bộ các bước được thực hiện trong nhiều lượt hội thoại liên tiếp, yêu cầu hệ thống duy trì trạng thái và ngữ cảnh xuyên suốt quá trình kiểm thử. Kết quả cho thấy các agent trong hệ thống phối hợp chặt chẽ với nhau dưới sự điều phối của Core Orchestrator, đồng thời hệ thống có khả năng hoàn thành một quy trình kiểm thử an ninh mạng hoàn chỉnh từ giai đoạn trinh sát đến khai thác thành công. Kết quả này khẳng định tính đúng đắn của kiến trúc multi-agent cũng như tính khả thi của giải pháp được đề xuất trong đề án.

### 4.6.4 Tổng hợp kết quả kiểm thử

Tổng cộng, tôi đã thiết kế và triển khai sáu trường hợp kiểm thử tích hợp, trong đó tập trung vào các chức năng cốt lõi nhất của hệ thống. Tất cả các trường hợp kiểm thử đều đạt kết quả mong đợi. Điều này cho thấy hệ thống đáp ứng tốt các yêu cầu chức năng đã đề ra, đặc biệt là khả năng điều phối agent, duy trì bộ nhớ dài hạn và tự động hóa quy trình pentest. Các hạn chế còn tồn tại chủ yếu liên quan đến thời gian thực thi dài của một số kiểm thử thực tế và sự rõ ràng của yêu cầu, đây cũng là các hướng cần được tối ưu và mở rộng trong tương lai.

#### 4.7 Triển khai

Trong quá trình triển khai hệ thống LPAA, một trong những thách thức quan trọng nhất là việc vận hành các mô hình ngôn ngữ lớn trong điều kiện tài nguyên phần cứng hạn chế. Đặc biệt, các mô hình hiện đại với hàng tỷ tham số thường yêu cầu dung lượng bộ nhớ GPU lớn cũng như băng thông tính toán cao, điều này gây khó khăn khi triển khai trên các thiết bị cá nhân thông dụng của pentester.

Trong khuôn khổ đề án, tôi đã tiến hành thử nghiệm chạy mô hình qwen3:4b-q4\_K\_M, một phiên bản mô hình Qwen3 4B đã được lượng tử hóa (lý thuyết về kỹ thuật này đã được nêu ở 3.3.1) ở mức 4-bit, trên laptop cá nhân sử dụng GPU RTX 3060 6GB. Kết quả thử nghiệm cho thấy mô hình được nạp thành công và có thể suy luận với tốc độ phản hồi nhanh, đáp ứng tốt về mặt thời gian xử lý. Tuy nhiên, qua quá trình sử dụng thực tế trong các kịch bản yêu cầu khả năng lập luận chuỗi tấn công, độ chính xác và chiều sâu lập luận của mô hình bị suy giảm rõ rệt. Một số phản hồi có xu hướng đơn giản hóa quá mức, bỏ sót ngữ cảnh quan trọng hoặc đưa ra quyết định sử dụng công cụ không hợp lý, cho thấy hạn chế cố hữu của việc lượng tử hóa đối với các mô hình cỡ nhỏ trong bài toán chuyên sâu về an toàn thông tin.

Để đảm bảo chất lượng suy luận và tính ổn định của hệ thống, tôi đã chuyển sang sử dụng một máy trạm PC với GPU RTX 3090 24GB để triển khai mô hình gpt-os:20b (được tiền huấn luyện ở mức lượng tử hóa MXFP4 [25]). Với dung lượng bộ nhớ lớn hơn và năng lực tính toán cao hơn, mô hình có thể chạy ở mức độ chính xác cao, cho phép xử lý hiệu quả các truy vấn dài, phức tạp và mang tính kỹ thuật chuyên sâu. Kết quả triển khai cho thấy thời gian phản hồi vẫn đảm bảo ở mức chấp nhận được, trong khi chất lượng câu trả lời được cải thiện đáng kể so với phương án chạy mô hình có tham số nhỏ trên GPU giới hạn. Điều này đặc biệt quan trọng đối với vai trò của LLM trong hệ thống, khi mô hình không chỉ trả lời câu hỏi mà còn tham gia điều phối agent, tổng hợp kết quả từ công cụ pentest và duy trì ngữ cảnh dài hạn.

Từ các kết quả thử nghiệm trên, hệ thống được đề xuất triển khai theo mô hình client-server. Cụ thể, một máy chủ trung tâm được trang bị GPU chuyên dụng sẽ đảm nhiệm việc chạy LLM thông qua nền tảng Ollama và cung cấp API nội bộ trong mạng LAN. Các pentester hoặc người dùng cuối có thể kết nối tới máy chủ này để khai thác năng lực suy luận của mô hình mà không cần sở hữu phần cứng cao cấp. Ở phía client, giao diện người dùng và Core Orchestrator có thể được triển khai linh hoạt trên từng máy cá nhân. Tương tự, MCP Server, chịu trách nhiệm thực thi các công cụ pentest, cũng có thể được cài đặt cục bộ trên từng máy hoặc triển

khai thành một server tập trung, tùy theo yêu cầu bảo mật và hạ tầng thực tế. Cách tiếp cận này giúp hệ thống đạt được sự cân bằng giữa hiệu năng, khả năng mở rộng và tính linh hoạt trong triển khai, đồng thời phù hợp với môi trường kiểm thử xâm nhập nội bộ.

### **Kết chương**

Tóm lại, chương 4 đã trình bày chi tiết quá trình thiết kế và xây dựng hệ thống LPAA. Thông qua việc áp dụng kiến trúc multi-agent theo mô hình sub-agents, hệ thống được tổ chức theo hướng mô-đun, phân tách rõ vai trò và trách nhiệm của từng thành phần. Chương cũng đã làm rõ thiết kế các agent, cơ sở dữ liệu kết hợp PostgreSQL và Qdrant, cùng với các chức năng chính của hệ thống.

Bên cạnh đó, các kịch bản và trường hợp kiểm thử cũng đã được xây dựng nhằm đánh giá khả năng hoạt động, tính ổn định và mức độ đáp ứng yêu cầu của hệ thống trong thực tế. Cuối cùng, việc triển khai hệ thống đã chứng minh tính khả thi của giải pháp đề xuất, tạo tiền đề cho Chương 5, nơi các giải pháp và đóng góp nổi bật của đồ án sẽ được trình bày và phân tích chi tiết.

## CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương này tổng hợp và phân tích các đóng góp nổi bật của hệ thống LPAA, bao gồm quyết định lựa chọn mô hình nhúng mã nguồn mở tối ưu cho cơ chế truy xuất RAG, phương pháp RAG cải tiến với Hybrid Search kết hợp Reranking, và thiết kế bộ nhớ dài hạn theo hướng MemGPT để duy trì trí nhớ của LLM. Trọng tâm của chương làm rõ những bài toán khó phát sinh trong quá trình xây dựng hệ thống, từ đó đề xuất cách tiếp cận giải quyết và lập luận kỹ thuật phía sau từng lựa chọn. Theo đó, mỗi đóng góp được trình bày thành một mục độc lập, lần lượt nêu bối cảnh/vấn đề, mô tả giải pháp đề xuất, và tổng kết kết quả đạt được hoặc tác động thực nghiệm (nếu có).

### 5.1 Đánh giá và lựa chọn Open-Source Embedding Models phục vụ cho hệ thống RAG

#### 5.1.1 Đặt vấn đề

Trong hệ thống RAG, mô hình nhúng là thành phần quyết định trực tiếp đến khả năng “tìm đúng tài liệu” cho truy vấn của người dùng. Tuy nhiên, do đặc thù dữ liệu thường chứa đầy đặc các thông số kỹ thuật và chuỗi định danh cần giữ nguyên dạng, đồng thời nguồn tri thức lại rất đa dạng về cấu trúc, dữ liệu thuộc lĩnh vực an toàn thông tin yêu cầu phải đặt ra những tiêu chuẩn truy xuất khắt khe hơn so với các lĩnh vực thông thường khác. Trong bối cảnh triển khai cục bộ, bài toán lựa chọn mô hình nhúng càng trở nên cấp thiết khi hệ thống phải đồng thời đáp ứng hai mục tiêu đối nghịch: tối ưu hóa chất lượng truy xuất nhưng vẫn đảm bảo chi phí tài nguyên chấp nhận được cho quá trình lưu trữ và truy xuất. Vì vậy, thay vì lựa chọn mô hình theo cảm tính hoặc theo xu hướng, đồ án đã xây dựng một quy trình đánh giá (benchmark) rõ ràng. Mục đích của quy trình này là để so sánh các mô hình nhúng mã nguồn mở dựa trên tập dữ liệu và phong cách truy vấn đặc thù của lĩnh vực an toàn thông tin, từ đó đưa ra quyết định lựa chọn mô hình phù hợp cho hệ thống RAG.

#### 5.1.2 Giải pháp đề xuất

Đồ án đã thành công xây dựng và áp dụng một quy trình benchmark nhằm đánh giá hiệu năng truy xuất của các mô hình nhúng mã nguồn mở cho hệ thống RAG. Về dữ liệu, thực nghiệm sử dụng bộ Cybersecurity Attack Dataset [26] của nhóm tác giả Tannu Barot, Savani Dhruv, Maispace được công bố trên Kaggle với quy mô khoảng 14,000 mẫu, phản ánh đa dạng ngữ cảnh của lĩnh vực an toàn thông tin như mô tả kỹ thuật tấn công, dấu hiệu nhận biết, các công cụ sử dụng,... Dựa trên nền tảng tập dữ liệu này, đồ án lựa chọn 8 mô hình nhúng mã nguồn mở đại diện

cho nhiều hướng tiếp cận khác nhau (nhóm BERT/biến thể BERT, nhóm mô hình nhúng theo phong cách LLM, v.v.) để so sánh và phân tích, cụ thể:

Mô hình nhúng	Architecture	Params	Ctx	Dim	Size
mxbai-embed-large	bert	334M	512	1024	669MB
nomic-embed-text	nomic-bert	137M	2048	768	274MB
bge-m3	bert	567M	8192	1024	1.2GB
bge-large	bert	334M	512	1024	670MB
qwen3-embedding:0.6b	qwen3	0.6B	32768	1024	639MB
qwen3-embedding:4b	qwen3	4B	40960	2560	2.5GB
embeddinggemma	gemma3	300M	2048	768	621MB
snowflake-arctic-embed	bert	334M	512	1024	669MB

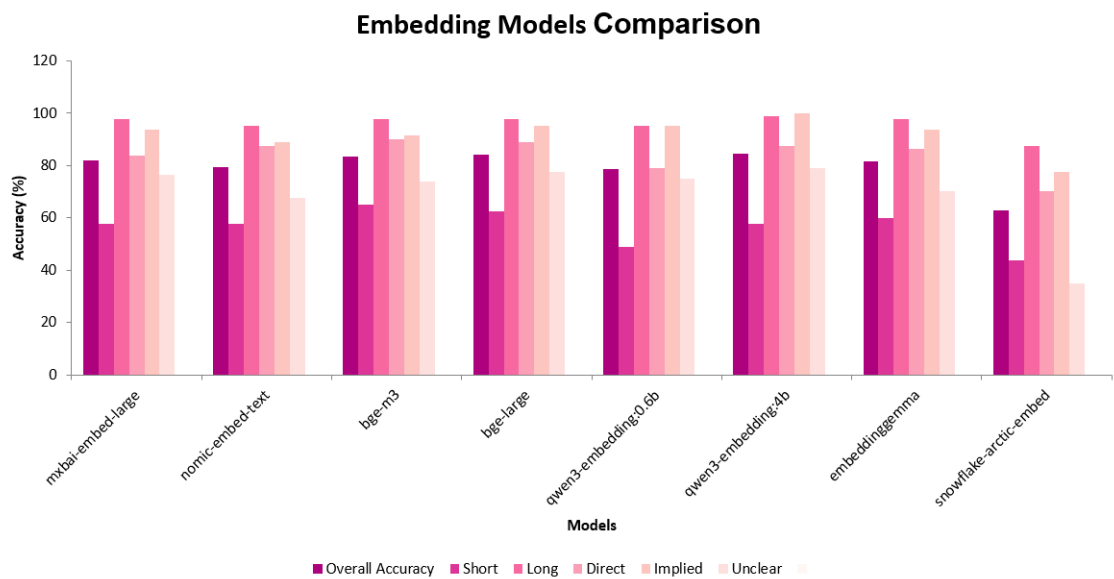
**Bảng 5.1:** Các mô hình nhúng mã nguồn mở được khảo sát

Về phương pháp thực hiện quá trình benchmark, trước hết đề án dựa trên tập dữ liệu an toàn thông tin nêu trên, lựa chọn ngẫu nhiên 20 đoạn nội dung (chunks) đại diện và xây dựng các câu hỏi tương ứng dựa trên chính các đoạn này nhằm tạo ra bộ đáp án gốc cho quá trình đánh giá về sau. Tiếp theo, với từng mô hình nhúng được đưa vào so sánh, toàn bộ dữ liệu và tập câu hỏi đều được véc-tơ hóa theo cùng một chiến lược phân đoạn thống nhất (mỗi đoạn 512 ký tự với 50 ký tự trùng lặp) để đánh giá công bằng hiệu suất truy vấn của từng mô hình. Trên cơ sở các véc-tơ thu được, hệ thống thực hiện truy vấn tương đồng (similarity search) giữa véc-tơ nhúng của câu hỏi và véc-tơ nhúng của các đoạn chunk trong cơ sở dữ liệu, lấy về danh sách 10 kết quả tốt nhất. Tại đây, hiệu năng truy xuất của mỗi mô hình sẽ được xác định bằng cách kiểm tra xem đoạn chunk gốc (được dùng để sinh câu hỏi) có xuất hiện trong danh sách 10 kết quả tốt nhất này hay không, từ đó tính toán độ chính xác truy xuất tổng thể, đồng thời phân rã kết quả theo từng nhóm câu hỏi (ngắn, dài, trực tiếp, hàm ý, mơ hồ) để đánh giá chi tiết khả năng biểu diễn ngữ nghĩa của mô hình dưới các dạng truy vấn khác nhau. Cụ thể danh sách các loại câu hỏi như sau:

Loại câu hỏi	Mô tả	Trọng tâm đánh giá
Câu hỏi ngắn	Câu hỏi đơn giản, trực tiếp, có độ dài dưới 10 từ	Hiểu ngữ nghĩa (kiểm tra khả năng hiểu cơ bản)
Câu hỏi dài	Câu hỏi chi tiết, toàn diện, chứa các thông tin cụ thể	Truy xuất theo ngữ cảnh (kiểm tra khả năng xử lý câu hỏi sâu và nhiều chi tiết)
Câu hỏi trực tiếp	Đề cập đến nội dung được nêu rõ trong văn bản	Hiểu ngữ nghĩa (kiểm tra khả năng truy xuất thông tin chính xác)
Câu hỏi hàm ý	Câu hỏi dựa trên ngữ cảnh, đòi hỏi khả năng suy luận	Truy xuất theo ngữ cảnh (kiểm tra khả năng hiểu ý nghĩa vượt ra ngoài nội dung hiển thị)
Câu hỏi không rõ ràng	Câu hỏi mơ hồ hoặc đa nghĩa	Kiểm tra cả hai tiêu chí đánh giá (khả năng của mô hình trong việc xử lý sự không chắc chắn và diễn giải ý nghĩa)

Bảng 5.2: Các loại câu hỏi và trọng tâm đánh giá

5.1.3 Kết quả đạt được



Hình 5.1: So sánh hiệu suất truy xuất của các mô hình nhúng

Kết quả thực nghiệm trên tập dữ liệu đã làm rõ mối tương quan giữa hiệu năng truy xuất và chi phí tài nguyên tính toán trong môi trường triển khai cục bộ. Số liệu thống kê cho thấy hai mô hình qwen3-embedding:4b và bge-large dẫn đầu về độ chính xác tổng thể, lần lượt đạt 84.50% và 84.25%. Đáng chú ý, mức chênh lệch không đáng kể (0.25%) giữa một mô hình có số lượng tham số lớn (4B) và một mô hình encoder cỡ trung bình như bge-large là minh chứng rõ ràng cho thấy việc gia tăng kích thước mô hình không đồng nghĩa với sự vượt trội về năng lực truy xuất. Thay vào đó, một mô hình được tối ưu hóa tốt như bge-large hoàn toàn có thể đạt

hiệu năng tương đương với mức tiêu thụ tài nguyên thấp hơn. Trong khi đó, bge-m3 dù đạt kết quả khả quan (83.50%) nhưng đòi hỏi dung lượng lưu trữ cho mô hình lớn hơn, chỉ phù hợp cho các kịch bản ưu tiên chất lượng thay vì hiệu suất vận hành.

Đi sâu phân tích theo đặc trưng của câu truy vấn, các mô hình bộc lộ những ưu thế riêng biệt. qwen3-embedding:4b thể hiện sự vượt trội ở các câu hỏi dài và chứa hàm ý ẩn, đạt độ chính xác ấn tượng lần lượt là 98.75% và 100%. Ngược lại, bge-m3 chứng tỏ khả năng xử lý tốt hơn đối với các truy vấn ngắn (65%) – loại dữ liệu thường gây khó khăn cho các mô hình nhúng do thiếu ngữ cảnh và tính đa nghĩa. Tuy nhiên, đối với nhóm câu hỏi không rõ ràng, hiệu năng giữa qwen3-embedding:4b (78.75%) và bge-large (77.50%) là gần như tương đương. Điều này củng cố nhận định rằng hiệu năng truy xuất mà mô hình 4B mang lại là không đủ lớn để đánh đổi lấy chi phí tài nguyên gia tăng trong bối cảnh ứng dụng thực tế.

Dựa trên cơ sở phân tích nêu trên, đồ án quyết định lựa chọn bge-large làm mô hình nhúng tiêu chuẩn cho hệ thống RAG cục bộ. Đây là giải pháp đạt được điểm cân bằng tối ưu giữa độ chính xác cao và chi phí tài nguyên khi triển khai. Các mô hình nhẹ hơn như nomic-embed-text hoặc embeddinggemma đạt yêu cầu để làm phương án dự phòng cho các môi trường có tài nguyên phần cứng hạn chế hơn. Trong khi đó, snowflake-arctic-embed bị loại bỏ khỏi danh sách ứng viên do năng lực xử lý hạn chế đối với các truy vấn ngữ nghĩa phức tạp vốn là đặc thù của tập dữ liệu thuộc lĩnh vực an toàn thông tin.

## 5.2 Hybrid Search kết hợp Reranking nhằm cải thiện truy xuất trong RAG

### 5.2.1 Đặt vấn đề

Trong các kiến trúc RAG hiện đại, cơ chế truy xuất dựa trên véc-tơ (Dense Retrieval) thường được xem là giải pháp tiêu chuẩn nhờ khả năng nắm bắt ngữ nghĩa vượt trội. Tuy nhiên, khi áp dụng phương pháp này vào miền dữ liệu đặc thù như an toàn thông tin, những hạn chế bắt đầu bộc lộ. Do bản chất của véc-tơ hóa là nén và trừu tượng hóa ngữ nghĩa, Dense Retrieval thường gặp khó khăn trong việc bảo toàn độ chính xác từ vựng - yếu tố quan trọng đối với dữ liệu pentest. Việc các thông số định danh như mã CVE, phiên bản dịch vụ hay cú pháp lệnh bị “hòa lẫn” trong không gian véc-tơ khiến hệ thống dễ bỏ sót các chỉ dẫn quan trọng hoặc truy xuất nhầm các kết quả có ngữ nghĩa tương đồng nhưng sai lệch hoàn toàn về mặt kỹ thuật. Sự thiếu chính xác ở tầng truy xuất này tạo ra rủi ro dây chuyền trong các kịch bản tấn công phức tạp: chỉ thị thiếu dữ liệu buộc LLM phải suy diễn, dẫn đến các quyết định sai lầm hoặc các vòng lặp thao tác vô nghĩa. Vì vậy, mục tiêu của đồ án là xây dựng một chiến lược truy xuất tối ưu hơn, đảm bảo sự cân bằng giữa tìm kiếm ngữ nghĩa và tìm kiếm từ khóa. Đây là bước đi then chốt nhằm giảm thiểu sai



số hệ thống và đảm bảo các phản hồi của agent có tính xác thực cao hơn.

### 5.2.2 Giải pháp đề xuất

Trong các hệ thống RAG hiện đại, tầng truy xuất đóng vai trò quyết định trong việc định vị ngữ cảnh mục tiêu thông qua các cơ chế biểu diễn dữ liệu khác nhau, mỗi cơ chế đều tồn tại những ưu điểm và hạn chế riêng biệt. Phương pháp phổ biến nhất là dense retrieval, nơi câu truy vấn và tài liệu được ánh xạ thành các véc-tơ liên tục để đo lường độ tương đồng ngữ nghĩa. Mặc dù cơ chế này nắm bắt tốt các câu hỏi mang tính khái quát, đồng nghĩa, hoặc diễn đạt khác nhau nhưng cùng ý, nó thường có xu hướng “làm mờ” các thông tin kỹ thuật đặc thù của ngành an toàn thông tin như mã CVE, phiên bản phần mềm, tham số dòng lệnh hay payload.

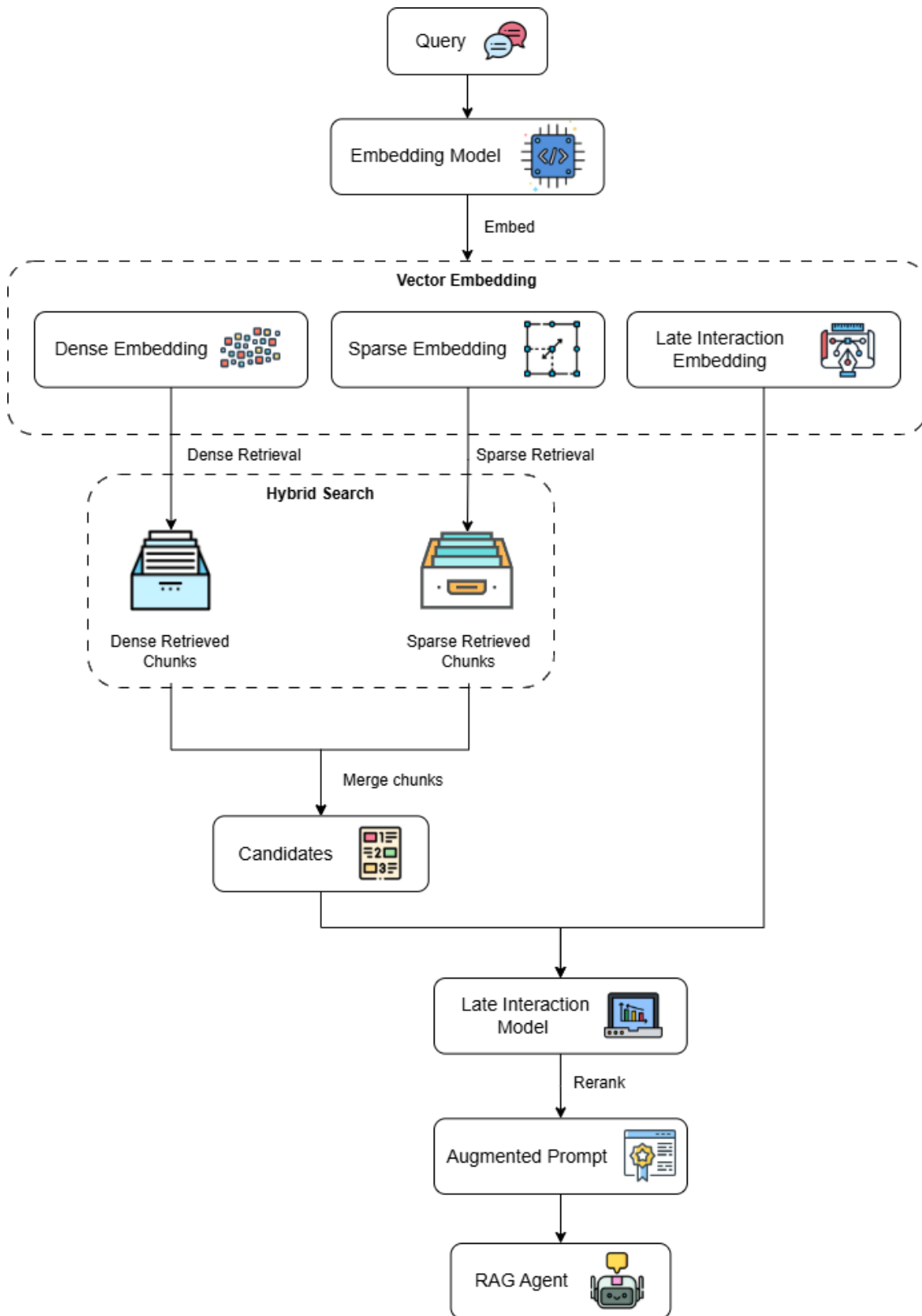
Ngược lại, sparse retrieval (điển hình là BM25) biểu diễn văn bản dựa trên tần suất từ khóa và trọng số IDF (Inverse Document Frequency - trọng số đánh giá sự quan trọng của một từ khóa thông qua độ phổ biến của nó trong tập dữ liệu), rất mạnh trong việc khớp chính xác (exact match) các chuỗi kỹ thuật nhưng lại hạn chế trong việc thấu hiểu ngữ nghĩa của câu từ.

Bổ sung cho hai hướng tiếp cận này là mô hình late interaction embeddings [27] (như ColBERT), thay vì nén cả tài liệu thành một véc-tơ duy nhất, late interaction lưu embedding theo từng token và chỉ thực hiện bước “tương tác muộn” khi xếp hạng (rerank): với mỗi token của truy vấn, hệ thống so khớp với mọi token của tài liệu, lấy điểm tương đồng lớn nhất (MaxSim), rồi cộng dồn để ra điểm liên quan cuối (relevance score). Cách này thường cho chất lượng rerank tốt vì giữ được ngữ cảnh chi tiết, nhưng đánh đổi bằng chi phí lưu trữ lớn hơn, do đó phù hợp nhất để dùng như một tầng Reranking trên tập ứng viên đã được lọc trước qua cơ chế Hybrid Search.

Hybrid Search [28] là kỹ thuật truy xuất kết hợp đồng thời hai hướng: dense retrieval (véc-tơ ngữ nghĩa) và sparse/keyword retrieval (ví dụ BM25). Dense giúp hiểu đúng ý của truy vấn theo ngữ cảnh, còn sparse đảm bảo khớp chính xác các từ khóa quan trọng. Khi truy vấn, hệ thống chạy song song hai phép tìm kiếm này, sau đó dùng một thuật toán hợp nhất thứ hạng (rank fusion) như Reciprocal Rank Fusion – RRF để trộn hai danh sách kết quả thành một danh sách xếp hạng chung. RRF ưu tiên các tài liệu được xếp hạng cao ở ít nhất một trong hai danh sách và “phạt” (penalize - chấm điểm thấp các tài liệu này trong danh sách) các tài liệu xếp hạng thấp, nhờ đó đảm bảo kết quả cuối vừa đúng ngữ nghĩa vừa không bỏ sót các mục khớp từ khóa quan trọng.

Để khắc phục các hạn chế nêu trên, đề án đề xuất áp dụng chiến lược Hybrid Search kết hợp với cơ chế Reranking. Theo đó, dữ liệu trong Qdrant được tổ chức

dưới nhiều dạng biểu diễn khác nhau, trong đó mỗi phân đoạn (chunk) được lưu trữ song song trên ba kênh véc-tơ riêng biệt nhằm phục vụ các mục tiêu truy xuất khác nhau: (i) Dense Embedding giúp nắm bắt ngữ nghĩa tổng quát của văn bản, (ii) Sparse Embedding đảm nhiệm việc khớp chính xác các từ khóa kỹ thuật và (iii) Late Interaction Embedding phục vụ cho giai đoạn tái xếp hạng (reranking) để nâng cao độ chính xác cuối cùng. Cụ thể, cơ chế vận hành của quy trình truy xuất bắt đầu bằng việc chuyển đổi câu hỏi đầu vào thành ba loại véc-tơ tương ứng (dense, sparse và late interaction). Dựa trên các véc-tơ này, hệ thống thực thi đồng thời hai nhánh tìm kiếm bổ trợ lẫn nhau: nhánh Dense Retrieval trả về danh sách dựa trên sự tương đồng ngữ nghĩa, còn nhánh Sparse Retrieval trả về danh sách dựa trên mức độ khớp từ khóa. Hai danh sách này sau đó được hợp nhất để tạo ra một tập ứng cử viên có độ bao phủ rộng, đảm bảo không bỏ sót thông tin quan trọng nào dù là về ý nghĩa hay thông số kỹ thuật. Cuối cùng, hệ thống thực hiện Reranking trên tập hợp ứng cử viên bằng cơ chế Late Interaction, tính toán lại điểm số dựa trên tương tác giữa các token để lọc ra top-K đoạn văn bản chất lượng nhất. Kết quả là ngữ cảnh được đưa vào augmented prompt vừa làm giàu ngữ nghĩa, vừa đảm bảo tính chính xác của các chi tiết kỹ thuật, giúp giảm thiểu rủi ro ảo giác và nâng cao độ tin cậy cho câu trả lời của AI Agent.



**Hình 5.2:** Chiến thuật RAG sử dụng Hybrid Search kết hợp Reranking

### 5.2.3 Kết quả đạt được

Việc tích hợp chiến lược Hybrid Search với Reranking đã mang lại bước tiến đáng kể về chất lượng truy xuất đối với dữ liệu đặc thù ngành an toàn thông tin.

Khắc phục hạn chế của phương pháp dense-only truyền thống, cơ chế này giảm thiểu hiệu quả tình trạng truy xuất các kết quả tương đồng ngữ nghĩa nhưng thiếu độ chính xác kỹ thuật. Thông qua việc bổ sung nhánh sparse retrieval (BM25), hệ thống đảm bảo các định danh quan trọng như mã CVE, phiên bản phần mềm, tham số dòng lệnh hay payload luôn được bảo toàn trong tập ứng cử viên. Cách tiếp cận này giúp loại bỏ rủi ro bỏ sót dữ liệu, đồng thời cải thiện độ bao phủ cho các truy vấn chứa nhiều thông tin kỹ thuật.

Tại tầng xử lý tiếp theo, kỹ thuật tái xếp hạng dựa trên cơ chế Late Interaction đóng vai trò là bộ lọc tinh chỉnh, sắp xếp lại tập ứng viên dựa trên mức độ tương thích ở cấp độ token. Kết quả thực tế cho thấy rằng các đoạn văn bản khớp chính xác với ý định truy vấn được ưu tiên xếp hạng cao hơn trong tập ứng viên. Ngược lại, các kết quả chỉ mang tính liên quan về chủ đề nhưng sai lệch về phiên bản hoặc thiếu chi tiết sẽ bị giảm thứ hạng. Quy trình này cung cấp cho augmented prompt một ngữ cảnh cô đọng, giàu chứng cứ xác thực, từ đó giảm thiểu hiện tượng hallucination vốn thấy của mô hình ngôn ngữ và nâng cao tính khả thi của các kịch bản tấn công trong môi trường thực tế.

### 5.3 Kiến trúc bộ nhớ dài hạn theo hướng MemGPT

#### 5.3.1 Đặt vấn đề

Về bản chất, kiểm thử xâm nhập là một quy trình nghiệp vụ phức tạp và kéo dài, trong đó dữ liệu đầu ra của bước trước là tiền đề để quyết định chiến lược tiếp theo. Tuy nhiên, việc ứng dụng LLM vào quy trình này vấp phải một thách thức lớn khi mà khối lượng thông tin tích lũy ngày một lớn trong khi cửa sổ ngữ cảnh lại có giới hạn, điều này là lí do trực tiếp dẫn đến hiện tượng tràn ngữ cảnh (context overflow). Hiện tượng tràn ngữ cảnh làm suy giảm khả năng nhận thức tình huống của Agent, dẫn đến các hành động sai lệch như lặp lại thao tác hoặc bỏ sót dữ kiện trọng yếu. Để giải quyết triệt để vấn đề này, đồ án đề xuất kiến trúc bộ nhớ dài hạn (Long-term Memory), cho phép lưu trữ có cấu trúc và truy xuất tri thức theo ngữ cảnh. Giải pháp này đảm bảo Agent duy trì được mạch suy luận (chain of thought) xuyên suốt các phiên kiểm thử mà không bị rào cản bởi giới hạn token.

#### 5.3.2 Giải pháp đề xuất

MemGPT [29] được định nghĩa như một “hệ điều hành” dành cho các mô hình ngôn ngữ lớn, giải quyết triệt để bài toán giới hạn cửa sổ ngữ cảnh thông qua kỹ thuật “Virtual Context Management”. Tương tự cơ chế phân trang (paging) giữa RAM và ổ cứng trong máy tính truyền thống, MemGPT kiến tạo một cấu trúc bộ nhớ phân tầng rõ rệt gồm Bộ nhớ chính (Main Context) và Bộ nhớ ngoài (External Context). Trong đó, Main Context nằm trực tiếp trong bộ nhớ ngữ cảnh của mô

hình, chứa các chỉ thị hệ thống (system instructions), vùng đọc/ghi các thông tin quan trọng (working context) và hàng đợi FIFO lưu lịch sử hội thoại gần nhất. Ngược lại, External Context đóng vai trò kho lưu trữ dài hạn, bao gồm Recall Storage (lưu toàn bộ lịch sử) và Archival Storage (cơ sở dữ liệu tri thức), cho phép hệ thống mở rộng khả năng ghi nhớ vượt xa giới hạn vật lý của mô hình.

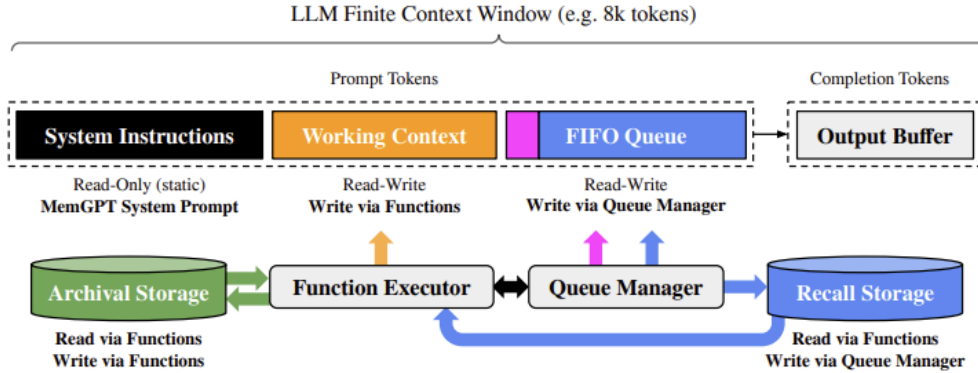


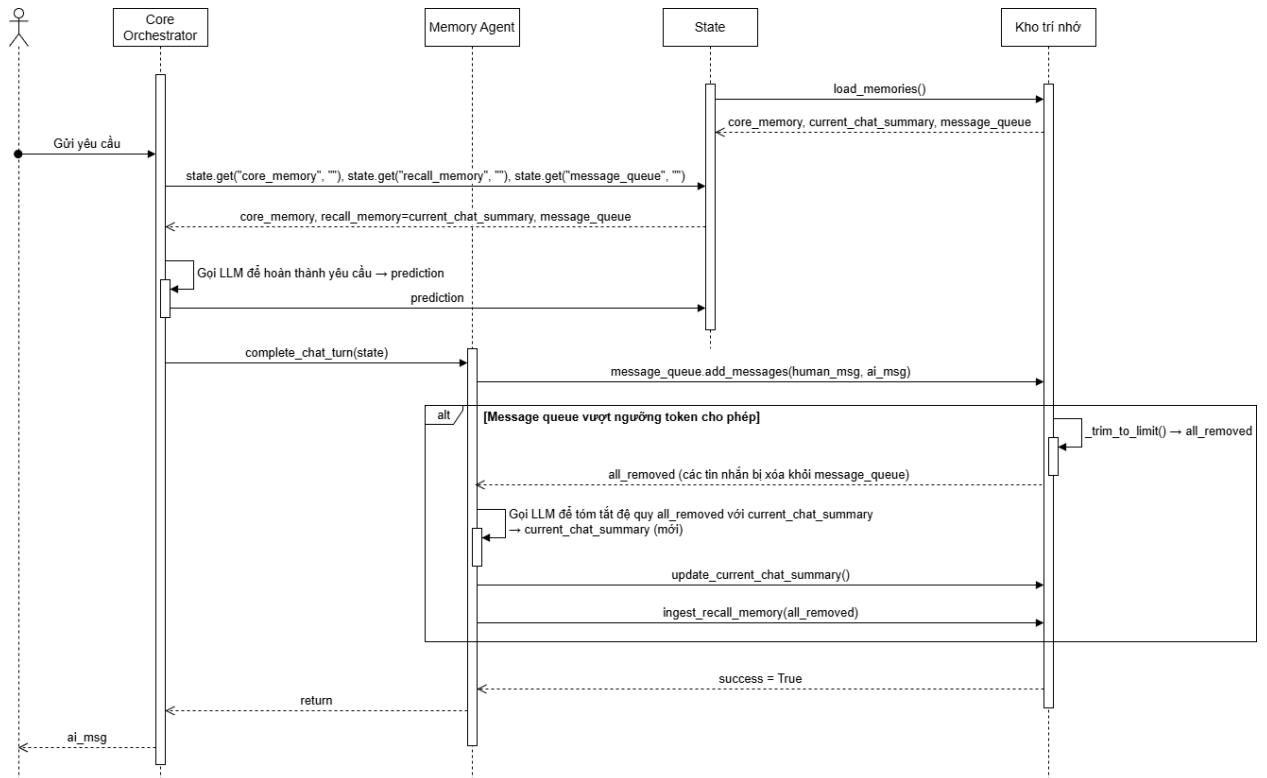
Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM’s prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

**Hình 5.3:** Chiến lược duy trì Long-term Memory của MemGPT [29]

Cơ chế vận hành của hệ thống dựa trên sự điều phối chặt chẽ giữa Queue Manager và Function Executor. Khi luồng thông tin nạp vào chạm ngưỡng cảnh báo, hệ thống kích hoạt tín hiệu “memory pressure”, buộc mô hình phải tự chủ động ra quyết định lưu trữ các thông tin quan trọng từ bộ nhớ đệm sang các vùng nhớ bền vững hoặc thực hiện tóm tắt đệ quy trước khi giải phóng không gian. Điểm đột phá của kiến trúc này nằm ở việc LLM không còn thụ động tiếp nhận dữ liệu mà trở thành một tác tử chủ động: nó sử dụng các lệnh gọi hàm (function call) để tự di chuyển dữ liệu ra/vào giữa các tầng bộ nhớ, và có khả năng thực hiện chuỗi suy luận liên tiếp (function chaining) để truy xuất dữ liệu trước khi đưa ra câu trả lời cuối cùng cho người dùng.

Nhờ khả năng quản lý bộ nhớ động này, MemGPT chuyển đổi mô hình ngôn ngữ thành một agent có bộ nhớ ảo gần như vô hạn. Khi đối mặt với các truy vấn liên quan đến dữ liệu quá khứ hoặc tài liệu có dung lượng lớn, hệ thống có thể chủ động tìm kiếm trong Recall/Archival Storage và nạp lại thông tin cần thiết vào Main Context để xử lý. Khả năng “phân trang” kết quả tìm kiếm và duy trì trạng thái qua các phiên làm việc dài cho phép MemGPT khắc phục hoàn toàn nhược điểm “quên” dữ liệu thường thấy ở LLM, đảm bảo tính liên tục và chiều sâu tri thức trong các tác vụ kiểm thử xâm nhập hoặc phân tích tài liệu phức tạp.

Dựa trên ý tưởng quản lý virtual context của MemGPT, đồ án áp dụng một kiến trúc bộ nhớ phân tầng nhằm tách bạch rõ phần ngữ cảnh in-context (nằm trong cửa sổ ngữ cảnh của mô hình) và phần tri thức out-of-context (lưu trữ ngoài để truy xuất khi cần). Cụ thể, hệ thống tổ chức bộ nhớ thành ba thành phần với ngân sách token cố định: core memory (500 token) dùng để lưu các tri thức bền vững của một phiên làm việc (mục tiêu, phát hiện đã xác nhận, các ràng buộc quan trọng); recall memory (256 token) là bản tóm tắt đệ quy của các thông điệp đã bị loại khỏi message queue; và message queue (1000 token) là hàng đợi FIFO lưu các thông điệp gần nhất. Tại mỗi lượt suy luận, Core Orchestrator sẽ tổng hợp dữ liệu từ cả core\_memory, recall\_memory, message\_queue cùng thông điệp hiện tại của người dùng để có cái nhìn tối thiểu nhưng đầy đủ về bức tranh tổng thể của phiên làm việc, từ đó tạo cơ sở cho việc ra quyết định và phản hồi.



**Hình 5.4:** Biểu đồ trình tự cho quá trình Nạp trí nhớ và xử lý hàng đợi thông điệp

Khi phát sinh thông tin cần lưu giữ lâu dài (bền vững theo phiên), Core Orchestrator gọi Memory Agent với tham số `operation=store` và `memory=<nội dung cần lưu>`. Tại đây, Memory Agent đọc lại các core memory hiện có và ra quyết định sử dụng công cụ `store_core_memory(memory, index)` để hoặc lưu core memory mới (`index=None`), hoặc cập nhật core memory hiện có. Trường hợp đặc biệt xảy ra khi tổng lượng token của core memory vượt quá ngưỡng 500 token, khi đó `store_core_memory` trả về lỗi giới hạn token, buộc Memory

Agent phải chuyển sang chiến lược cập nhật bằng cách chọn `index` ứng với `core memory` liên quan nhất với nội dung sắp thêm, nhằm hợp nhất thông tin và tránh làm phình `core memory`. Cơ chế này giúp `core memory` không chỉ cô đọng mà còn bảo toàn được các trí nhớ quan trọng.

Ngược lại, khi cần trích xuất tri thức từ quá khứ, `Core Orchestrator` sẽ gọi tới `Memory Agent` với `operation=search` và `memory=<ký ức/thông tin cần tìm>`. `Memory Agent` sẽ chuẩn hóa yêu cầu thành một truy vấn ngắn gọn và tối ưu cho RAG (ưu tiên giữ các thông tin quan trọng như IP/port, CVE, credentials, tên dịch vụ, lỗi,...), sau đó dùng công cụ `search_memory(query)` để tìm kiếm trên collection `recall_memory` trong cơ sở dữ liệu véc-tơ. Kết quả trả về được `Memory Agent` chất lọc để loại bỏ các thông tin dư thừa nhằm cung cấp các dữ kiện phù hợp nhất cho `Core Orchestrator`. Nhờ vậy, hệ thống có thể phục hồi tri thức cần thiết mà không phải mang toàn bộ lịch sử hội thoại vào cửa sổ ngữ cảnh.

Sau mỗi lượt hội thoại, cặp câu hỏi-trả lời được đẩy vào cuối `message queue`. Ngay sau đó, hệ thống tự động kích hoạt hàm `_trim_to_limit()` để kiểm soát cửa sổ ngữ cảnh: nếu `message queue` vượt quá 1000 token, hệ thống sẽ loại bỏ các lượt hội thoại cũ (chỉ giữ lại lượt vừa thêm), đồng thời kích hoạt một LLM nhỏ để tóm tắt và hợp nhất một cách đệ quy các tin nhắn bị loại bỏ với `recall memory` (tương ứng trong biểu đồ trình tự là `current_chat_summary`) hiện tại, từ đó cập nhật lại `current_chat_summary`. Cơ chế này giúp duy trì tính liên tục của phiên làm việc, bảo toàn các tín hiệu quan trọng trong một ngân sách token nhỏ, đồng thời giữ cho `Core Orchestrator` luôn vận hành trong giới hạn ngữ cảnh ổn định.

### 5.3.3 Kết quả đạt được

Bằng cách vận dụng cơ chế quản lý bộ nhớ của `MemGPT`, đồ án đã khắc phục được vấn đề phân mảnh ngữ cảnh trong các chiến dịch pentest dài hơi. Kiến trúc này cho phép duy trì mạch xử lý liên mạch ngay cả khi khối lượng dữ liệu log và lịch sử tương tác vượt xa giới hạn tiếp nhận của cửa sổ ngữ cảnh. Thông qua việc phân tầng bộ nhớ thành `core memory`, `recall memory` và `message queue`, hệ thống ngăn chặn hiệu quả hiện tượng bỏ quên dữ liệu khi chuyển đổi giữa các pha tấn công. Các tri thức vận hành trọng yếu, từ phạm vi kiểm thử, định danh mục tiêu, hiện trạng cổng dịch vụ đến thông tin xác thực và kết quả khai thác, đều được neo giữ một cách có cấu trúc. Điều này cho phép mô hình đưa ra quyết định dựa trên bức tranh toàn cảnh về trạng thái hệ thống thay vì bị giới hạn trong ngữ cảnh cục bộ ngắn hạn.

Trong thực tế vận hành, cơ chế này giúp tối ưu hóa hiệu suất bằng cách loại bỏ các thao tác thừa và giảm thiểu tình trạng lặp lại quy trình. Khi người dùng truy vấn lại một phiên cũ hoặc tiếp tục mạch làm việc, hệ thống có khả năng truy xuất các dữ kiện đã xác thực từ bộ nhớ. Khả năng này giúp tái sử dụng kết quả quét, tránh thực thi lại các véc-tơ tấn công đã thất bại và duy trì mạch tư duy liền mạch. Song song đó, quy trình tự động lược bỏ hàng đợi tin nhắn và hợp nhất vào recall memory giúp bảo lưu trí nhớ một cách cô đọng, đảm bảo agent luôn nắm bắt đúng trạng thái hiện tại ngay cả trong các cuộc hội thoại kéo dài.

Tóm lại, kiến trúc này nâng cao tính nhất quán và logic của toàn bộ quá trình kiểm thử. Nó đảm bảo rằng các hành động tiếp theo luôn được xây dựng dựa trên những gì đã đạt được, từ đó tạo ra một quy trình làm việc liền mạch. Đây là yếu tố then chốt biến một chatbot thông thường thành một trợ lý pentest thực thụ, có khả năng xử lý các kịch bản kiểm thử phức tạp kéo dài qua nhiều phiên làm việc.

### **5.4 Xây dựng Kali MCP Server hỗ trợ phương thức MCP HTTP streamable**

#### **5.4.1 Đặt vấn đề**

Trong bài toán kiểm thử xâm nhập, mô hình ngôn ngữ lớn nếu không được trang bị công cụ hành động sẽ chỉ dừng lại ở mức tư vấn và suy luận, không thể trực tiếp tham gia vào các quy trình pentest thực tế. Việc thiếu khả năng hành động này làm hạn chế đáng kể giá trị ứng dụng của LLM trong các kịch bản an toàn thông tin mang tính thực tiễn.

Một cách tiếp cận phổ biến là cho phép LLM sử dụng các công cụ thông qua cơ chế gọi hàm hoặc qua phương thức MCP stdio. Cách làm này tuy đạt được hiệu năng cao và độ trễ thấp, nhưng việc thực thi các công cụ pentest trong cùng môi trường với tiến trình chạy LLM tiềm ẩn nhiều rủi ro bảo mật. Bên cạnh đó, mô hình triển khai cục bộ này làm giảm tính linh hoạt của kiến trúc, khi MCP Server chỉ có thể hoạt động nội bộ trên máy cá nhân, khó mở rộng theo mô hình tập trung hoặc phục vụ đồng thời nhiều client trong các kịch bản triển khai thực tế.

#### **5.4.2 Giải pháp đề xuất**

Đề án đã thực hiện xây dựng một Kali MCP Server đơn giản nhằm cung cấp bộ công cụ pentest chuẩn hóa cho LLM thông qua giao thức MCP. Giải pháp này giúp mô hình ngôn ngữ không chỉ dừng lại ở việc suy luận mà còn có khả năng thực thi các tác vụ kỹ thuật. Các công cụ như Nmap, Metasploit và tập lệnh Kali Linux được đóng gói thành các “MCP tool” với giao diện thống nhất, tạo điều kiện để Pentest Agent dễ dàng điều phối, phân tích và tự động hóa quy trình kiểm thử xâm nhập.



Về kiến trúc, hệ thống triển khai MCP Server như một dịch vụ mạng độc lập sử dụng giao thức HTTP streamable (cụ thể về giao thức đã được trình bày ở mục 3.8.1). Thông qua FastMCP và uvicorn, mô hình này cho phép một server trung tâm phục vụ đồng thời nhiều client. Cơ chế streaming đặc biệt hữu ích cho các tác vụ pentest dài hơi, giúp agent nhận kết quả theo thời gian thực để ra quyết định tiếp theo ngay lập tức mà không cần chờ tác vụ hoàn tất.

Để đảm bảo an toàn, toàn bộ môi trường thực thi được đóng gói trong Docker container, tách biệt với hệ thống chạy LLM. Việc cô lập này giúp giảm thiểu rủi ro bảo mật, ngăn chặn các công cụ pentest tác động ngược lên hệ thống chính, đồng thời tăng tính linh hoạt khi triển khai. Ngoài ra, MCP Server cũng được tích hợp cơ chế xác thực qua token để bước đầu cung cấp khả năng kiểm soát quyền truy cập, ngăn chặn việc kẻ xấu lợi dụng hệ thống thực hiện các cuộc tấn công trái phép.

### 5.4.3 Kết quả đạt được

Kết quả triển khai cho thấy mô hình ngôn ngữ lớn đã có khả năng sử dụng các công cụ pentest thông qua giao thức MCP, đánh dấu bước chuyển quan trọng từ vai trò tư vấn thụ động sang thực thi kỹ thuật chủ động. Toàn bộ các công cụ pentest được cô lập trong môi trường Docker sandbox, giúp đảm bảo an toàn cho hệ thống chính và giảm thiểu rủi ro bảo mật trong quá trình thực thi. Đồng thời, việc áp dụng kiến trúc client-server kết hợp MCP HTTP streamable cho phép Kali MCP Server phục vụ đồng thời nhiều client với khả năng streaming kết quả theo thời gian thực, đáp ứng hiệu quả các tác vụ pentest có thời gian chạy kéo dài.

### Kết chương

Tóm lại, chương này đã hệ thống hóa toàn diện các đóng góp kỹ thuật nổi bật trong quá trình xây dựng hệ thống LPAA. Các thành quả nghiên cứu bao gồm: (i) Thiết lập quy trình benchmark để xác định mô hình nhúng mã nguồn mở tối ưu cho miền dữ liệu an toàn thông tin; (ii) Xây dựng chiến lược Hybrid Search kết hợp Reranking, giúp nâng cao đáng kể độ chính xác và độ bao phủ của hệ thống RAG; (iii) Phát triển kiến trúc bộ nhớ dài hạn dựa trên nguyên lý MemGPT; và (iv) Xây dựng một Kali MCP Server đơn giản cung cấp bộ công cụ pentest cho LLM.

## CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

### 6.1 Kết luận

Đồ án đã hoàn thành xây dựng một hệ thống trợ lý pentest chạy cục bộ theo hướng đa tác tử, có khả năng điều phối công cụ và duy trì tri thức vận hành qua nhiều phiên làm việc. So với các công cụ chatbot thông thường, hệ thống trong đồ án đi xa hơn ở khả năng hành động (tool execution), truy xuất tri thức (RAG tối ưu cho dữ liệu thuộc lĩnh vực an toàn thông tin), và duy trì bộ nhớ dài hạn (long-term memory).

Trong quá trình thực hiện, đồ án đã hoàn thiện các thành phần cụ thể như sau: xây dựng kiến trúc multi-agent với cơ chế điều phối subagents; tích hợp RAG sử dụng hybrid search kết hợp reranking nhằm cải thiện khả năng truy xuất của hệ thống; và kiến trúc bộ nhớ dài hạn theo hướng MemGPT để ghi nhận thông tin và duy trì bộ nhớ xuyên suốt phiên làm việc. Ngoài ra, hệ thống còn tích hợp thiết kế MCP server chạy cách ly trong Docker sandbox nhằm đảm bảo tính an toàn khi thực thi công cụ. Các thành phần này liên kết chặt chẽ với nhau, tạo thành một hệ thống có thể vận hành xuyên suốt quá trình pentest thay vì chỉ trả lời các câu hỏi ngắn hạn như các mẫu chatbot truyền thống.

Bên cạnh các kết quả đạt được, đồ án vẫn tồn tại bốn hạn chế cần khắc phục. Thứ nhất, chiến lược RAG hiện tại chỉ tập trung vào văn bản, chưa hỗ trợ xử lý thông tin từ hình ảnh, một nguồn dữ liệu quan trọng trong tài liệu kỹ thuật. Thứ hai, cơ chế bảo mật của MCP Server còn đơn giản, chỉ sử dụng một token xác thực duy nhất (static token) cho mọi phiên, chưa đảm bảo tính biệt lập cao. Thứ ba, khả năng tự chủ của Agent còn hạn chế, chưa thể tự động lập kế hoạch và thực thi pentest trọn vẹn mà vẫn cần sự điều phối từng bước của người dùng. Cuối cùng, về hiệu năng vận hành theo kiến trúc client-server, hệ thống cần được kiểm thử rộng rãi hơn để đánh giá và tinh chỉnh nhằm đạt được hiệu suất tối ưu trong thực tế.

Xuyên suốt quá trình nghiên cứu và phát triển hệ thống, tôi đã đúc kết được nhiều bài học quý giá. Nhờ những bài học này, tôi đã có cái nhìn khác về các hệ thống AI hiện nay, cụ thể: sức mạnh của hệ thống AI Agent không nằm ở việc sở hữu mô hình ngôn ngữ tối tân nhất, mà phụ thuộc tiên quyết vào tư duy kiến trúc tổng thể. Yếu tố cốt lõi để xây dựng hệ thống thành công chính là khả năng điều phối luồng nghiệp vụ chặt chẽ, chiến lược cung cấp tri thức (RAG), khả năng duy trì ngữ cảnh liên tục và trang bị các công cụ thực thi phù hợp. Thực nghiệm đã chứng minh, chiến lược phân rã bài toán phức tạp cho các agent chuyên trách là giải pháp tối ưu, không chỉ giúp giảm thiểu triệt để hiện tượng ảo giác (hallucination)

mà còn đảm bảo từng tác vụ được xử lý chính xác và hiệu quả hơn.

## 6.2 Hướng phát triển

Các hướng phát triển tiếp theo, bên cạnh việc khắc phục những điểm yếu hiện hữu, hệ thống sẽ tập trung vào ba mục tiêu sau: tăng độ chính xác khi xử lý, mở rộng tri thức và nâng cao khả năng hoạt động độc lập. Trước hết, hệ thống có thể cải thiện đáng kể hiệu suất làm việc nếu bộ xử lý trung tâm, là mô hình ngôn ngữ lớn, được tinh chỉnh (fine-tune) chuyên sâu cho lĩnh vực an toàn thông tin. Việc này sẽ không chỉ giúp LLM hiểu rõ hơn các kỹ thuật tấn công mà còn nâng cao khả năng suy luận trong các môi trường pentest phức tạp. Nhờ việc làm này, hệ thống sẽ giảm thiểu sai sót khi phân tích lỗ hổng hay đọc kết quả từ công cụ, đồng thời nâng cao chất lượng của các kịch bản tấn công mà nó đề xuất.

Thứ hai, hệ thống RAG cần được nâng cấp để xử lý cả hình ảnh. Trên thực tế, nhiều kiến thức bảo mật quan trọng thường nằm trong các ảnh chụp màn hình, sơ đồ hoặc tài liệu quét (scan). Vì vậy, hệ thống cần có khả năng đọc và lấy nội dung từ các hình ảnh này để lưu trữ cùng với văn bản. Điều này giúp việc tìm kiếm thông tin trở nên đầy đủ hơn, đồng thời hỗ trợ người dùng tốt hơn với nhiều định dạng tài liệu khác nhau. Ngoài ra, để nâng cao hiệu suất của quá trình truy xuất, hệ thống cần áp dụng thông minh các chiến lược tiền xử lý và truy xuất dữ liệu sao cho với mỗi miền dữ liệu khác nhau sẽ có phương thức làm việc tối ưu riêng. Điều này giúp cải thiện tình trạng áp dụng các chiến lược xử lý dữ liệu quá tốn kém vào các loại tài liệu vốn đã được chuẩn hóa và xử lý tốt.

Thứ ba, cần bổ sung thêm các công cụ kiểm thử và hoàn thiện môi trường sandbox. Mục tiêu là để hệ thống xử lý được mọi giai đoạn, từ dò quét thông tin đến khai thác lỗi một cách an toàn và thống nhất. Các công cụ mới cần được mô tả cách dùng rõ ràng và có cơ chế ghi lại nhật ký hoạt động. Việc này giúp giảm rủi ro khi vận hành và đảm bảo kết quả kiểm thử luôn đạt độ chính xác cao. Bên cạnh đó, để tăng khả năng hoạt động độc lập, hệ thống có thể phát triển theo hướng xây dựng các kịch bản làm việc cụ thể (playbook). Playbook giúp chuẩn hóa chuỗi hành động theo từng mục tiêu và từng dịch vụ, xác định điều kiện chuyển tiếp và cơ chế xử lý lỗi cho từng dịch vụ cụ thể. Khi kết hợp với kiến trúc multi-agent và bộ nhớ dài hạn, playbook sẽ giúp hệ thống vận hành nhất quán hơn, giảm phụ thuộc vào chỉ dẫn chi tiết của người dùng và tiến gần hơn tới khả năng của một công cụ pentest hoàn toàn tự động.

## TÀI LIỆU THAM KHẢO

- [1] GH05STCREW, *Pentestagent*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/GH05TCREW/pentestagent>.
- [2] 0x4m4, *Hexstrike ai mcp agents v6.0*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/0x4m4/hexstrike-ai>.
- [3] usestrix, *Strix*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/usestrix/strix>.
- [4] langchain-ai, *Langchain*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/langchain-ai/langchain>.
- [5] Dave Bergmann, Cole Stryker, *What is langchain?* Accessed: Dec. 26, 2025. [Online]. Available: <https://www.ibm.com/think/topics/langchain>.
- [6] deepset-ai, *Haystack*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/deepset-ai/haystack>.
- [7] run-llama, *Llamaindex*. Accessed: Dec. 26, 2025. [Online]. Available: [https://github.com/run-llama/llama\\_index](https://github.com/run-llama/llama_index).
- [8] langchain-ai, *Langgraph*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/langchain-ai/langgraph>.
- [9] Significant-Gravitas, *Autogpt: Build, deploy, and run ai agents*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/Significant-Gravitas/AutoGPT>.
- [10] vllm-project, *Vllm - easy, fast, and cheap llm serving for everyone*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/vllm-project/vllm>.
- [11] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, Jan. 2025, ISSN: 1046-8188. DOI: 10.1145/3703155. [Online]. Available: <https://doi.org/10.1145/3703155>.
- [12] docling-project, *Docling*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/docling-project/docling>.
- [13] langchain-community, *Pdfminerloader*. Accessed: Dec. 26, 2025. [Online]. Available: [https://docs.langchain.com/oss/python/integrations/document\\_loaders/pdfminer](https://docs.langchain.com/oss/python/integrations/document_loaders/pdfminer).
- [14] apache, *Welcome to apache tika https://tika.apache.org/*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/apache/tika>.

- [15] qdrant, *Qdrant - vector search engine for the next generation of ai applications*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/qdrant/qdrant>.
- [16] Qdrant, *What is qdrant?* Accessed: Dec. 26, 2025. [Online]. Available: <https://qdrant.tech/documentation/overview/what-is-qdrant>.
- [17] qdrant, *What is fastembed?* Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/qdrant/fastembed>.
- [18] facebookresearch, *Faiss*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/facebookresearch/faiss>.
- [19] chroma-core, *Chroma - the open-source embedding database*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/chroma-core/chroma>.
- [20] milvus-io, *What is milvus?* Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/milvus-io/milvus>.
- [21] modelcontextprotocol, *Model context protocol (mcp)*. Accessed: Dec. 26, 2025. [Online]. Available: <https://github.com/modelcontextprotocol/modelcontextprotocol/>.
- [22] modelcontextprotocol, *Architecture overview*. Accessed: Dec. 26, 2025. [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>.
- [23] LangChain Docs, *Multi-agent*. Accessed: Dec. 26, 2025. [Online]. Available: <https://docs.langchain.com/oss/python/langchain/multi-agent>.
- [24] LangChain Docs, *Subagents*. Accessed: Dec. 26, 2025. [Online]. Available: <https://docs.langchain.com/oss/python/langchain/multi-agent/subagents>.
- [25] openai, *Introducing gpt-oss*. Accessed: Dec. 26, 2025. [Online]. Available: <https://openai.com/index/introducing-gpt-oss/>.
- [26] Tannu Barot, Savani Dhruv, Maispace, *Cybersecurity attack dataset*. Accessed: Dec. 26, 2025. [Online]. Available: <https://www.kaggle.com/datasets/tannubarot/cybersecurity-attack-and-defence-dataset>.
- [27] Leonie Monigatti, Danny Williams, Victoria Slocum, *An overview of late interaction retrieval models: Colbert, colpali, and colqwen*. Accessed: Dec. 26, 2025. [Online]. Available: <https://weaviate.io/blog/late-interaction-overview>.

- [28] Erika Shorten, *Hybrid search explained*. Accessed: Dec. 26, 2025. [Online]. Available: <https://weaviate.io/blog/hybrid-search-explained>.
- [29] C. Packer et al., “MemGPT: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.

# PHỤ LỤC